

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ VIỄN THÔNG

**ĐỀ TÀI: MÔ HÌNH IOT BÁN LẺ LINH KIỆN ĐIỆN TỬ
THÔNG MINH TÍCH HỢP HYBRID RAG VÀ MULTI-
AGENT AI**

GVHD: PGS.TS. Phạm Ngọc Sơn

Sinh viên: Nguyễn Ngọc Bảo Trân

MSSV: 22161326

Nguyễn Thị Ngọc Hải

MSSV: 22161249

TP. HỒ CHÍ MINH - 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ VIỄN THÔNG

**ĐỀ TÀI: MÔ HÌNH IOT BÁN LẺ LINH KIỆN ĐIỆN TỬ THÔNG
MINH TÍCH HỢP HYBRID RAG VÀ MULTI-AGENT AI**

GVHD: PGS.TS. Phạm Ngọc Sơn

Sinh viên: Nguyễn Ngọc Bảo Trân

MSSV: 22161326

Nguyễn Thị Ngọc Hải

MSSV: 22161249

TP. HỒ CHÍ MINH - 06/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Mô hình IoT bán lẻ linh kiện điện tử thông minh tích hợp Hybrid RAG và Multi-Agent AI

Sinh viên: + Nguyễn Ngọc Bảo Trân

MSSV: 22161326

+ Nguyễn Thị Ngọc Hải

MSSV: 22161249

Hướng dẫn: PGS.TS. Phạm Ngọc Sơn

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đề tài được hình thành từ những vấn đề thực tế trong hoạt động kinh doanh linh kiện điện tử, như việc quản lý nhiều loại sản phẩm, cập nhật tồn kho và đơn hàng theo thời gian thực, cũng như hỗ trợ người dùng lựa chọn linh kiện phù hợp. Cách đặt vấn đề có cơ sở, mục tiêu nghiên cứu được xác định rõ và bám sát nhu cầu ứng dụng công nghệ trong bán lẻ hiện nay. Hướng giải quyết thông qua việc kết hợp hệ thống Web, cơ sở dữ liệu thời gian thực, AI Backend, Hybrid RAG, Multi-Agent và Smart Locker là phù hợp với mục tiêu đề tài. Giải pháp không chỉ hỗ trợ quản lý bán hàng và kho linh kiện mà còn nâng cao khả năng tư vấn kỹ thuật và tự động hóa quy trình nhận hàng. Vì vậy, đề tài có ý nghĩa thực tiễn rõ ràng, đồng thời thể hiện khả năng vận dụng các công nghệ AIoT vào một mô hình ứng dụng cụ thể.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp thực hiện của đề tài được xây dựng theo trình tự tương đối hợp lý, bắt đầu từ việc nghiên cứu các nền tảng liên quan như IoT, phát triển Web, Firebase Realtime Database, mô hình RAG, MultiAgent và cơ sở dữ liệu vector. Trên cơ sở đó, nhóm sinh viên tiến hành phân tích yêu cầu, thiết kế kiến trúc tổng thể và triển khai từng thành phần, gồm giao diện Web bằng ReactJS, cơ sở dữ liệu Firebase, AI Backend sử dụng FastAPI và mô hình Smart Locker dùng ESP32. Việc phân chia thành các module riêng biệt giúp quá trình phát triển, kiểm thử và đánh giá được thực hiện rõ ràng hơn.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Kết quả thực hiện phù hợp với mục tiêu đề tài. Hệ thống đã hoàn thiện các chức năng chính của một mô hình bán lẻ linh kiện điện tử thông minh, bao gồm quản lý sản phẩm, giỏ hàng, đơn hàng, tồn kho, đồng bộ dữ liệu thời gian thực, gửi thông báo qua Email/ SMS, hỗ trợ tư vấn bằng AI và nhận hàng thông qua Smart Locker.

4. Kết luận và đề xuất:

Kết luận của báo cáo phù hợp với mục tiêu ban đầu: mô hình chứng minh khả năng tích hợp Web, AI và IoT trong quản lý bán lẻ linh kiện điện tử.

Các đề xuất phát triển như tối ưu định tuyến Agent, mở rộng cơ sở tri thức, nâng cấp Smart Locker, bổ sung QR/Rfid/camera, phân quyền chi tiết và phân tích dữ liệu kinh doanh là hợp lý, có tính cải tiến và gắn với khả năng triển khai thực tế.

5. Hình thức trình bày và bố cục báo cáo:

Báo cáo có bố cục tương đối rõ ràng theo các phần: tổng quan, cơ sở lý thuyết, thiết kế - triển khai, kết quả đánh giá, kết luận và hướng phát triển. Nội dung có nhiều hình ảnh giao diện, sơ đồ luồng xử lý, bảng kiểm thử và kết quả thực nghiệm, giúp minh họa quá trình thực hiện.

Tuy nhiên, nhóm sinh viên cần rà soát lỗi chính tả, thống nhất thuật ngữ, tinh gọn phần lý thuyết và bổ sung trích dẫn trước khi nộp bản chính thức.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Nhóm sinh viên thể hiện khả năng phối hợp nhiều mảng kỹ thuật khác nhau gồm Web, cơ sở dữ liệu realtime, AI Backend, truy xuất tri thức và phần cứng IoT. Khối lượng triển khai cho thấy tinh thần chủ động tìm hiểu công nghệ mới, khả năng làm việc nhóm và năng lực tích hợp hệ thống. Tính sáng tạo thể hiện ở việc đưa Multi-Agent và Hybrid RAG vào bài toán tư vấn linh kiện, đồng thời kết nối với Smart Locker để hoàn thiện quy trình mua - nhận hàng.

7. Tài liệu trích dẫn

Đồ án có trích dẫn tài liệu tham khảo. Các tài liệu tham khảo nhìn chung phù hợp với các công nghệ sử dụng trong đề tài như ESP32, Firebase, ReactJS, FastAPI, AI, RAG, Qdrant, LangGraph và các mô hình ngôn ngữ lớn.

8. Đánh giá về sự trùng lặp của đề tài

Chưa phát hiện trùng lặp với các đề tài trong nước.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

- Phần cơ sở lý thuyết còn khá dài so với trọng tâm đề án; nên tinh gọn các nội dung chung, tăng nhấn mạnh vào phần thiết kế, kết quả thực nghiệm và phân tích nguyên nhân sai số/hạn chế.
- Hệ thống mới được kiểm thử ở mức mô hình.
- Bổ sung tài liệu tham khảo về các công nghệ chính trong đề án.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

Nhóm sinh viên có tinh thần nghiêm túc trong quá trình thực hiện, thể hiện qua việc hoàn thành đồng thời nhiều hạng mục: Web quản lý, cơ sở dữ liệu realtime, AI Assistant, đánh giá thuật toán và phần cứng Smart Locker.

Nhóm có ý thức tiếp thu, chỉnh sửa và hoàn thiện sản phẩm theo mục tiêu đề tài. Cần tiếp tục rèn luyện kỹ năng trình bày báo cáo ngắn gọn, tập trung vào kết quả và minh chứng thực nghiệm.

Đề nghị của giảng viên hướng dẫn

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 24 tháng 06 năm 2026

Người nhận xét (Ký và ghi rõ họ tên)



PGS.TS. Phạm Ngọc Sơn

LỜI CẢM ƠN

Trong suốt quá trình học tập, nghiên cứu và thực hiện đồ án tốt nghiệp, chúng em đã nhận được sự quan tâm, hướng dẫn và giúp đỡ quý báu từ quý thầy cô, gia đình và bạn bè.

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc đến PGS.TS Phạm Ngọc Sơn, giảng viên hướng dẫn đã tận tình chỉ bảo, định hướng và hỗ trợ chúng em trong suốt quá trình thực hiện đề tài. Những ý kiến đóng góp chuyên môn, sự động viên và những kinh nghiệm quý báu của thầy là nguồn động lực quan trọng giúp chúng em hoàn thành đồ án này.

Chúng em xin chân thành cảm ơn quý thầy cô Trường Đại học Công Nghệ Kỹ thuật Thành phố Hồ Chí Minh đã tận tình giảng dạy, truyền đạt kiến thức và tạo điều kiện thuận lợi cho chúng em trong suốt quá trình học tập và nghiên cứu.

Chúng em cũng xin gửi lời cảm ơn đến gia đình, bạn bè và những người đã luôn động viên, hỗ trợ và tạo điều kiện tốt nhất để chúng em hoàn thành chương trình học cũng như đồ án tốt nghiệp.

Mặc dù đã cố gắng hoàn thiện đồ án với tất cả sự nỗ lực, song do hạn chế về thời gian, kinh nghiệm và kiến thức thực tế nên đồ án không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

TÓM TẮT

Đề tài “Mô hình IoT bán lẻ linh kiện điện tử thông minh tích hợp hybrid RAG và multi-agent AI” được thực hiện nhằm xây dựng một nền tảng bán lẻ linh kiện điện tử theo hướng thông minh, kết hợp giữa công nghệ Web, trí tuệ nhân tạo (AI) và internet vạn vật (IoT). Mô hình được thiết kế để hỗ trợ quản lý sản phẩm, đơn hàng, tồn kho, tư vấn kỹ thuật và tự động hóa quy trình nhận hàng.

Mô hình “IoT bán lẻ linh kiện điện tử thông minh tích hợp hybrid RAG và multi-agent AI” bao gồm các thành phần chính: Web Dashboard được phát triển bằng ReactJS dành cho người dùng và quản trị viên; Firebase Realtime Database đóng vai trò lưu trữ và đồng bộ dữ liệu thời gian thực; AI Backend được xây dựng trên nền tảng FastAPI, áp dụng kiến trúc Multi-Agent kết hợp Hybrid RAG nhằm hỗ trợ tư vấn kỹ thuật, đề xuất linh kiện và phân tích dữ liệu; Smart Locker sử dụng vi điều khiển ESP32 cho phép người dùng nhận hàng thông qua mã xác thực và đồng bộ trạng thái với mô hình trung tâm.

Trong AI Backend, mô hình sử dụng Workflow được xây dựng bằng LangGraph với các thành phần Router, Retrieval, Grader, Web Fallback và Generate nhằm nâng cao chất lượng truy xuất tri thức và khả năng phản hồi của mô hình ngôn ngữ lớn. Đồng thời, cơ chế Hybrid RAG kết hợp Priority Retrieval, BM25 Truy xuất và Qdrant Vector Truy xuất giúp tăng độ chính xác trong quá trình tìm kiếm thông tin.

Kết quả triển khai cho thấy mô hình hoạt động ổn định, đáp ứng đầy đủ các chức năng đặt ra như quản lý bán hàng, quản lý kho, tư vấn kỹ thuật bằng AI Agent và nhận hàng tự động qua Smart Locker. Đề tài góp phần minh chứng khả năng ứng dụng hiệu quả của AI, IoT và điện toán đám mây trong việc xây dựng các mô hình bán lẻ thông minh, đồng thời tạo tiền đề cho việc nghiên cứu và phát triển các mô hình AIoT Smart Retail trong tương lai.

Từ khóa: AIoT Smart Retail, AI Agent, Hybrid RAG, FastAPI, ReactJS, Firebase, Smart Locker, ESP32, LangGraph, Qdrant.

MỤC LỤC

DANH MỤC HÌNH	xii
DANH MỤC BẢNG.....	xvi
DANH MỤC CÁC TỪ VIẾT TẮT.....	xvii
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI.....	1
1.1. Lý do chọn đề tài	1
1.2. Mục tiêu đề tài	2
1.3. Đối tượng và phạm vi nghiên cứu	3
1.3.1. Đối tượng nghiên cứu	3
1.3.2. Phạm vi nghiên cứu	4
1.4. Phương pháp nghiên cứu	5
1.5. Bố cục đề tài	6
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	8
2.1. Vi điều khiển ESP32.....	8
2.2. Cơ sở dữ liệu thời gian thực Firebase	10
2.2.1. Firebase Realtime Database	10
2.2.2. Cơ chế đồng bộ dữ liệu thời gian thực.....	11
2.3. Công nghệ phát triển ứng dụng Web	12
2.3.1. Thư viện ReactJS	12
2.3.2. Framework FastAPI.....	13
2.4. Công nghệ gửi Email và SMS	15
2.4.1. Dịch vụ EmailJS	15
2.4.2. Dịch vụ eSMS.....	16
2.5. Trí tuệ nhân tạo và mô hình ngôn ngữ lớn.....	17
2.5.1. Mô hình ngôn ngữ lớn (LLM)	17

2.5.2. Kiến trúc Transformer	17
2.5.3. Kỹ thuật thiết kế Prompt (Prompt Engineering)	18
2.5.4. Nền tảng OpenRouter	19
2.6. Kỹ thuật Retrieval-Augmented Generation (RAG)	20
2.6.1. Nguyên lý hoạt động của RAG	21
2.6.2. Các thành phần chính của RAG	22
2.7. Kỹ thuật truy xuất thông tin trong Hybrid RAG	23
2.7.1. Truy xuất thông tin (Information Retrieval)	23
2.7.2. Thuật toán BM25 Retrieval	24
2.7.3. Truy xuất theo mô hình vector (Vector Retrieval)	26
2.7.4. Reranking	26
2.8. Cơ sở dữ liệu vector Qdrant	28
2.8.1. Khái niệm Vector Database	28
2.8.2. Embedding và biểu diễn dữ liệu bằng Vector	28
2.8.3. Kỹ thuật tìm kiếm tương đồng Similarity Search	29
2.8.4. Qdrant Vector Database	30
2.9. Mô hình Agent và Multi-Agent	31
2.9.1. Khái niệm Agent	31
2.9.2. Mô hình Multi-Agent	32
2.9.3. Agent điều phối (Supervisor Agent)	33
2.10. Framework LangGraph	35
CHƯƠNG 3: THIẾT KẾ VÀ TRIỂN KHAI MÔ HÌNH	38
3.1. Kiến trúc tổng quan mô hình	38
3.2. Thiết kế giao diện mô hình	39
3.3. Thiết kế cấu trúc Firebase Realtime Database	45

3.4. Kiến trúc AI Backend	48
3.4.1. Kiến trúc tổng thể AI Backend	48
3.4.2. Thiết kế LangGraph Workflow.....	50
3.4.3. Thiết kế Supervisor-Based Multi-Agent.....	53
3.4.4. Thiết kế Hybrid RAG	57
3.4.5. Thiết kế Knowledge Base và Qdrant	68
3.4.6. Thiết kế Truy xuất Grader và Web Fallback	72
3.4.7. Thiết kế OpenRouter LLM Service	76
3.4.8. Thiết kế FastAPI AI Backend	81
3.5. Kiến trúc tổng thể phần cứng.....	86
3.5.1. Thiết kế giải thuật tìm mã PIN trên Firebase.....	89
3.5.2. Thiết kế giải thuật xử lý mã nhập từ người dùng.....	91
3.6. Thiết kế dịch vụ thông báo Email và SMS	92
CHƯƠNG 4: KẾT QUẢ TRIỂN KHAI VÀ ĐÁNH GIÁ MÔ HÌNH.....	94
4.1. Kết quả triển khai Frontend ReactJS	94
4.1.1. Giao diện trang chủ User Home	94
4.1.2. Giao diện User Dashboard.....	95
4.1.3. Giao diện Admin Dashboard	104
4.2. Kết quả triển khai tương tác giữa frontend và firebase.....	110
4.3. Kết quả gửi thông báo về Email và SMS.....	118
4.4. Kết quả triển khai và đánh giá AI Backend	120
4.4.1. Kết quả kết nối Web Chatbot với AI Backend	120
4.4.2. Kết quả đọc dữ liệu từ Firebase Realtime Database cho phản hồi AI	122
4.4.3. Kết quả triển khai Qdrant Vector Database	124
4.4.4. Kết quả đánh giá khả năng định tuyến câu hỏi giữa các Agent.....	127

4.4.5. Kết quả đánh giá Truy xuất và Hybrid Search.....	140
4.4.6. Đánh giá khả năng giảm Hallucination bằng RAG	145
4.4.7. Kết quả đánh giá thời gian phản hồi AI.....	147
4.4.8. Đánh giá tổng thể mô hình AI Backend	148
4.5. Kết quả phân cứng smart locker	149
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	156
5.1. Kết luận.....	156
5.2. Hạn chế của đề tài.....	157
5.3. Hướng phát triển	157
TÀI LIỆU THAM KHẢO.....	160

DANH MỤC HÌNH

Hình 2.1. Module ESP32 DevKit	8
Hình 2.2. Sơ đồ chân của ESP32	9
Hình 2.3. Kiến trúc tổng quan nền tảng Firebase	10
Hình 2.4. Cấu trúc dữ liệu dạng JSON Tree trong Firebase Realtime Database	11
Hình 2.5. Quy trình hoạt động của ReactJS.....	13
Hình 2.6. Kiến trúc hoạt động của FastAPI.....	14
Hình 2.7. Mô hình giao tiếp Client - Server thông qua RESTful API.....	15
Hình 2.8. Kiến trúc LLM cơ bản	17
Hình 2.9. Kiến trúc Transformer	18
Hình 2.10. Mối quan hệ giữa Prompt và phản hồi của LLM.....	19
Hình 2.11. Kiến trúc hoạt động của OpenRouter	20
Hình 2.12. Kiến trúc tổng quát của Retrieval-Augmented Generation	21
Hình 2.13. Các thành phần chính của mô hình RAG	22
Hình 2.14. Kiến trúc tổng quát của Hybrid RAG	23
Hình 2.15. Quy trình tìm kiếm ngữ nghĩa bằng Vector Retrieval	26
Hình 2.16. Quy trình Reranking kết quả truy xuất	27
Hình 2.17. Mô hình Hybrid Truy xuất kết hợp BM25 và Vector Search	28
Hình 2.18. Nguyên lý tìm kiếm tương đồng bằng Cosine Similarity	30
Hình 2.19. Kiến trúc tổng quan của Qdrant.....	31
Hình 2.20. Mô hình hoạt động cơ bản của Agent.....	32
Hình 2.21. Kiến trúc tổng quát của Multi-Agent System.....	33
Hình 2.22. Kiến trúc Supervisor-Based Multi-Agent.....	34
Hình 2.23. Đồ thị Framework LangGraph.....	35
Hình 2.24. Mô hình State Graph trong LangGraph	36
Hình 3.1. Kiến trúc tổng quan mô hình	38
Hình 3.2. Cấu trúc thư mục thư mục pages của mô hình	40
Hình 3.3. Lưu đồ chức năng đăng nhập và phân quyền người dùng	41
Hình 3.4. Lưu đồ chức năng xử lý giỏ hàng	42
Hình 3.5. Luồng xử lý đơn hàng.....	44

Hình 3.6. Luồng hoạt động của Queue Display.....	45
Hình 3.7. Cấu trúc tổng thể Firebase Realtime Database	46
Hình 3.8. Cấu trúc dữ liệu node lockers	47
Hình 3.9. Cấu trúc dữ liệu node locker_access.....	47
Hình 3.10. Sơ đồ cấu trúc thư mục và kiến trúc mã nguồn mô hình AI.	48
Hình 3.11. Sơ đồ kiến trúc mô hình AI.	49
Hình 3.12. Workflow xử lý truy vấn bằng LangGraph	50
Hình 3.13. Kiến trúc Supervisor-Based Multi-Agent.....	53
Hình 3.14. Kiến trúc Hybrid RAG.....	57
Hình 3.15. Quy trình xây dựng Knowledge Base và Qdrant.....	68
Hình 3.16. Cơ chế Truy xuất Grader và Web Fallback	72
Hình 3.17. Luồng xử lý yêu cầu trong FastAPI AI Backend	81
Hình 3.18. Luồng xử lý User AI.....	84
Hình 3.19. Luồng xử lý Admin AI	84
Hình 3.20. Sơ đồ nguyên lý phần cứng	86
Hình 3.21. Lưu đồ luồng hoạt động của mô hình khóa tủ	87
Hình 3.22. Lưu đồ kiểm tra quyền mở tủ.....	89
Hình 3.23. Lưu đồ quá trình kiểm tra mã ID	91
Hình 4.1. Giao diện trang chủ User Home	94
Hình 4.2. Giao diện đăng nhập và xác thực người dùng của mô hình AIoT Retail	95
Hình 4.3. Giao diện hiển thị danh sách sản phẩm của mô hình AIoT Retail	95
Hình 4.4. Giao diện danh sách sản phẩm yêu thích của mô hình AIoT Retail.....	96
Hình 4.5. Giao diện giỏ hàng và thanh toán của mô hình AIoT Retail	96
Hình 4.6. Giao diện Project Hub của mô hình AIoT Retail	97
Hình 4.7. Giao diện trợ lý AI của mô hình AIoT Retail.....	98
Hình 4.8. Giao diện lịch sử giao dịch của mô hình AIoT Retail	98
Hình 4.9. Kết quả tìm kiếm linh kiện theo từ khóa trong mô hình AIoT Retail.....	99
Hình 4.10. Kết quả thêm sản phẩm vào giỏ hàng trong mô hình AIoT Retail	100
Hình 4.11. Kết quả cập nhật số lượng sản phẩm trong giỏ hàng.....	100
Hình 4.12. Kết quả tính toán tổng tiền tự động trong giỏ hàng.....	101
Hình 4.13. Kết quả đặt hàng và tạo mã nhận hàng trong mô hình AIoT Retail	102

Hình 4.14. Giao diện thanh toán và lựa chọn hình thức nhận hàng của mô hình AIoT Retail	103
Hình 4.15. Giao diện Dashboard quản lý mô hình AIoT Retail	104
Hình 4.16. Giao diện quản lý kho linh kiện của mô hình AIoT Retail.....	105
Hình 4.17. Giao diện theo dõi đơn hàng thời gian thực của mô hình AIoT Retail.....	106
Hình 4.18. Giao diện giám sát Smart Locker của mô hình AIoT Retail	107
Hình 4.19. Giao diện báo cáo và thống kê của mô hình AIoT Retail.....	107
Hình 4.20. Giao diện cài đặt của AIoT Retail	108
Hình 4.21. Kết quả cập nhật tồn kho linh kiện và thông báo realtime	108
Hình 4.22. Kết quả xuất báo cáo tồn kho ra tệp CSV.....	109
Hình 4.23. Kết quả tương tác giữa Frontend và Firebase Realtime Database.....	110
Hình 4.24. Kết quả đồng bộ dữ liệu tồn kho giữa Frontend và Firebase.....	111
Hình 4.25. Kết quả đồng bộ dữ liệu Queue Display giữa Frontend và Firebase Realtime Database.....	112
Hình 4.26. Kết quả đồng bộ lịch sử giao dịch giữa Frontend và Firebase	113
Hình 4.27. Kết quả đồng bộ giỏ hàng nháp giữa Frontend và Firebase	114
Hình 4.28. Kết quả đồng bộ mô hình Smart Locker giữa Frontend và Firebase.....	115
Hình 4.29. Kết quả xác nhận đơn hàng và cấp mã lấy hàng Smart Locker.....	116
Hình 4.30. Kết quả giám sát trạng thái Smart Locker realtime trên Frontend và Firebase	117
Hình 4.31. Kết quả triển khai mô hình gửi SMS thông báo khách hàng.....	118
Hình 4.32. Kết quả gửi Email xác nhận đơn hàng và đồng bộ Queue Realtime	119
Hình 4.33. Giao diện Web Chatbot kết nối AI Backend	121
Hình 4.34. Kết quả phản hồi của AI trên Web Chatbot.....	121
Hình 4.35. Kết quả AI truy xuất dữ liệu tồn kho từ Firebase Realtime Database	122
Hình 4.36. Collection <code>iot_knowledge</code> trong Qdrant	124
Hình 4.37. Dữ liệu vector embedding được lưu trong Qdrant.....	125
Hình 4.38. Kết quả truy xuất dữ liệu từ Qdrant.....	126
Hình 4.39. Kết quả đánh báo cáo đánh giá từ terminal	130
Hình 4.40. Biểu đồ tỷ lệ định tuyến đúng và sai của mô hình AI.....	130
Hình 4.41. Biểu đồ độ chính xác định tuyến theo từng Agent	131
Hình 4.42. Kết quả định tuyến consultant agent.....	132

Hình 4.43. Kết quả định tuyến inventory agent.....	134
Hình 4.44. Kết quả định tuyến combo agent	134
Hình 4.45. Kết quả Web Fallback	135
Hình 4.46. Kết quả của truy vấn “Smart locker hoạt động như thế nào?”.	136
Hình 4.47. Kết quả của truy vấn cảm biến siêu âm HC-SR04 đo được khoảng cách bao xa?”.	137
Hình 4.48. Kết quả của truy vấn mạch L298N dùng để điều khiển động cơ gì?”.	137
Hình 4.49. Kết quả của truy vấn “Set linh kiện lắp ráp cánh tay robot 4 bậc”	138
Hình 4.50. Kết quả của truy vấn “Gợi ý trộn bộ làm máy rót rượu tự động”	138
Hình 4.51. Kết quả của truy vấn “Kiểm tra xem module sim A7670C còn bao nhiêu cái trong kho?”	139
Hình 4.52. Kết quả của truy vấn “Thời tiết Hà Nội cuối tuần này có mưa không?”.	140
Hình 4.53. Kết quả của truy vấn “Kiểm tra hàng tồn của servo sg90”	140
Hình 4.54. Biểu đồ truy xuất Score trung bình theo từng Agent	141
Hình 4.55. Kết quả Truy xuất Grading trong AI Backend	141
Hình 4.56. Log Hybrid Truy xuất trong terminal backend.....	144
Hình 4.57. Response API của AI Backend.....	145
Hình 4.58. Phản hồi AI khi không sử dụng RAG.....	146
Hình 4.59. Phản hồi AI khi sử dụng RAG.....	146
Hình 4.60. Truy xuất log thể hiện source được truy xuất	146
Hình 4.61. Biểu đồ thời gian phản hồi trung bình theo từng Agent	148
Hình 4.62. Kết quả khởi động LCD và kết nối mô hình Smart Locker.....	149
Hình 4.63. Mô hình Smart Locker ở trạng thái chờ nhập ID.....	151
Hình 4.64. Người dùng thao tác nhập mã nhận hàng trên keypad.....	151
Hình 4.65. Kết quả xác thực mã nhận hàng và mở ngăn tủ tương ứng	152
Hình 4.66. Kết quả tự động đóng tủ sau thời gian timeout.....	153
Hình 4.67. Dữ liệu Firebase khi trạng thái tủ chuyển từ opened sang closed	154

DANH MỤC BẢNG

Bảng 3.1: Chức năng chi tiết của các file giao diện thuộc thư mục pages	40
Bảng 3.2. Các thành phần chính của phần cứng.....	88
Bảng 4.1: Các testcase dùng kiểm thử	127
Bảng 4.2. Kết quả đánh giá định tuyến Agent	129
Bảng 4.3. Tính chính xác theo từng Agent	131
Bảng 4.4. Kết quả Retrieval	141
Bảng 4.5. Truy xuất Score theo từng Agent	141
Bảng 4.6. Một số kết quả truy xuất tiêu biểu	143
Bảng 4.7. Kết quả đo thời gian phản hồi	147
Bảng 4.8. Kết quả tổng hợp mô hình AI.....	148

DANH MỤC CÁC TỪ VIẾT TẮT

AI	Artificial Intelligence (Trí tuệ nhân tạo)
AIoT	Artificial Intelligence of Things (Trí tuệ nhân tạo kết hợp Internet vạn vật)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
BM25	Best Matching 25
CPU	Central Processing Unit (Bộ xử lý trung tâm)
ESP32	Espressif 32-bit Microcontroller
FastAPI	FastAPI Framework
Firebase	Firebase Realtime Database Platform
GPIO	General Purpose Input/Output
HTTP	HyperText Transfer Protocol
Hybrid RAG	Hybrid Retrieval-Augmented Generation
IoT	Internet of Things (Internet vạn vật)
I2C	Inter-Integrated Circuit
JSON	JavaScript Object Notation
LangGraph	Language Graph Framework
LLM	Large Language Model (Mô hình ngôn ngữ lớn)
MQTT	Message Queuing Telemetry Transport
Multi-Agent	Multi-Agent System (Hệ đa tác tử)
Qdrant	Vector Database Qdrant
RAG	Retrieval-Augmented Generation
ReactJS	React JavaScript Library
REST	Representational State Transfer
RFID	Radio Frequency Identification
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver Transmitter

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong bối cảnh cuộc Cách mạng công nghiệp lần thứ tư đang diễn ra mạnh mẽ trên phạm vi toàn cầu, các công nghệ như Internet vạn vật (Internet of Things - IoT), điện toán đám mây (Cloud Computing), dữ liệu lớn (Big Data) và trí tuệ nhân tạo (Artificial Intelligence - AI) đang từng bước làm thay đổi phương thức vận hành của nhiều lĩnh vực trong đời sống và sản xuất. Trong đó, lĩnh vực bán lẻ là một trong những ngành chịu tác động mạnh mẽ nhất bởi quá trình chuyển đổi số, khi các doanh nghiệp ngày càng có nhu cầu ứng dụng công nghệ để nâng cao hiệu quả quản lý, tối ưu hóa quy trình vận hành và cải thiện trải nghiệm khách hàng.

Đối với các cửa hàng kinh doanh linh kiện điện tử và thiết bị IoT, việc quản lý sản phẩm, kiểm soát tồn kho, theo dõi đơn hàng và hỗ trợ khách hàng lựa chọn linh kiện phù hợp luôn là những bài toán quan trọng. Trong thực tế, số lượng linh kiện điện tử trên thị trường hiện nay rất đa dạng với nhiều chủng loại, thông số kỹ thuật và khả năng tương thích khác nhau. Điều này khiến người dùng, đặc biệt là sinh viên, người mới học IoT hoặc các nhà phát triển hệ thống nhúng gặp nhiều khó khăn trong quá trình tìm kiếm, lựa chọn và sử dụng linh kiện.

Bên cạnh đó, các hệ thống bán lẻ truyền thống hiện nay chủ yếu tập trung vào các chức năng quản lý sản phẩm và giao dịch mua bán, trong khi khả năng hỗ trợ tư vấn kỹ thuật còn hạn chế. Khi khách hàng cần tìm hiểu về cách kết nối phần cứng, lựa chọn module phù hợp, xây dựng dự án IoT hoặc xử lý các vấn đề kỹ thuật phát sinh, họ thường phải tự tìm kiếm thông tin từ nhiều nguồn khác nhau. Điều này không chỉ làm mất thời gian mà còn có thể dẫn đến việc tiếp cận các thông tin thiếu chính xác hoặc không phù hợp với nhu cầu thực tế.

Trong những năm gần đây, sự phát triển mạnh mẽ của các mô hình ngôn ngữ lớn (Large Language Model - LLM) đã mở ra nhiều hướng tiếp cận mới trong việc xây dựng các hệ thống hỗ trợ thông minh. Các mô hình này có khả năng hiểu ngôn ngữ tự nhiên, phân tích yêu cầu của người dùng và tạo ra các phản hồi mang tính hội thoại. Tuy nhiên, các mô hình ngôn ngữ lớn vẫn tồn tại một số hạn chế như kiến thức bị giới hạn trong dữ liệu huấn luyện, khó cập nhật thông tin mới và đôi khi có thể sinh ra các câu trả lời chưa chính xác.

Để khắc phục những hạn chế đó, kỹ thuật Retrieval-Augmented Generation (RAG) đã được đề xuất nhằm kết hợp khả năng truy xuất tri thức từ cơ sở dữ liệu bên ngoài với khả năng sinh ngôn ngữ của mô hình AI. Đồng thời, kiến trúc Multi-Agent cho phép phân chia nhiệm vụ

vụ cho nhiều tác tử chuyên biệt, từ đó nâng cao hiệu quả xử lý và khả năng mở rộng của hệ thống. Việc kết hợp các công nghệ hiện đại như Multi-Agent, Hybrid RAG, Vector Database và các mô hình ngôn ngữ lớn tạo tiền đề cho việc xây dựng các hệ thống tư vấn thông minh có khả năng cung cấp thông tin chính xác, cập nhật và phù hợp với từng ngữ cảnh cụ thể.

Song song với đó, công nghệ IoT cũng ngày càng được ứng dụng rộng rãi trong lĩnh vực quản lý và giám sát. Các thiết bị IoT có khả năng thu thập dữ liệu theo thời gian thực, hỗ trợ giám sát trạng thái thiết bị, quản lý tài nguyên và kết nối với các hệ thống phần mềm thông qua Internet. Việc tích hợp IoT với hệ thống quản lý bán lẻ giúp nâng cao khả năng giám sát, tự động hóa và đồng bộ dữ liệu trong toàn bộ hệ thống.

Xuất phát từ những nhu cầu thực tiễn nêu trên, nhóm tác giả lựa chọn thực hiện đề tài: “Nghiên cứu và xây dựng mô hình AIoT Smart Retail ứng dụng trí tuệ nhân tạo đa tác tử và Hybrid RAG hỗ trợ quản lý kho và tư vấn kỹ thuật”. Đề tài hướng tới việc xây dựng một nền tảng tích hợp giữa công nghệ Web, IoT và trí tuệ nhân tạo nhằm hỗ trợ quản lý bán lẻ linh kiện điện tử, đồng thời cung cấp khả năng tư vấn kỹ thuật thông minh cho người dùng. Đây không chỉ là một hướng nghiên cứu có ý nghĩa khoa học mà còn có tiềm năng ứng dụng thực tiễn cao trong quá trình chuyển đổi số của các mô hình bán lẻ hiện nay.

1.2. Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là nghiên cứu, thiết kế và xây dựng mô hình AIoT Smart Retail có khả năng quản lý bán lẻ linh kiện điện tử, giám sát dữ liệu IoT theo thời gian thực và hỗ trợ tư vấn kỹ thuật thông minh dựa trên trí tuệ nhân tạo.

Để đạt được mục tiêu tổng quát trên, đề tài đặt ra các mục tiêu cụ thể như sau:

- Nghiên cứu các công nghệ liên quan đến IoT, phát triển ứng dụng Web, điện toán đám mây và trí tuệ nhân tạo.
- Xây dựng hệ Web phục vụ quản lý sản phẩm, danh mục linh kiện, đơn hàng và tồn kho.
- Thiết kế cơ sở dữ liệu thời gian thực cho phép đồng bộ dữ liệu giữa các thiết bị IoT và cơ sở quản lý.
- Xây dựng phần cứng sử dụng ESP32 để thu thập và truyền dữ liệu lên nền tảng đám mây.

- Xây dựng AI Backend dựa trên FastAPI nhằm xử lý các yêu cầu liên quan đến trí tuệ nhân tạo.
- Nghiên cứu và triển khai kiến trúc Multi-Agent để phân chia nhiệm vụ và tối ưu hóa quá trình xử lý câu hỏi.
- Xây dựng cơ sở tri thức về linh kiện điện tử, thiết bị IoT và các kiến thức kỹ thuật liên quan.
- Triển khai mô hình Hybrid RAG bằng cách kết hợp BM25 Truy xuất và Vector Truy xuất nhằm nâng cao hiệu quả truy xuất thông tin.
- Tích hợp mô hình ngôn ngữ lớn để hỗ trợ tư vấn linh kiện, giải đáp thắc mắc kỹ thuật và hỗ trợ người dùng trong quá trình phát triển các dự án IoT.
- Đánh giá hiệu năng và khả năng hoạt động của mô hình thông qua các kịch bản thử nghiệm thực tế.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các công nghệ và giải pháp phục vụ xây dựng mô hình bán lẻ thông minh tích hợp giữa IoT, công nghệ Web và trí tuệ nhân tạo. Trong đó, đề tài tập trung nghiên cứu các phương pháp quản lý dữ liệu theo thời gian thực, các kỹ thuật truy xuất tri thức và các mô hình trí tuệ nhân tạo hỗ trợ tư vấn thông minh.

Cụ thể, các đối tượng nghiên cứu chính của đề tài bao gồm:

- Công nghệ Internet vạn vật (IoT) ứng dụng trong việc thu thập, truyền nhận và giám sát dữ liệu từ các thiết bị phần cứng.
- Vi điều khiển ESP32 và các phương thức kết nối dữ liệu thông qua mạng Internet.
- Firebase Realtime Database và Firebase Authentication phục vụ lưu trữ, đồng bộ và quản lý dữ liệu theo thời gian thực.
- Các công nghệ phát triển ứng dụng Web hiện đại như ReactJS và FastAPI.
- Các mô hình ngôn ngữ lớn (Large Language Model - LLM) phục vụ xử lý ngôn ngữ tự nhiên.
- Kiến trúc Multi-Agent trong các ứng dụng trí tuệ nhân tạo hiện đại.
- Kỹ thuật Retrieval-Augmented Generation (RAG) và Hybrid RAG.
- Các phương pháp truy xuất thông tin như BM25 Truy xuất và Vector Retrieval.

- Cơ sở dữ liệu vector Qdrant phục vụ lưu trữ và tìm kiếm dữ liệu ngữ nghĩa.
- Các mô hình hỗ trợ tư vấn kỹ thuật và quản lý bán lẻ thông minh.

Ngoài ra, đề tài cũng nghiên cứu các mô hình tổ chức dữ liệu và quy trình xử lý thông tin trong môi trường bán lẻ linh kiện điện tử nhằm xây dựng mô hình có khả năng hỗ trợ người dùng hiệu quả hơn trong quá trình tìm kiếm, lựa chọn và sử dụng sản phẩm.

1.3.2. Phạm vi nghiên cứu

Do giới hạn về thời gian thực hiện, nguồn lực triển khai và phạm vi của một đồ án tốt nghiệp đại học, đề tài tập trung nghiên cứu và triển khai các nội dung chính sau:

- Về phần cứng, đề tài sử dụng vi điều khiển ESP32 làm thiết bị thu thập và truyền dữ liệu lên nền tảng Firebase. Mô hình tập trung vào việc minh họa khả năng kết nối và đồng bộ dữ liệu thời gian thực giữa thiết bị IoT và phần mềm.
- Về phần mềm, đề tài xây dựng ứng dụng Web phục vụ quản lý sản phẩm, quản lý kho, quản lý đơn hàng và theo dõi dữ liệu IoT. Mô hình hỗ trợ các chức năng cơ bản phục vụ hoạt động quản lý bán lẻ linh kiện điện tử.
- Về trí tuệ nhân tạo, đề tài tập trung nghiên cứu và triển khai AI Backend dựa trên kiến trúc Multi-Agent kết hợp Hybrid RAG nhằm hỗ trợ tư vấn linh kiện điện tử, giải đáp các câu hỏi kỹ thuật liên quan đến IoT và hỗ trợ phân tích dữ liệu tồn kho.
- Về dữ liệu, cơ sở tri thức được xây dựng từ các tài liệu kỹ thuật, tài liệu hướng dẫn và các kiến thức liên quan đến linh kiện điện tử, thiết bị IoT và hệ thống nhúng. Dữ liệu được xử lý, phân đoạn và lưu trữ trong cơ sở dữ liệu vector phục vụ quá trình truy xuất thông tin.
- Về đánh giá mô hình, đề tài tập trung đánh giá khả năng hoạt động của các chức năng chính như quản lý dữ liệu, đồng bộ dữ liệu thời gian thực, truy xuất tri thức, tư vấn kỹ thuật và phản hồi câu hỏi của người dùng.
- Bên cạnh đó, đề tài không tập trung nghiên cứu việc huấn luyện mô hình ngôn ngữ lớn từ đầu do yêu cầu rất lớn về dữ liệu, tài nguyên tính toán và thời gian huấn luyện. Thay vào đó, đề tài sử dụng các mô hình ngôn ngữ có sẵn thông qua nền tảng OpenRouter để xây dựng các chức năng hỗ trợ thông minh.

Ngoài ra, đề tài cũng chưa đi sâu vào các nội dung như tối ưu hiệu năng trên quy mô lớn, triển khai mô hình phân tán, xây dựng mô hình học máy chuyên biệt cho từng lĩnh vực hoặc

phát triển thành chuỗi hệ thống thương mại hoàn chỉnh. Những nội dung này sẽ được xem xét như các hướng phát triển trong tương lai.

1.4. Phương pháp nghiên cứu

Để hoàn thành các mục tiêu đề ra, đề tài sử dụng kết hợp nhiều phương pháp nghiên cứu khác nhau nhằm đảm bảo tính khoa học, tính thực tiễn và khả năng triển khai của mô hình.

Trước hết, phương pháp nghiên cứu lý thuyết được sử dụng để tìm hiểu các công nghệ, mô hình và giải pháp liên quan đến đề tài. Quá trình nghiên cứu tập trung vào các lĩnh vực như Internet vạn vật, phát triển ứng dụng Web, điện toán đám mây, trí tuệ nhân tạo, mô hình ngôn ngữ lớn, Hybrid RAG, Multi-Agent và cơ sở dữ liệu vector. Các tài liệu khoa học, sách chuyên ngành, bài báo nghiên cứu và tài liệu kỹ thuật được sử dụng làm cơ sở xây dựng nền tảng lý thuyết cho đề tài.

Tiếp theo, phương pháp phân tích và thiết kế mô hình được áp dụng để xác định các yêu cầu chức năng và phi chức năng của đề tài. Trên cơ sở đó, kiến trúc tổng thể của mô hình được xây dựng nhằm đảm bảo khả năng mở rộng, khả năng bảo trì và hiệu quả hoạt động trong thực tế.

Phương pháp phát triển phần mềm được sử dụng để triển khai các thành phần của mô hình bao gồm giao diện Web, cơ sở dữ liệu, AI Backend và các dịch vụ hỗ trợ. Các công nghệ phù hợp được lựa chọn nhằm đáp ứng yêu cầu của đề tài và đảm bảo khả năng tích hợp giữa các thành phần khác nhau.

Đối với phần cứng, phương pháp thực nghiệm được sử dụng để xây dựng và kiểm thử mô hình nhúng IoT dựa trên ESP32. Quá trình thực nghiệm giúp đánh giá khả năng kết nối, truyền dữ liệu và đồng bộ thông tin giữa thiết bị IoT và nền tảng phần mềm.

Ngoài ra, phương pháp tích hợp mô hình được áp dụng nhằm kết nối các thành phần phần cứng, phần mềm và trí tuệ nhân tạo thành một hệ thống nhất. Quá trình này giúp đánh giá khả năng phối hợp giữa các thành phần và phát hiện các vấn đề phát sinh trong quá trình vận hành thực tế.

Cuối cùng, phương pháp đánh giá và phân tích kết quả được sử dụng để kiểm tra mức độ đáp ứng của mô hình đối với các mục tiêu đề ra. Các tiêu chí như khả năng hoạt động ổn định, độ chính xác của truy xuất tri thức, chất lượng phản hồi của AI và khả năng đồng bộ dữ liệu được sử dụng để đánh giá hiệu quả của giải pháp được xây dựng.

1.5. Bố cục đề tài

Để đảm bảo tính logic và thuận tiện cho việc trình bày, nội dung báo cáo được tổ chức thành năm chương chính, phản ánh toàn bộ quá trình nghiên cứu, thiết kế, triển khai và đánh giá mô hình AIoT Smart Retail.

Chương 1: Tổng quan về đề tài

Chương này trình bày bối cảnh thực tiễn và lý do lựa chọn đề tài, đồng thời phân tích những khó khăn và nhu cầu trong lĩnh vực kinh doanh linh kiện điện tử hiện nay. Bên cạnh đó, chương cũng giới thiệu mục tiêu nghiên cứu, đối tượng và phạm vi nghiên cứu, phương pháp thực hiện cũng như bố cục tổng thể của báo cáo.

Chương 2: Cơ sở lý thuyết

Chương này trình bày các cơ sở lý thuyết và các công nghệ nền tảng được sử dụng trong quá trình xây dựng mô hình. Nội dung bao gồm kiến thức về ReactJS, Firebase Realtime Database, Firebase Authentication, FastAPI, kiến trúc Agentic AI, Hybrid RAG, Vector Database, các mô hình ngôn ngữ lớn (LLM), EmailJS, eSMS và các công nghệ IoT liên quan đến vi điều khiển ESP32. Đây là cơ sở quan trọng để thiết kế và triển khai trong các chương tiếp theo.

Chương 3: Thiết kế và triển khai mô hình

Chương này tập trung trình bày quá trình phân tích và thiết kế mô hình. Nội dung bao gồm kiến trúc tổng thể của đề tài, thiết kế cơ sở dữ liệu Firebase, thiết kế Web Frontend, thiết kế AI Backend, thiết kế Smart Locker và các cơ chế trao đổi dữ liệu giữa các thành phần. Ngoài ra, chương cũng trình bày các sơ đồ luồng xử lý, sơ đồ chức năng và các giải pháp kỹ thuật được lựa chọn trong quá trình phát triển mô hình AIoT.

Chương 4: Kết quả triển khai và đánh giá mô hình

Chương này trình bày kết quả triển khai thực tế của toàn bộ mô hình AIoT Smart Retail. Các chức năng của Web Dashboard, AI Assistant, Smart Locker và các dịch vụ hỗ trợ được kiểm thử và đánh giá thông qua nhiều kịch bản vận hành khác nhau. Bên cạnh việc minh họa bằng hình ảnh thực tế, chương còn phân tích kết quả hoạt động, đánh giá tính ổn định, khả năng đồng bộ dữ liệu và mức độ đáp ứng các yêu cầu đặt ra ban đầu.

Chương 5: Kết luận và hướng phát triển

Chương cuối cùng tổng kết các kết quả đạt được trong quá trình thực hiện đề tài, đánh giá những đóng góp của mô hình đối với lĩnh vực bán lẻ thông minh kết hợp AI và IoT.

Đồng thời, chương cũng chỉ ra các hạn chế còn tồn tại và đề xuất các hướng nghiên cứu, mở rộng và nâng cấp mô hình trong tương lai nhằm nâng cao khả năng ứng dụng thực tế và hướng tới triển khai trên quy mô lớn.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Vi điều khiển ESP32

ESP32 là dòng vi điều khiển tích hợp WiFi và Bluetooth được phát triển bởi Espressif Systems nhằm phục vụ các ứng dụng Internet of Things (IoT), hệ thống nhúng và các thiết bị kết nối không dây [1]. ESP32 được xây dựng trên kiến trúc vi xử lý Xtensa LX6 32 bit với khả năng xử lý mạnh mẽ, hỗ trợ nhiều giao thức truyền thông và nhiều chế độ tiết kiệm năng lượng.

Nhờ khả năng tích hợp WiFi chuẩn IEEE 802.11 b/g/n và Bluetooth/BLE trên cùng một chip, ESP32 giúp giảm chi phí phần cứng và đơn giản hóa việc thiết kế mô hình. Đây là một trong những dòng vi điều khiển được sử dụng rộng rãi trong các ứng dụng IoT hiện nay¹.



Hình 2.1. Module ESP32 DevKit

a. Các chuẩn giao tiếp của ESP32

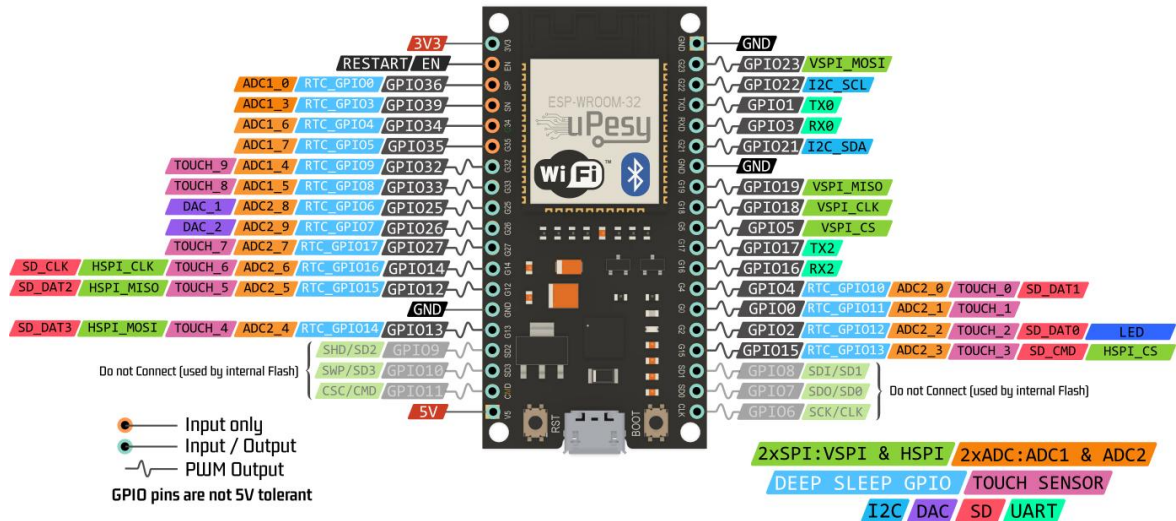
ESP32 hỗ trợ nhiều chuẩn giao tiếp ngoại vi khác nhau như GPIO, UART, SPI, I2C, PWM, WiFi và Bluetooth.

- GPIO dùng để giao tiếp với các thiết bị số.

¹ Espressif Systems, "ESP32 Series Datasheet", Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

- UART hỗ trợ truyền nhận dữ liệu nối tiếp.
- SPI hỗ trợ truyền dữ liệu tốc độ cao.
- I2C sử dụng hai dây tín hiệu SDA và SCL để kết nối với các cảm biến và thiết bị ngoại vi.
- PWM được sử dụng trong điều khiển động cơ Servo và điều chỉnh độ sáng LED.
- WiFi và Bluetooth phục vụ các ứng dụng kết nối không dây.

ESP32 Wroom DevKit Full Pinout



Hình 2.2. Sơ đồ chân của ESP32

b. Đặc điểm kỹ thuật của ESP32

Theo tài liệu kỹ thuật của Espressif Systems, ESP32 có các đặc điểm nổi bật như sau:

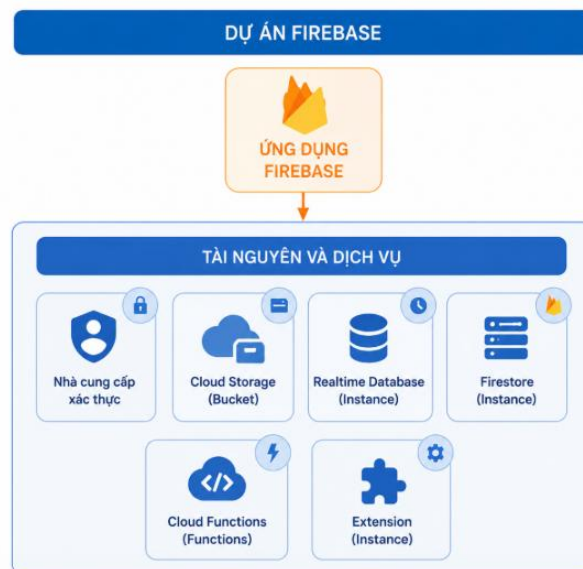
- Bộ xử lý Xtensa LX6 32 bit.
- Tần số hoạt động tối đa 240 MHz.
- Hỗ trợ WiFi 802.11 b/g/n.
- Hỗ trợ Bluetooth Classic và BLE.
- Nhiều chân GPIO đa chức năng.
- Hỗ trợ các giao thức UART, SPI, I2C, PWM và ADC.
- Tích hợp các chế độ tiết kiệm năng lượng.

2.2. Cơ sở dữ liệu thời gian thực Firebase

Firebase là nền tảng phát triển ứng dụng được Google cung cấp nhằm hỗ trợ xây dựng các ứng dụng Web, Android và iOS thông qua nhiều dịch vụ tích hợp sẵn như Authentication, Realtime Database, Cloud Firestore, Cloud Storage, Hosting và Cloud Functions.²

Firebase hoạt động theo mô hình Backend-as-a-Service (BaaS), cho phép nhà phát triển sử dụng các dịch vụ đám mây có sẵn thay vì phải xây dựng và quản lý hệ thống máy chủ riêng. Nhờ đó, thời gian phát triển ứng dụng được rút ngắn đáng kể và khả năng mở rộng của mô hình được nâng cao.

Trong số các dịch vụ mà Firebase cung cấp, Firebase Realtime Database là một trong những thành phần quan trọng nhất nhờ khả năng lưu trữ và đồng bộ dữ liệu theo thời gian thực.



Hình 2.3. Kiến trúc tổng quan nền tảng Firebase

2.2.1. Firebase Realtime Database

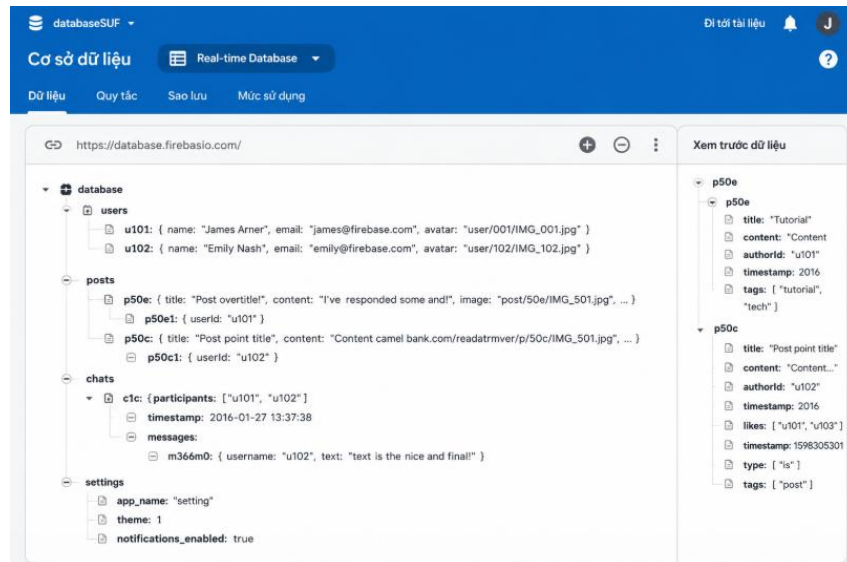
Firebase Realtime Database là cơ sở dữ liệu NoSQL được lưu trữ trên nền tảng đám mây và hỗ trợ khả năng đồng bộ dữ liệu theo thời gian thực giữa nhiều thiết bị khác nhau.

Khác với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống lưu trữ dữ liệu dưới dạng bảng, Firebase Realtime Database tổ chức dữ liệu theo cấu trúc cây JSON (JSON Tree). Trong cấu

²Google, "Firebase Realtime Database Documentation", Available: <https://firebase.google.com/docs/database>

trúc này, dữ liệu được lưu dưới dạng các cặp khóa - giá trị và được sắp xếp theo mô hình phân cấp.

Mô hình lưu trữ này giúp việc truy xuất dữ liệu trở nên linh hoạt và phù hợp với các ứng dụng Web thời gian thực, hệ thống IoT và các nền tảng cần cập nhật dữ liệu liên tục.



Hình 2.4. Cấu trúc dữ liệu dạng JSON Tree trong Firebase Realtime Database

2.2.2. Cơ chế đồng bộ dữ liệu thời gian thực

Đặc điểm nổi bật nhất của Firebase Realtime Database là khả năng đồng bộ dữ liệu theo thời gian thực. Khi dữ liệu được cập nhật trên máy chủ, Firebase sẽ tự động gửi các thay đổi đến tất cả các thiết bị đang theo dõi dữ liệu đó mà không cần thực hiện các thao tác truy vấn lặp lại.

Cơ chế này được thực hiện thông qua kết nối liên tục giữa máy khách và máy chủ Firebase. Nhờ đó, dữ liệu trên các thiết bị luôn được duy trì ở trạng thái đồng nhất trong thời gian rất ngắn sau khi có sự thay đổi xảy ra.

Ngoài ra, Firebase còn hỗ trợ khả năng lưu trữ dữ liệu cục bộ trên thiết bị. Khi mất kết nối Internet, ứng dụng vẫn có thể truy cập dữ liệu đã được lưu trong bộ nhớ đệm. Sau khi kết nối được khôi phục, các thay đổi sẽ được đồng bộ trở lại máy chủ một cách tự động.

Ưu điểm của Firebase Realtime Database

Firebase Realtime Database sở hữu nhiều ưu điểm nổi bật như khả năng đồng bộ dữ liệu theo thời gian thực, khả năng mở rộng linh hoạt trên nền tảng đám mây và hỗ trợ nhiều nền tảng phát triển khác nhau.

Bên cạnh đó, Firebase còn cung cấp các cơ chế xác thực người dùng, phân quyền truy cập dữ liệu và mã hóa kết nối nhằm đảm bảo tính an toàn trong quá trình truyền nhận dữ liệu. Các đặc điểm này giúp Firebase trở thành một trong những nền tảng được sử dụng phổ biến trong các ứng dụng Web thời gian thực và hệ thống IoT hiện nay.

2.3. Công nghệ phát triển ứng dụng Web

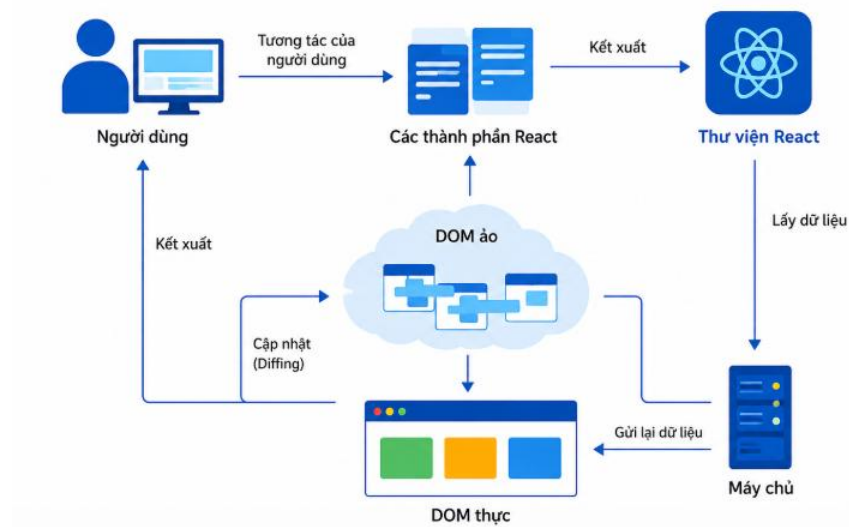
2.3.1. Thư viện ReactJS

ReactJS là thư viện JavaScript mã nguồn mở được phát triển bởi Meta (Facebook) nhằm hỗ trợ xây dựng giao diện người dùng (User Interface - UI) cho các ứng dụng Web hiện đại³. ReactJS sử dụng kiến trúc dựa trên Component, trong đó giao diện được chia thành nhiều thành phần nhỏ độc lập giúp quá trình phát triển, bảo trì và tái sử dụng mã nguồn trở nên dễ dàng hơn. [2]

Một trong những đặc điểm nổi bật của ReactJS là cơ chế Virtual DOM (Virtual Document Object Model). Thay vì cập nhật trực tiếp toàn bộ trang Web mỗi khi dữ liệu thay đổi, ReactJS chỉ cập nhật các thành phần thực sự cần thiết. Điều này giúp giảm chi phí xử lý và cải thiện hiệu năng của ứng dụng.

Bên cạnh đó, ReactJS hỗ trợ cơ chế One-Way Data Binding giúp luồng dữ liệu được quản lý theo một chiều từ Component cha xuống Component con. Cách tiếp cận này giúp hạn chế các lỗi phát sinh trong quá trình phát triển và dễ dàng kiểm soát trạng thái của ứng dụng.

³ Meta Platforms, "React Documentation", Available: <https://react.dev/>



Hình 2.5. Quy trình hoạt động của ReactJS

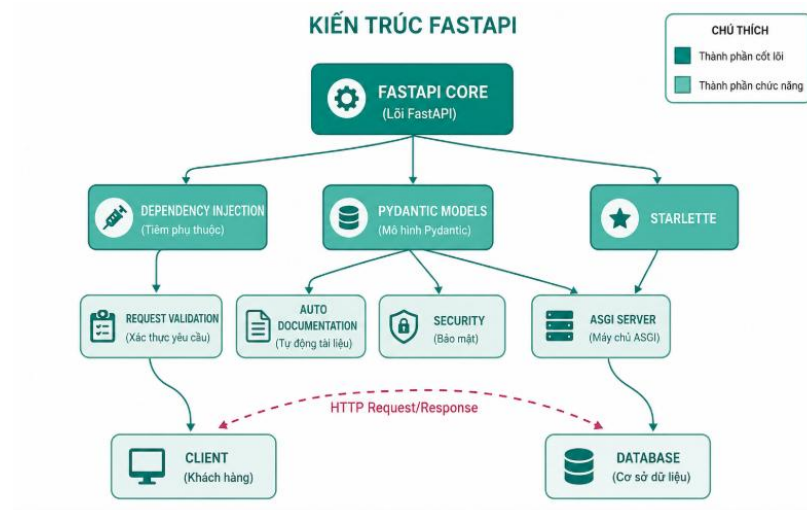
2.3.2. Framework FastAPI

FastAPI là một Web Framework mã nguồn mở được phát triển bằng ngôn ngữ Python nhằm hỗ trợ xây dựng các RESTful API hiệu năng cao . FastAPI được xây dựng dựa trên hai thư viện Starlette và Pydantic, cho phép xử lý bất đồng bộ (Asynchronous Programming) và kiểm tra dữ liệu đầu vào một cách tự động⁴.

Một trong những ưu điểm nổi bật của FastAPI là khả năng tự động sinh tài liệu API thông qua Swagger UI và ReDoc. Khi xây dựng API, tài liệu kỹ thuật được tạo tự động dựa trên các khai báo trong mã nguồn, giúp quá trình kiểm thử và phát triển mô hình trở nên thuận tiện hơn .

Ngoài ra, FastAPI hỗ trợ cơ chế Dependency Injection, Validation và Type Hinting, giúp nâng cao chất lượng mã nguồn và giảm thiểu lỗi trong quá trình phát triển ứng dụng .

⁴ Sebastián Ramírez, "FastAPI Documentation", Available:<https://fastapi.tiangolo.com/>



Hình 2.6. Kiến trúc hoạt động của FastAPI

RESTful API

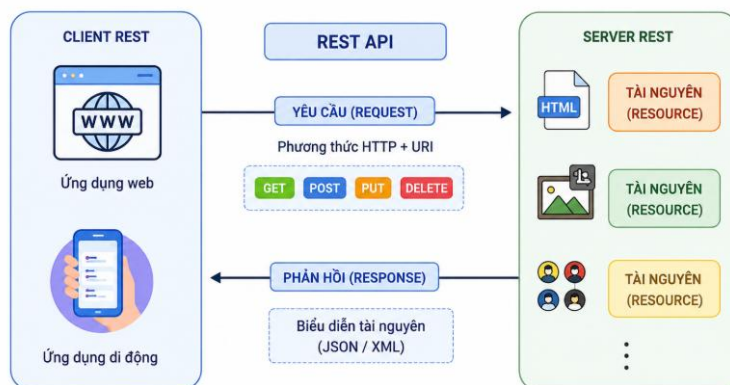
REST (Representational State Transfer) là một phong cách kiến trúc phần mềm được sử dụng phổ biến trong việc xây dựng các dịch vụ Web. REST định nghĩa cách thức giao tiếp thông qua giao thức HTTP và sử dụng các tài nguyên (Resources) để trao đổi dữ liệu.

Trong kiến trúc RESTful API, mỗi tài nguyên được định danh thông qua một URL duy nhất. Các thao tác trên tài nguyên được thực hiện thông qua các phương thức HTTP như:

- GET: Truy xuất dữ liệu.
- POST: Tạo mới dữ liệu.
- PUT: Cập nhật dữ liệu.
- PATCH: Cập nhật một phần dữ liệu.
- DELETE: Xóa dữ liệu.

Dữ liệu trao đổi giữa máy khách và máy chủ thường được biểu diễn dưới định dạng JSON do tính đơn giản, khả năng mở rộng và tính tương thích cao với nhiều nền tảng khác nhau.

RESTful API hiện được sử dụng rộng rãi trong các hệ thống Web, ứng dụng di động, nền tảng điện toán đám mây và các hệ thống IoT nhờ khả năng mở rộng linh hoạt và dễ dàng tích hợp giữa các thành phần phần mềm.



Hình 2.7. Mô hình giao tiếp Client - Server thông qua RESTful API

2.4. Công nghệ gửi Email và SMS

2.4.1. Dịch vụ EmailJS

EmailJS là nền tảng dịch vụ cho phép gửi thư điện tử trực tiếp từ ứng dụng Web mà không cần xây dựng máy chủ SMTP riêng. EmailJS hoạt động như một cầu nối giữa ứng dụng Web và các dịch vụ thư điện tử phổ biến như Gmail, Outlook hay Yahoo Mail.

Thông qua EmailJS, các nhà phát triển có thể cấu hình mẫu thư (Email Template), danh sách người nhận và các tham số động để tự động hóa quá trình gửi thư điện tử. Việc gửi Email được thực hiện thông qua API do EmailJS cung cấp, giúp giảm đáng kể khối lượng công việc trong quá trình phát triển mô hình⁵.

Một ưu điểm quan trọng của EmailJS là khả năng tích hợp dễ dàng với các ứng dụng Frontend sử dụng JavaScript, ReactJS hoặc các Framework Web hiện đại khác. Điều này cho phép ứng dụng gửi Email trực tiếp từ giao diện người dùng mà không cần triển khai thêm dịch vụ Backend chuyên biệt cho việc gửi thư điện tử.

Cơ chế gửi Email bằng EmailJS

Quá trình gửi Email bằng EmailJS thường bao gồm ba thành phần chính:

- Email Service.
- Email Template.
- Public Key xác thực ứng dụng.

⁵ EmailJS, "EmailJS Documentation", Available: <https://www.emailjs.com/docs/>

Khi người dùng thực hiện một hành động cần gửi Email, ứng dụng sẽ gửi dữ liệu đến EmailJS thông qua API. EmailJS sẽ sử dụng Template đã được cấu hình trước để tạo nội dung Email và chuyển tiếp thư đến máy chủ thư điện tử tương ứng.

Quá trình này giúp giảm độ phức tạp của mô hình vì ứng dụng không cần trực tiếp quản lý SMTP Server hay các giao thức gửi thư điện tử truyền thống.

2.4.2. Dịch vụ eSMS

eSMS là nền tảng cung cấp dịch vụ gửi tin nhắn SMS thông qua API được sử dụng rộng rãi tại Việt Nam. Dịch vụ này cho phép các ứng dụng phần mềm gửi tin nhắn trực tiếp đến số điện thoại của người dùng thông qua kết nối Internet.

eSMS hỗ trợ nhiều loại hình tin nhắn khác nhau như SMS Brandname, SMS OTP và SMS quảng cáo. Thông qua các API được cung cấp, ứng dụng có thể gửi tin nhắn tự động đến người dùng mà không cần xây dựng hạ tầng riêng.

Việc sử dụng dịch vụ SMS Gateway giúp giảm chi phí triển khai mô hình đồng thời đảm bảo khả năng gửi tin nhắn nhanh chóng và ổn định.

Cơ chế gửi SMS bằng eSMS

Mô hình sử dụng API do eSMS cung cấp để gửi tin nhắn SMS đến người dùng. Quá trình gửi tin nhắn bắt đầu khi ứng dụng gửi yêu cầu HTTP đến máy chủ eSMS kèm theo nội dung tin nhắn, số điện thoại người nhận và thông tin xác thực tài khoản⁶

Sau khi tiếp nhận yêu cầu, eSMS thực hiện kiểm tra thông tin xác thực và chuyển tiếp tin nhắn đến nhà mạng. Nhà mạng sau đó gửi SMS đến thiết bị di động của người nhận. Nhờ cơ chế API này, quá trình gửi SMS có thể được tự động hóa hoàn toàn và dễ dàng tích hợp với các ứng dụng Web, mô hình IoT hoặc các nền tảng quản lý trực tuyến.

Ưu điểm của EmailJS và eSMS

EmailJS và eSMS đều cung cấp cơ chế gửi thông báo tự động thông qua API, giúp các ứng dụng dễ dàng tích hợp chức năng gửi Email và SMS mà không cần xây dựng hạ tầng truyền thông riêng.

Việc sử dụng các dịch vụ này giúp giảm thời gian phát triển mô hình, nâng cao khả năng mở rộng và đảm bảo độ tin cậy trong quá trình gửi thông báo đến người dùng. Ngoài

⁶ eSMS, "SMS API Documentation", Available:<https://esms.vn/sms-api/>

ra, các nền tảng này còn hỗ trợ theo dõi trạng thái gửi, thống kê số lượng thông báo và quản lý lịch sử truyền thông một cách hiệu quả.

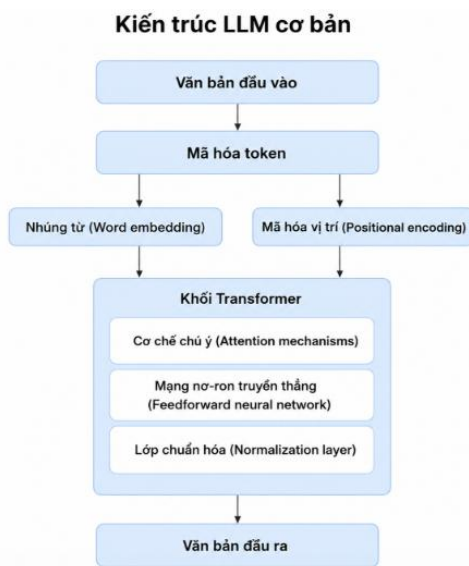
2.5. Trí tuệ nhân tạo và mô hình ngôn ngữ lớn

2.5.1. Mô hình ngôn ngữ lớn (LLM)

Mô hình ngôn ngữ lớn (Large Language Model - LLM) là một dạng mô hình trí tuệ nhân tạo được huấn luyện trên tập dữ liệu văn bản có quy mô rất lớn nhằm học các quy luật ngôn ngữ tự nhiên và tạo ra văn bản có ý nghĩa tương tự như con người. [3]

Các mô hình LLM hiện đại được xây dựng dựa trên kiến trúc Transformer do Vaswani và cộng sự đề xuất vào năm 2017. Thông qua quá trình huấn luyện trên hàng tỷ từ và câu văn, mô hình có khả năng hiểu ngữ cảnh, trả lời câu hỏi, tóm tắt văn bản, dịch ngôn ngữ và hỗ trợ nhiều tác vụ xử lý ngôn ngữ tự nhiên khác nhau.

Trong những năm gần đây, sự phát triển của các mô hình như GPT, Gemini, Claude, Llama và DeepSeek đã mở rộng đáng kể khả năng ứng dụng của trí tuệ nhân tạo trong nhiều lĩnh vực khác nhau như giáo dục, y tế, thương mại điện tử và công nghiệp.



Hình 2.8. Kiến trúc LLM cơ bản

2.5.2. Kiến trúc Transformer

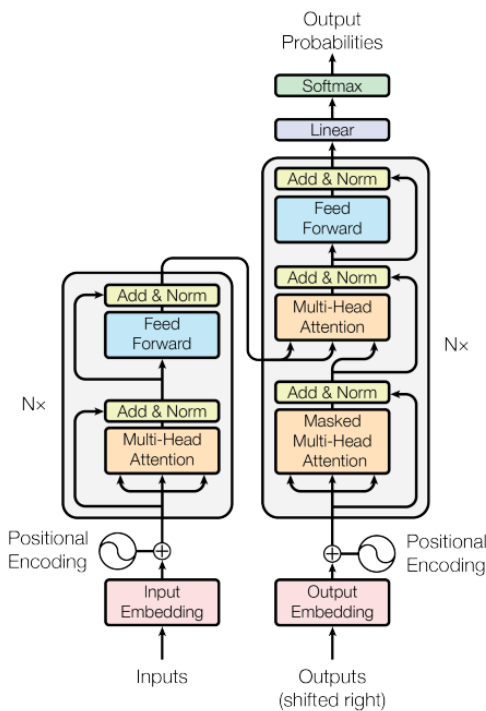
Transformer là kiến trúc nền tảng của hầu hết các mô hình ngôn ngữ lớn hiện nay. Điểm nổi bật của Transformer là cơ chế Self-Attention, cho phép mô hình xác định mức độ liên quan giữa các từ trong câu mà không cần xử lý tuần tự như các mạng RNN hay LSTM truyền thống.

Nhờ cơ chế này, Transformer có khả năng xử lý song song dữ liệu đầu vào, tăng tốc độ huấn luyện và cải thiện khả năng nắm bắt ngữ cảnh trong các văn bản dài. [4]

Kiến trúc Transformer bao gồm hai thành phần chính:

- Encoder.
- Decoder.

Encoder có nhiệm vụ trích xuất đặc trưng từ dữ liệu đầu vào, trong khi Decoder thực hiện quá trình sinh dữ liệu đầu ra dựa trên thông tin đã được học.



Hình 2.9. Kiến trúc Transformer (Nguồn: *Attention Is All You Need*)

2.5.3. Kỹ thuật thiết kế Prompt (Prompt Engineering)

Prompt Engineering là kỹ thuật thiết kế và tối ưu hóa câu lệnh đầu vào nhằm hướng dẫn mô hình ngôn ngữ lớn tạo ra phản hồi chính xác và phù hợp với yêu cầu của người dùng⁷. Chất lượng của Prompt ảnh hưởng trực tiếp đến chất lượng phản hồi được tạo ra. Một Prompt tốt thường bao gồm:

- Vai trò của mô hình.
- Ngữ cảnh liên quan.

⁷ OpenAI, "Prompt Engineering Guide", Available: <https://platform.openai.com/docs/guides/prompt-engineering>

- Nhiệm vụ cần thực hiện.
- Định dạng đầu ra mong muốn.

Prompt Engineering đóng vai trò quan trọng trong việc nâng cao hiệu quả khai thác các mô hình ngôn ngữ lớn, đặc biệt trong các mô hình chatbot, trợ lý ảo và nền tảng hỏi đáp thông minh.



Hình 2.10. Mối quan hệ giữa Prompt và phản hồi của LLM

2.5.4. Nền tảng OpenRouter

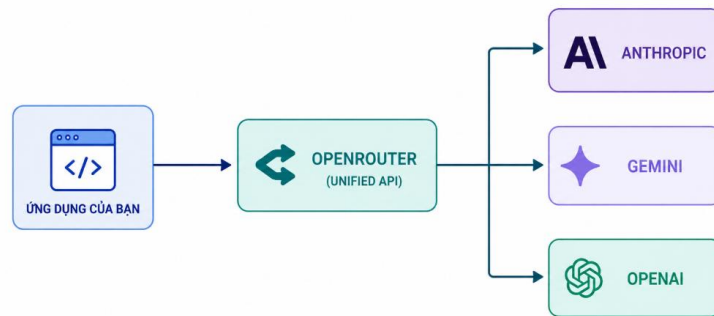
OpenRouter là nền tảng trung gian cho phép truy cập nhiều mô hình ngôn ngữ lớn thông qua một API thống nhất⁸. Thay vì tích hợp riêng từng nhà cung cấp AI, OpenRouter cung cấp một giao diện lập trình duy nhất giúp các ứng dụng dễ dàng chuyển đổi giữa nhiều mô hình khác nhau.

OpenRouter hiện hỗ trợ nhiều mô hình phổ biến như:

- GPT của OpenAI.
- Claude của Anthropic.
- Gemini của Google.
- Llama của Meta.
- DeepSeek.
- Mistral.

⁸ OpenRouter, "OpenRouter Documentation", Available: <https://openrouter.ai/docs>

Thông qua OpenRouter, nhà phát triển có thể lựa chọn mô hình phù hợp với yêu cầu về chi phí, tốc độ xử lý và chất lượng phản hồi mà không cần thay đổi kiến trúc ứng dụng.



Hình 2.11. Kiến trúc hoạt động của OpenRouter

Ưu điểm của mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn mang lại nhiều lợi ích trong các mô hình trí tuệ nhân tạo hiện đại như:

- Hiểu và sinh ngôn ngữ tự nhiên.
- Hỗ trợ hội thoại đa lượt.
- Tóm tắt và phân tích văn bản.
- Trả lời câu hỏi dựa trên ngữ cảnh.
- Hỗ trợ ra quyết định và tư vấn.

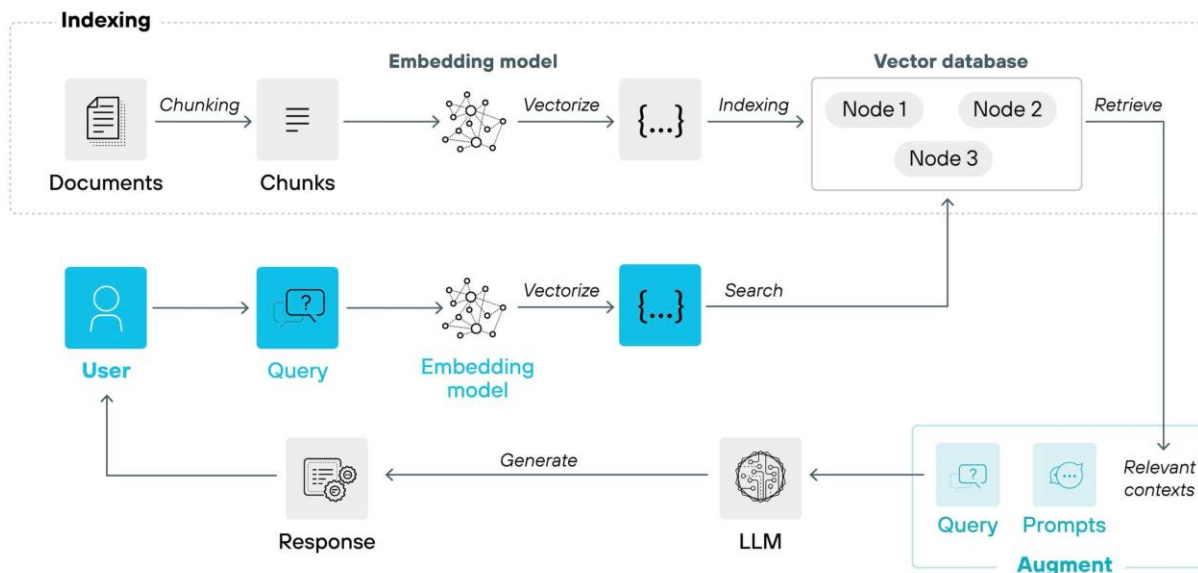
Nhờ khả năng xử lý ngôn ngữ tự nhiên mạnh mẽ, LLM đang trở thành thành phần cốt lõi trong nhiều mô hình AI thế hệ mới.

2.6. Kỹ thuật Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) là phương pháp kết hợp giữa cơ chế truy xuất thông tin (Retrieval) và mô hình ngôn ngữ lớn (Generation) nhằm nâng cao chất lượng phản hồi của mô hình trí tuệ nhân tạo. [5]

Trong các mô hình ngôn ngữ lớn truyền thống, câu trả lời được sinh ra dựa trên dữ liệu đã được sử dụng trong quá trình huấn luyện. Điều này dẫn đến một số hạn chế như thông tin lỗi thời, thiếu dữ liệu chuyên ngành hoặc xuất hiện hiện tượng Hallucination khi mô hình tạo ra các thông tin không chính xác.

RAG được đề xuất nhằm khắc phục các hạn chế trên bằng cách bổ sung cơ chế truy xuất tri thức từ các nguồn dữ liệu bên ngoài trước khi đưa thông tin vào mô hình ngôn ngữ lớn để sinh phản hồi.



Hình 2.12. Kiến trúc tổng quát của Retrieval-Augmented Generation

2.6.1. Nguyên lý hoạt động của RAG

Quá trình hoạt động của RAG thường bao gồm ba giai đoạn chính :

- Tiếp nhận câu hỏi từ người dùng.
- Truy xuất các tài liệu liên quan từ cơ sở tri thức.
- Kết hợp dữ liệu truy xuất với câu hỏi để tạo Prompt cho mô hình ngôn ngữ lớn.

Khi người dùng gửi truy vấn, mô hình sẽ tìm kiếm các tài liệu có mức độ liên quan cao trong cơ sở dữ liệu tri thức. Các tài liệu này được đưa vào Prompt cùng với câu hỏi ban đầu trước khi gửi đến mô hình ngôn ngữ lớn.

Nhờ sử dụng thêm dữ liệu từ nguồn bên ngoài, phản hồi được tạo ra có độ chính xác cao hơn và bám sát dữ liệu thực tế hơn so với phương pháp chỉ sử dụng LLM đơn thuần.

Ưu điểm của RAG

RAG mang lại nhiều lợi ích quan trọng cho các mô hình AI hiện đại :

- Giảm hiện tượng Hallucination.
- Tăng độ chính xác của câu trả lời.
- Khai thác được tri thức riêng của tổ chức hoặc doanh nghiệp.
- Dễ dàng cập nhật dữ liệu mà không cần huấn luyện lại mô hình.
- Hỗ trợ truy xuất nguồn tham khảo cho câu trả lời.

So với việc tinh chỉnh (Fine-tuning) mô hình ngôn ngữ lớn, RAG có chi phí triển khai thấp hơn và khả năng cập nhật tri thức nhanh hơn.

2.6.2. Các thành phần chính của RAG

Một hệ thống RAG thường bao gồm các thành phần chính sau:

Bộ truy xuất dữ liệu (Retriever)

Retriever có nhiệm vụ tìm kiếm và lựa chọn các tài liệu liên quan đến câu hỏi của người dùng từ cơ sở tri thức. Quá trình này có thể sử dụng tìm kiếm từ khóa, tìm kiếm ngữ nghĩa hoặc kết hợp nhiều phương pháp khác nhau.

Kho tri thức (Knowledge Base)

Knowledge Base là nơi lưu trữ các tài liệu, văn bản hoặc dữ liệu chuyên ngành phục vụ quá trình truy xuất. Các tài liệu thường được chia nhỏ thành nhiều đoạn (Chunk) để tăng hiệu quả tìm kiếm.

Mô hình ngôn ngữ lớn (Generator)

Generator là mô hình ngôn ngữ lớn chịu trách nhiệm sinh phản hồi cuối cùng dựa trên câu hỏi của người dùng và dữ liệu được Retriever cung cấp.



Hình 2.13. Các thành phần chính của RAG

Vai trò của RAG trong các mô hình AI hiện đại

Sự xuất hiện của RAG đã tạo ra một hướng tiếp cận mới trong việc xây dựng các mô hình hỏi đáp thông minh. Thay vì phụ thuộc hoàn toàn vào dữ liệu huấn luyện của mô hình

ngôn ngữ lớn, RAG cho phép kết hợp giữa khả năng suy luận của LLM và dữ liệu chuyên biệt từ các nguồn tri thức bên ngoài.

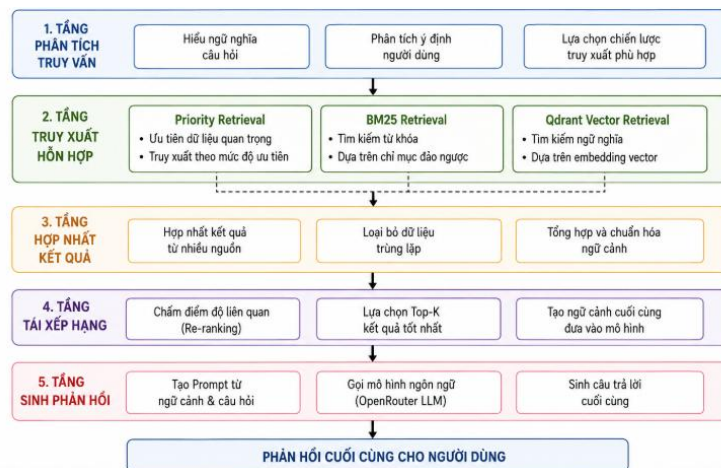
Nhờ đó, RAG hiện đang được sử dụng rộng rãi trong các hệ thống trợ lý ảo, chatbot doanh nghiệp, công cụ tìm kiếm thông minh và các nền tảng tư vấn kỹ thuật dựa trên trí tuệ nhân tạo.

2.7. Kỹ thuật truy xuất thông tin trong Hybrid RAG

Mặc dù Retrieval - Augmented Generation (RAG) giúp nâng cao chất lượng phản hồi của mô hình ngôn ngữ lớn thông qua việc truy xuất tri thức từ các nguồn dữ liệu bên ngoài, tuy nhiên hiệu quả của mô hình vẫn phụ thuộc đáng kể vào chất lượng của cơ chế tìm kiếm tài liệu.

Trong thực tế, các phương pháp tìm kiếm dựa trên từ khóa có khả năng xác định chính xác các tài liệu chứa cụm từ cần tìm nhưng thường gặp khó khăn trong việc hiểu ngữ nghĩa của câu truy vấn. Ngược lại, các phương pháp tìm kiếm ngữ nghĩa dựa trên Vector Embedding có khả năng hiểu ý nghĩa của văn bản nhưng đôi khi bỏ sót các từ khóa quan trọng.

Hybrid RAG là phương pháp kết hợp nhiều kỹ thuật truy xuất khác nhau nhằm tận dụng ưu điểm của từng phương pháp và nâng cao hiệu quả tìm kiếm tri thức trước khi đưa dữ liệu vào mô hình ngôn ngữ lớn.



Hình 2.14. Kiến trúc tổng quát của Hybrid RAG

2.7.1. Truy xuất thông tin (Information Retrieval)

Truy xuất thông tin (Information Truy xuất - IR) là lĩnh vực nghiên cứu các phương pháp tìm kiếm, thu thập và xếp hạng các tài liệu có liên quan đến nhu cầu thông tin của người

dùng từ một tập dữ liệu lớn. Mục tiêu của mô hình là truy xuất thông tin là tìm ra những tài liệu phù hợp nhất với truy vấn đầu vào thay vì chỉ tìm kiếm dựa trên sự trùng khớp chính xác của từ khóa.

Trong một hệ thống truy xuất thông tin điển hình, dữ liệu được lưu trữ dưới dạng tập hợp các tài liệu. Khi người dùng gửi truy vấn, hệ thống sẽ tiến hành phân tích nội dung truy vấn, so sánh với các tài liệu trong cơ sở dữ liệu và tính toán độ liên quan giữa truy vấn và từng tài liệu. Kết quả cuối cùng được sắp xếp theo thứ tự mức độ liên quan giảm dần để trả về cho người dùng.

Các hệ thống truy xuất thông tin hiện đại thường sử dụng nhiều kỹ thuật khác nhau như mô hình không gian vector (Vector Space Model), thuật toán BM25, truy xuất ngữ nghĩa dựa trên vector nhúng (Embedding) và các phương pháp kết hợp nhằm nâng cao độ chính xác của kết quả tìm kiếm. Việc đánh giá chất lượng của hệ thống truy xuất thông tin thường dựa trên các tiêu chí như độ chính xác (Precision), độ bao phủ (Recall) và độ liên quan của kết quả trả về.

Trong các mô hình ứng dụng trí tuệ nhân tạo hiện nay, truy xuất thông tin đóng vai trò quan trọng trong việc cung cấp tri thức cho mô hình ngôn ngữ lớn. Thay vì chỉ dựa vào kiến thức đã được huấn luyện trước, mô hình có thể truy xuất dữ liệu từ các nguồn tri thức bên ngoài để tạo ra các câu trả lời chính xác và cập nhật hơn. Đây cũng là nền tảng cho sự ra đời của các hệ thống Retrieval-Augmented Generation (RAG) được sử dụng rộng rãi trong các ứng dụng hỏi đáp thông minh hiện nay.

2.7.2. Thuật toán BM25 Retrieval

BM25 (Best Matching 25) là một thuật toán truy xuất thông tin dựa trên mô hình Bag-of-Words, được sử dụng rộng rãi trong các hệ thống tìm kiếm tài liệu và truy xuất tri thức. Thuật toán này đánh giá mức độ liên quan giữa một truy vấn và các tài liệu trong tập dữ liệu dựa trên tần suất xuất hiện của các từ khóa truy vấn trong từng tài liệu, mà không xét đến vị trí hay khoảng cách giữa các từ trong văn bản. [6]

Ý tưởng chính của BM25 là một tài liệu sẽ được xem là có độ liên quan cao nếu chứa nhiều từ khóa của truy vấn và các từ khóa đó xuất hiện với tần suất phù hợp. Đồng thời, thuật toán cũng xem xét độ dài của tài liệu nhằm tránh trường hợp các tài liệu dài có lợi thế chỉ vì chứa nhiều từ hơn.

Điểm số BM25 của một tài liệu DDD đối với truy vấn QQQ được tính theo công thức:

$$score(D, Q) = \sum_{i=1}^n IDF(q_i) \cdot \frac{f(q_i, D)(k_1+1)}{f(q_i, D) + k_1(1-b + b \frac{|D|}{avgdl})} \quad (2.1)$$

Trong đó:

- q_i : từ khóa thứ i trong truy vấn.
- $f(q_i, D)$: số lần xuất hiện của từ khóa q_i trong tài liệu D .
- $|D|$: độ dài của tài liệu D (tính theo số từ).
- $avgdl$: độ dài trung bình của tất cả tài liệu trong tập dữ liệu.
- k_1 : tham số điều chỉnh ảnh hưởng của tần suất từ khóa, thường nằm trong khoảng từ 1.2 đến 2.0.
- b : tham số chuẩn hóa độ dài tài liệu, thường được chọn bằng 0.75.
- $IDF(q_i)$: trọng số nghịch đảo tần suất xuất hiện của từ khóa trong toàn bộ tập tài liệu.

Giá trị IDF được tính theo công thức:

$$IDF(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right) \quad (2.2)$$

Trong đó:

- N : tổng số tài liệu trong tập dữ liệu.
- $n(q_i)$: số lượng tài liệu có chứa từ khóa q_i .

Thành phần IDF giúp đánh giá mức độ quan trọng của một từ khóa. Những từ xuất hiện trong nhiều tài liệu sẽ có giá trị IDF thấp vì khả năng phân biệt thấp. Ngược lại, những từ chỉ xuất hiện trong một số ít tài liệu sẽ có giá trị IDF cao hơn và đóng góp nhiều hơn vào điểm số cuối cùng.

Thuật toán BM25 có nhiều ưu điểm như đơn giản, hiệu quả, tốc độ xử lý nhanh và không yêu cầu quá trình huấn luyện mô hình. Vì vậy, BM25 thường được sử dụng làm tầng truy xuất từ khóa trong các hệ thống Hybrid Truy xuất và Hybrid RAG. Trong mô hình AI Backend của đề tài, BM25 được sử dụng để tìm kiếm các tài liệu có chứa trực tiếp các từ khóa liên quan đến câu hỏi của người dùng trước khi kết hợp với phương pháp tìm kiếm ngữ nghĩa dựa trên Vector Database nhằm nâng cao độ chính xác của quá trình truy xuất tri thức.

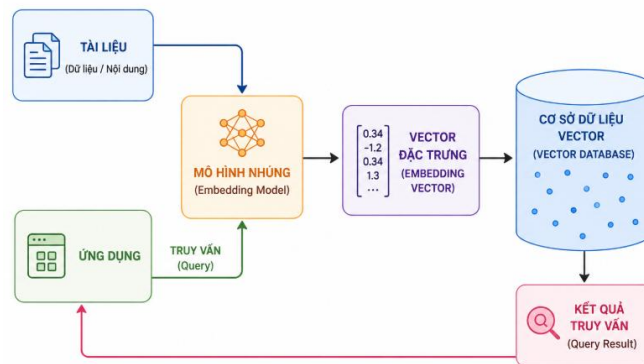
Ưu điểm của BM25 là tốc độ xử lý nhanh, khả năng tìm kiếm chính xác các tài liệu chứa từ khóa cần thiết và không yêu cầu quá trình huấn luyện mô hình.

2.7.3. Truy xuất theo mô hình vector (Vector Retrieval)

Vector Truy xuất là phương pháp tìm kiếm ngữ nghĩa dựa trên Vector Embedding. Trong phương pháp này, văn bản được chuyển đổi thành các vector số học có nhiều chiều thông qua các mô hình Embedding.

Các vector này được lưu trữ trong cơ sở dữ liệu vector. Khi người dùng gửi truy vấn, câu hỏi cũng được chuyển đổi thành vector và mô hình sẽ tìm kiếm các vector có khoảng cách gần nhất với vector truy vấn.

Khác với BM25, Vector Truy xuất không chỉ dựa vào từ khóa mà còn có khả năng hiểu ngữ nghĩa của câu văn. Điều này giúp mô hình tìm được các tài liệu liên quan ngay cả khi chúng không chứa chính xác từ khóa xuất hiện trong truy vấn.



Hình 2.15. Quy trình tìm kiếm ngữ nghĩa bằng Vector Retrieval

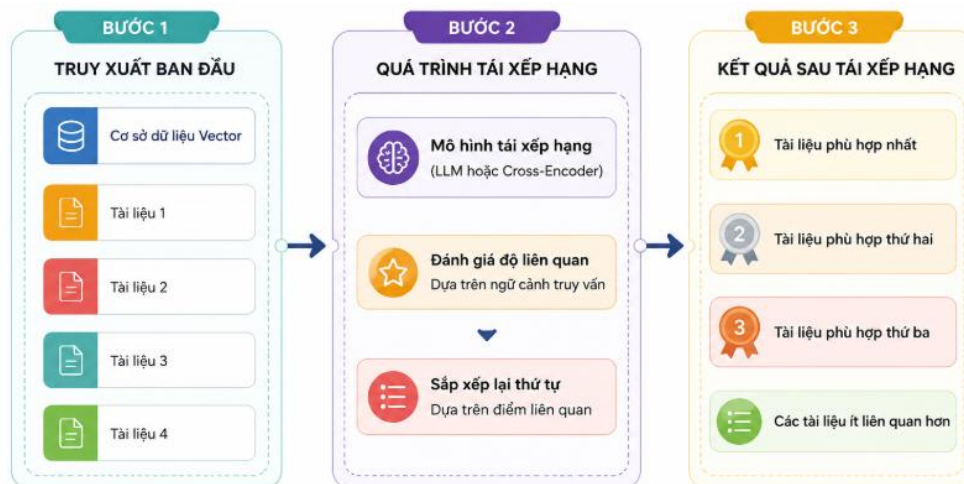
2.7.4. Reranking

Sau khi nhận được kết quả từ nhiều cơ chế truy xuất khác nhau, mô hình thường sử dụng bước Reranking nhằm đánh giá lại mức độ liên quan của từng tài liệu trước khi lựa chọn các tài liệu tốt nhất.

Quá trình Reranking giúp:

- Loại bỏ các tài liệu không liên quan.
- Tăng độ chính xác của kết quả tìm kiếm.
- Nâng cao chất lượng dữ liệu đầu vào cho mô hình ngôn ngữ lớn.

Thông thường, chỉ một số lượng nhỏ tài liệu có điểm số cao nhất sẽ được lựa chọn để đưa vào Prompt.



Hình 2.16. Quy trình Reranking kết quả truy xuất

2.7.4. Truy xuất kết hợp (Hybrid Retrieval)

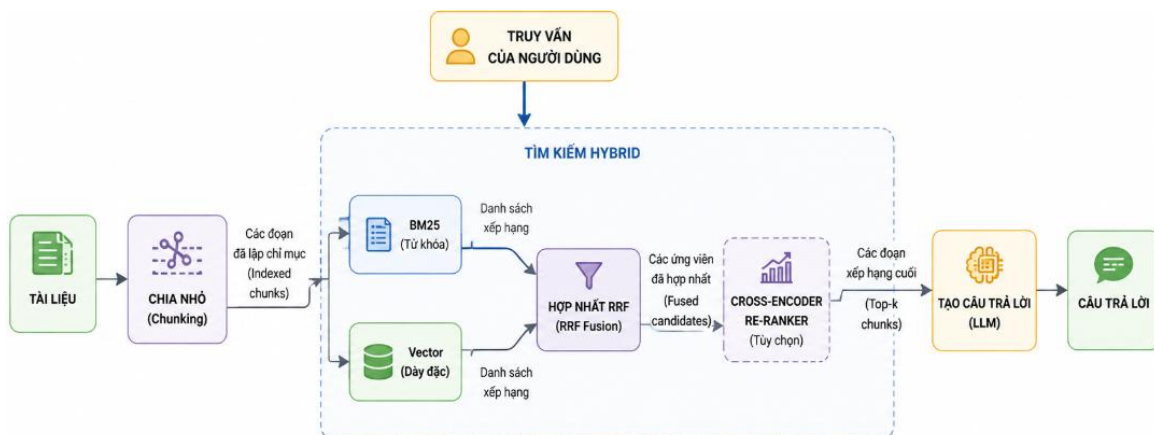
Hybrid Truy xuất là cơ chế kết hợp giữa BM25 Truy xuất và Vector Truy xuất nhằm tận dụng đồng thời khả năng tìm kiếm theo từ khóa và khả năng tìm kiếm ngữ nghĩa.

Trong mô hình này, truy vấn của người dùng được gửi đồng thời đến hai hệ thống tìm kiếm:

- BM25 Search.
- Vector Search.

Kết quả từ hai hệ thống được hợp nhất và đánh giá lại thông qua bước Reranking. Các tài liệu có mức độ liên quan cao nhất sẽ được sử dụng làm tri thức đầu vào cho mô hình ngôn ngữ lớn.

Cách tiếp cận này giúp cải thiện đáng kể khả năng truy xuất thông tin, đặc biệt trong các hệ thống hỏi đáp chuyên ngành và các nền tảng sử dụng Retrieval-Augmented Generation.



Hình 2.17. Mô hình Hybrid Truy xuất kết hợp BM25 và Vector Search

Ưu điểm của Hybrid RAG

Hybrid RAG mang lại nhiều ưu điểm so với các hệ thống RAG truyền thống chỉ sử dụng một cơ chế truy xuất dữ liệu:

- Tăng độ chính xác của kết quả tìm kiếm.
- Giảm khả năng bỏ sót tài liệu liên quan.
- Kết hợp hiệu quả giữa tìm kiếm từ khóa và tìm kiếm ngữ nghĩa.
- Nâng cao chất lượng phản hồi của mô hình ngôn ngữ lớn.
- Phù hợp với các hệ thống tri thức chuyên ngành có khối lượng dữ liệu lớn.

Nhờ những ưu điểm trên, Hybrid RAG đang trở thành hướng tiếp cận phổ biến trong các hệ thống hỏi đáp thông minh và các nền tảng AI hiện đại.

2.8. Cơ sở dữ liệu vector Qdrant

2.8.1. Khái niệm Vector Database

Sự phát triển của các mô hình trí tuệ nhân tạo và các hệ thống tìm kiếm ngữ nghĩa đã làm xuất hiện nhu cầu lưu trữ và truy xuất dữ liệu dưới dạng vector. Khác với cơ sở dữ liệu truyền thống lưu trữ dữ liệu dưới dạng bảng hoặc tài liệu, Vector Database được thiết kế để lưu trữ các vector có số chiều lớn và thực hiện tìm kiếm dựa trên mức độ tương đồng giữa các vector⁹.

Trong các hệ thống AI hiện đại, văn bản, hình ảnh hoặc âm thanh thường được chuyển đổi thành các vector thông qua các mô hình Embedding. Những vector này biểu diễn đặc trưng ngữ nghĩa của dữ liệu và được sử dụng để thực hiện các tác vụ tìm kiếm ngữ nghĩa, phân loại và truy xuất tri thức.

Vector Database đóng vai trò quan trọng trong các hệ thống Retrieval-Augmented Generation (RAG), nơi cần tìm kiếm nhanh các tài liệu có nội dung tương đồng với câu truy vấn của người dùng.

2.8.2. Embedding và biểu diễn dữ liệu bằng Vector

Embedding là quá trình chuyển đổi dữ liệu từ dạng văn bản hoặc dữ liệu phi cấu trúc sang dạng vector số học có nhiều chiều.

⁹ Qdrant, "Qdrant Documentation", Available: <https://qdrant.tech/documentation/>

Mỗi vector biểu diễn các đặc trưng ngữ nghĩa của dữ liệu gốc. Các dữ liệu có nội dung tương tự nhau thường được biểu diễn bởi các vector nằm gần nhau trong không gian vector.

Ví dụ, các câu: ESP32 hỗ trợ WiFi. Vi điều khiển ESP32 có khả năng kết nối mạng không dây.

Có thể được biểu diễn bằng các vector khác nhau nhưng vẫn nằm gần nhau trong không gian vector do mang ý nghĩa tương đồng.

Quá trình Embedding là nền tảng của các hệ thống tìm kiếm ngữ nghĩa hiện đại và đóng vai trò quan trọng trong việc xây dựng các hệ thống RAG.

2.8.3. Kỹ thuật tìm kiếm tương đồng Similarity Search

Similarity Search là kỹ thuật tìm kiếm các vector có mức độ tương đồng cao nhất với vector truy vấn.

Sau khi câu hỏi của người dùng được chuyển đổi thành vector, mô hình sẽ tính toán khoảng cách giữa vector truy vấn và các vector được lưu trong cơ sở dữ liệu. Những vector có khoảng cách nhỏ nhất sẽ được xem là có mức độ liên quan cao nhất.

Các độ đo tương đồng thường được sử dụng bao gồm:

- Cosine Similarity.
- Euclidean Distance.
- Dot Product.

Trong đó, Cosine Similarity là phương pháp được sử dụng phổ biến nhất do khả năng đánh giá sự tương đồng về hướng của các vector thay vì chỉ dựa trên độ lớn của chúng.

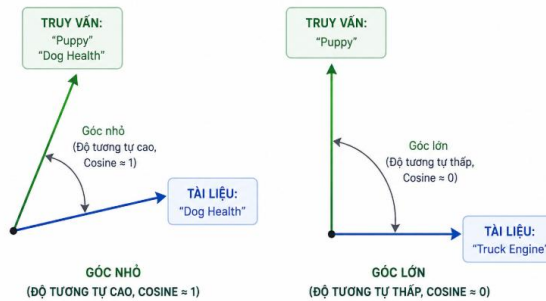
Công thức Cosine Similarity được xác định như sau:

$$\cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2.3)$$

Trong đó:

- (A) và (B) là hai vector embedding.
- (A.B) là tích vô hướng của hai vector.
- (|A|) và (|B|) là độ dài của các vector tương ứng.

Giá trị Cosine Similarity nằm trong khoảng từ -1 đến 1. Giá trị càng gần 1 thì mức độ tương đồng ngữ nghĩa giữa hai văn bản càng cao.



Hình 2.18. Nguyên lý tìm kiếm tương đồng bằng Cosine Similarity

2.8.4. Qdrant Vector Database

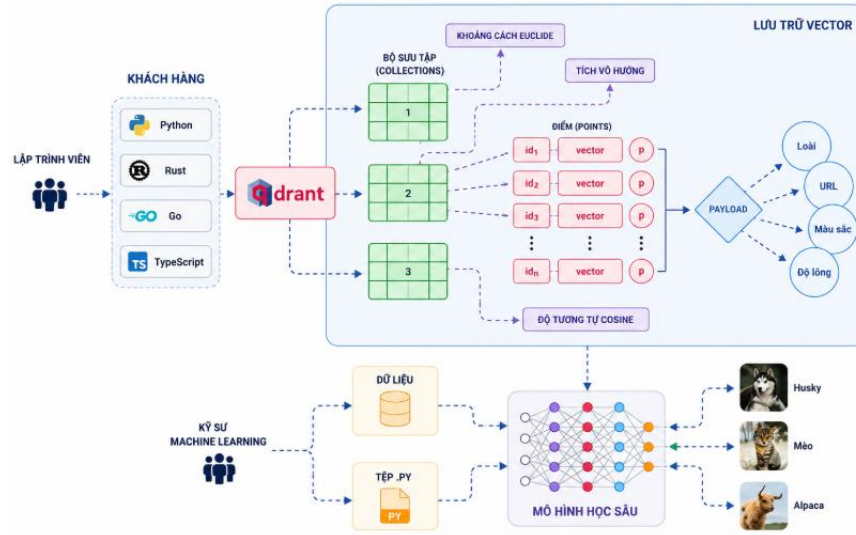
Qdrant là cơ sở dữ liệu vector mã nguồn mở được thiết kế chuyên biệt cho các mô hình trí tuệ nhân tạo và tìm kiếm ngữ nghĩa. Qdrant cho phép lưu trữ, lập chỉ mục và tìm kiếm các vector có số chiều lớn với hiệu năng cao.

Qdrant hỗ trợ nhiều tính năng quan trọng như:

- Vector Search.
- Hybrid Search.
- Metadata Filtering.
- Similarity Ranking.
- Distributed Deployment.

Nhờ kiến trúc được tối ưu hóa cho dữ liệu vector, Qdrant có khả năng xử lý hàng triệu vector trong thời gian ngắn mà vẫn duy trì hiệu quả tìm kiếm cao.

Ngoài ra, Qdrant còn hỗ trợ API REST và gRPC giúp việc tích hợp với các hệ thống AI và các ứng dụng Web trở nên thuận tiện hơn.



Hình 2.19. Kiến trúc tổng quan của Qdrant

Vai trò của Qdrant trong hệ thống Retrieval-Augmented Generation

Trong các hệ thống RAG, Qdrant đóng vai trò là kho lưu trữ vector cho các tài liệu đã được xử lý Embedding.

Quá trình hoạt động thường bao gồm các bước:

- Chuyển đổi tài liệu thành vector.
- Lưu vector vào Qdrant.
- Chuyển đổi câu truy vấn thành vector.
- Thực hiện Similarity Search.
- Trả về các tài liệu có độ tương đồng cao nhất.

Nhờ khả năng tìm kiếm ngữ nghĩa hiệu quả, Qdrant giúp nâng cao chất lượng truy xuất tri thức và cải thiện độ chính xác của phản hồi được tạo bởi mô hình ngôn ngữ lớn.

2.9. Mô hình Agent và Multi - Agent

2.9.1. Khái niệm Agent

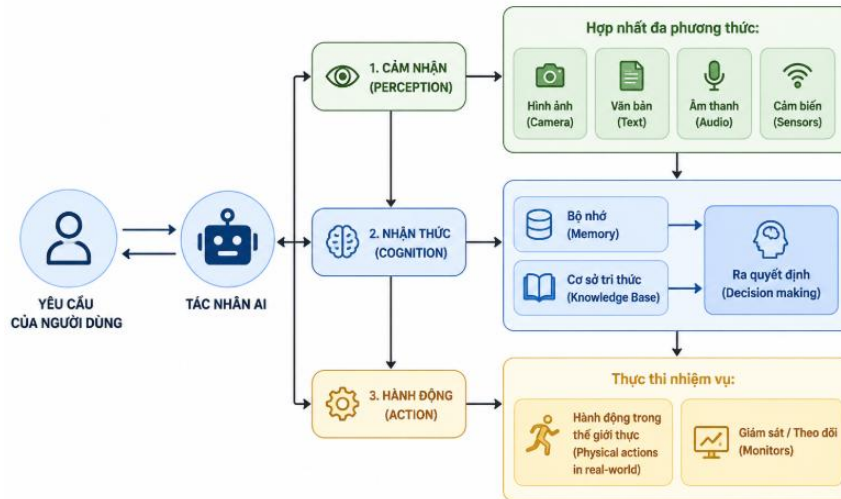
Trong lĩnh vực trí tuệ nhân tạo, Agent là một thực thể phần mềm có khả năng nhận thông tin từ môi trường, phân tích dữ liệu, đưa ra quyết định và thực hiện các hành động nhằm đạt được mục tiêu đã được xác định trước. [7]

Một Agent thông thường bao gồm ba thành phần chính:

- Khả năng tiếp nhận thông tin từ môi trường.
- Cơ chế xử lý và ra quyết định.

- Khả năng thực hiện hành động.

Tùy theo mức độ phức tạp, Agent có thể hoạt động dựa trên các luật cố định hoặc sử dụng các mô hình trí tuệ nhân tạo để đưa ra quyết định linh hoạt hơn.



Hình 2.20. Mô hình hoạt động cơ bản của Agent

2.9.2. Mô hình Multi - Agent

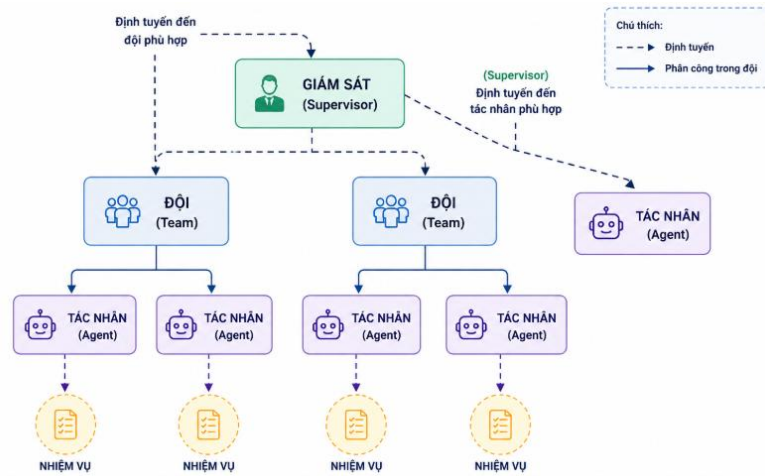
Multi - Agent System (MAS) là mô hình bao gồm nhiều Agent hoạt động đồng thời và phối hợp với nhau để giải quyết các nhiệm vụ phức tạp.

Thay vì sử dụng một Agent duy nhất để xử lý mọi yêu cầu, thì Multi - Agent được chia thành nhiều Agent chuyên trách cho từng nhóm nhiệm vụ khác nhau. Mỗi Agent có kiến thức và chức năng riêng, giúp nâng cao hiệu quả xử lý và khả năng mở rộng mô hình.

Trong Multi - Agent System, các Agent có thể:

- Trao đổi thông tin với nhau.
- Phối hợp thực hiện nhiệm vụ.
- Chia sẻ tri thức.
- Hỗ trợ lẫn nhau trong quá trình ra quyết định.

Nhờ đó, mô hình “IoT bán lẻ linh kiện điện tử thông minh tích hợp hybrid RAG và Multi - Agent AI” có thể xử lý các bài toán lớn và phức tạp hiệu quả hơn so với mô hình Agent đơn lẻ.



Hình 2.21. Kiến trúc tổng quát của Multi-Agent System

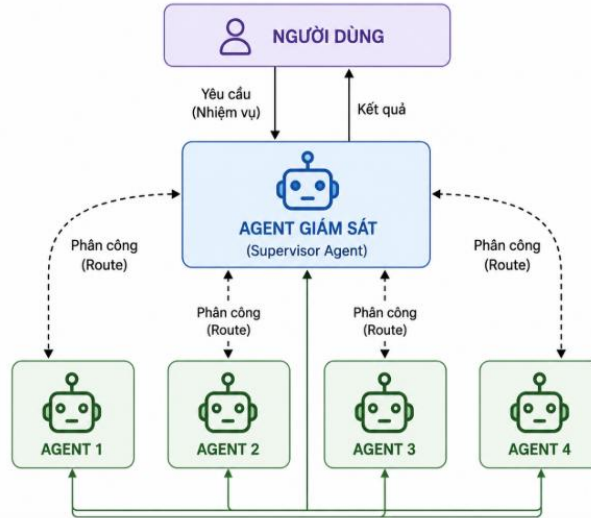
2.9.3. Agent điều phối (Supervisor Agent)

Supervisor-Based Multi-Agent là một kiến trúc phổ biến trong các hệ thống Agent hiện đại. Trong mô hình này, một Agent trung tâm đóng vai trò điều phối (Supervisor) sẽ chịu trách nhiệm phân tích yêu cầu và lựa chọn Agent phù hợp để xử lý nhiệm vụ.

Khi nhận được yêu cầu từ người dùng, Supervisor Agent sẽ:

- Phân tích nội dung yêu cầu.
- Xác định lĩnh vực cần xử lý.
- Chọn Agent chuyên trách phù hợp.
- Thu thập kết quả từ Agent.
- Trả về phản hồi cuối cùng.

Nhờ cơ chế điều phối tập trung, mô hình có thể quản lý hiệu quả số lượng lớn Agent và giảm sự phức tạp trong quá trình giao tiếp giữa các Agent.



Hình 2.22. Kiến trúc Supervisor-Based Multi-Agent

Ưu điểm của Multi-Agent System

Multi-Agent System mang lại nhiều lợi ích trong việc xây dựng các mô hình trí tuệ nhân tạo hiện đại.

Thứ nhất, hệ thống có khả năng mở rộng cao. Khi xuất hiện yêu cầu mới, có thể bổ sung thêm Agent chuyên trách mà không ảnh hưởng đến các Agent hiện có.

Thứ hai, việc phân chia nhiệm vụ giúp tăng tính chuyên môn hóa. Mỗi Agent chỉ tập trung vào một nhóm nghiệp vụ cụ thể, từ đó nâng cao chất lượng xử lý.

Thứ ba, kiến trúc Multi-Agent giúp giảm tải cho mô hình ngôn ngữ lớn bằng cách giới hạn phạm vi xử lý của từng Agent, góp phần cải thiện hiệu năng và giảm chi phí tính toán.

Ngoài ra, Multi-Agent System còn giúp tăng khả năng bảo trì và phát triển mô – đun (module) rời do các thành phần được tách biệt và hoạt động tương đối độc lập.

Xu hướng ứng dụng Multi-Agent trong các mô hình AI hiện đại

Cùng với sự phát triển của các mô hình ngôn ngữ lớn, Multi-Agent đang trở thành một trong những hướng nghiên cứu nổi bật trong lĩnh vực trí tuệ nhân tạo. Nhiều mô hình AI hiện nay sử dụng nhiều Agent chuyên biệt để thực hiện các nhiệm vụ như tìm kiếm thông tin, lập kế hoạch, phân tích dữ liệu và hỗ trợ ra quyết định.

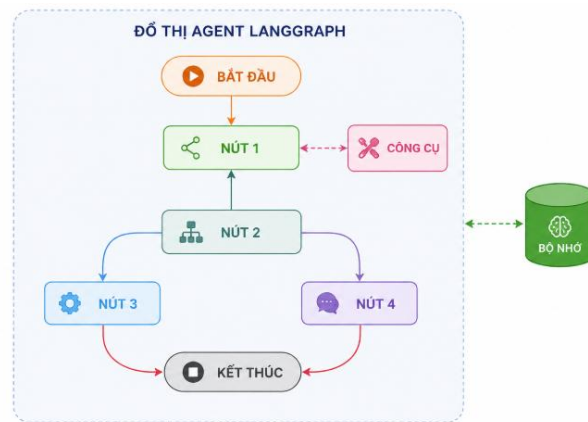
Việc kết hợp giữa Multi-Agent và Large Language Model giúp tận dụng đồng thời khả năng suy luận của mô hình ngôn ngữ lớn và tính chuyên môn hóa của từng Agent, từ đó nâng cao hiệu quả xử lý các bài toán thực tế có độ phức tạp cao.

2.10. Framework LangGraph

LangGraph là một Framework mã nguồn mở được phát triển nhằm hỗ trợ xây dựng các mô hình trí tuệ nhân tạo có khả năng xử lý theo luồng công việc (Workflow) và quản lý trạng thái (State Management). LangGraph được xây dựng dựa trên nền tảng LangChain và được thiết kế để hỗ trợ các ứng dụng sử dụng Large Language Model theo mô hình đồ thị trạng thái (State Graph).

Khác với các ứng dụng AI truyền thống chỉ thực hiện một lần gọi đến mô hình ngôn ngữ lớn, LangGraph cho phép xây dựng các luồng xử lý gồm nhiều bước liên tiếp, nhiều nhánh quyết định và nhiều Agent phối hợp với nhau trong cùng một hệ thống¹⁰.

Nhờ khả năng quản lý trạng thái và điều phối luồng xử lý, LangGraph hiện đang được sử dụng rộng rãi trong các hệ thống Multi-Agent, AI Assistant và các nền tảng Retrieval-Augmented Generation hiện đại.



Hình 2.23. Đồ thị Framework LangGraph

STATE GRAPH

State Graph là thành phần cốt lõi của LangGraph. Trong mô hình này, toàn bộ dữ liệu được quản lý thông qua một trạng thái chung (State) và được truyền xuyên suốt giữa các Node trong quá trình xử lý.

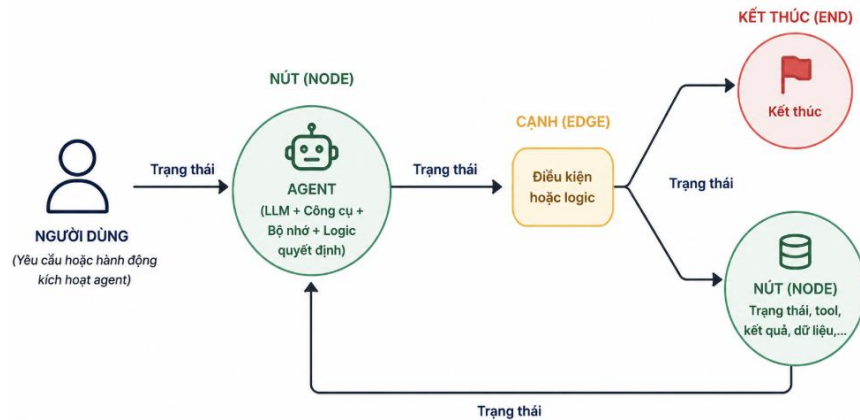
State có thể chứa nhiều loại thông tin khác nhau như:

- Câu hỏi của người dùng.
- Dữ liệu truy xuất.

¹⁰ LangChain, "LangGraph Documentation", Available: <https://langchain-ai.github.io/langgraph/>

- Kết quả trung gian.
- Thông tin Agent được lựa chọn.
- Phản hồi cuối cùng.

Mỗi Node có thể đọc hoặc cập nhật State trước khi chuyển sang bước xử lý tiếp theo. Nhờ đó, toàn bộ Workflow có thể duy trì ngữ cảnh và trạng thái trong suốt quá trình hoạt động.



Hình 2.24. Mô hình State Graph trong LangGraph

NODE VÀ EDGE

Trong LangGraph, Workflow được xây dựng từ hai thành phần chính là Node và Edge.

Node

Node đại diện cho một bước xử lý cụ thể trong Workflow. Mỗi Node đảm nhận một nhiệm vụ riêng biệt như:

- Phân loại câu hỏi.
- Truy xuất tri thức.
- Đánh giá dữ liệu.
- Tìm kiếm Web.
- Sinh phản hồi.

Sau khi hoàn thành nhiệm vụ, Node sẽ cập nhật State và chuyển dữ liệu sang Node tiếp theo.

Edge

Edge là các liên kết giữa các Node trong Workflow. Edge xác định hướng di chuyển của dữ liệu và thứ tự thực hiện các bước xử lý.

LangGraph hỗ trợ hai loại Edge:

- Sequential Edge (luồng tuần tự).
- Conditional Edge (luồng có điều kiện).

Conditional Edge cho phép Workflow rẽ nhánh dựa trên kết quả xử lý của Node trước đó, giúp hệ thống có khả năng đưa ra quyết định linh hoạt hơn.

Workflow điều kiện trong LangGraph

Một trong những ưu điểm nổi bật của LangGraph là khả năng xây dựng Workflow có điều kiện. Thay vì luôn thực hiện theo một luồng cố định, hệ thống có thể lựa chọn các nhánh xử lý khác nhau dựa trên kết quả của các bước trước đó.

Ví dụ, sau khi thực hiện truy xuất dữ liệu, hệ thống có thể đánh giá mức độ liên quan của kết quả. Nếu dữ liệu đạt yêu cầu, Workflow sẽ chuyển sang bước sinh phản hồi. Ngược lại, hệ thống có thể thực hiện tìm kiếm bổ sung hoặc chuyển sang một quy trình xử lý khác. Khả năng này giúp nâng cao tính linh hoạt và hiệu quả của các mô hình AI phức tạp.

Ưu điểm của LangGraph

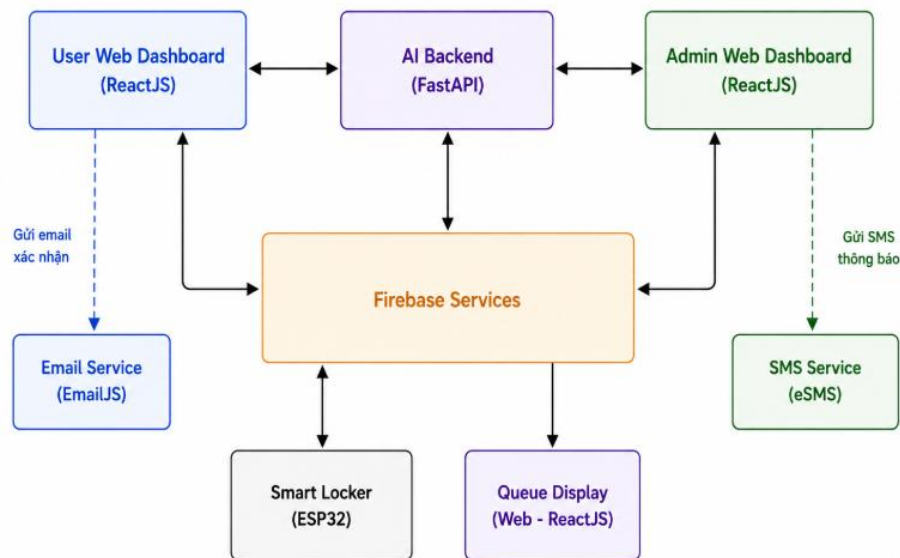
LangGraph mang lại nhiều lợi ích trong việc xây dựng các mô hình AI hiện đại:

- Hỗ trợ quản lý trạng thái xuyên suốt quá trình xử lý.
- Dễ dàng xây dựng Workflow nhiều bước.
- Hỗ trợ Workflow có điều kiện.
- Hỗ trợ kiến trúc Multi-Agent.
- Dễ dàng tích hợp với LangChain và các mô hình ngôn ngữ lớn.
- Tăng khả năng mở rộng và bảo trì.

Nhờ các đặc điểm trên, LangGraph đang trở thành một trong những Framework phổ biến trong việc xây dựng các kiến trúc Agentic AI và Retrieval-Augmented Generation.

CHƯƠNG 3: THIẾT KẾ VÀ TRIỂN KHAI MÔ HÌNH

3.1. Kiến trúc tổng quan mô hình



Hình 3.1. Kiến trúc tổng quan mô hình

Sơ đồ kiến trúc tổng thể mô hình AIoT Smart Retail thể hiện các thành phần chính và mối liên kết giữa giao diện người dùng, nền tảng Firebase, AI Backend, thiết bị Smart Locker và các dịch vụ thông báo.

Trong mô hình, User Web Dashboard và Admin Web Dashboard được xây dựng bằng ReactJS, đóng vai trò là giao diện thao tác chính cho người dùng và quản trị viên. Người dùng có thể xem sản phẩm, đặt hàng, theo dõi trạng thái đơn hàng và nhận thông báo. Quản trị viên có thể quản lý sản phẩm, quản lý kho, xử lý đơn hàng, theo dõi trạng thái và gửi thông báo đến người dùng.

Firebase Services đóng vai trò trung tâm lưu trữ và đồng bộ dữ liệu của toàn mô hình. Các dashboard web đọc và ghi dữ liệu lên Firebase để cập nhật thông tin sản phẩm, đơn hàng, trạng thái hàng đợi và trạng thái tủ locker theo thời gian thực. Firebase cũng hỗ trợ xác thực người dùng thông qua Firebase Authentication và lưu trữ dữ liệu nghiệp vụ thông qua Firebase Realtime Database.

AI Backend được triển khai bằng FastAPI, đảm nhiệm vai trò xử lý các chức năng trí tuệ nhân tạo của mô hình. User Dashboard và Admin Dashboard có thể gửi yêu cầu đến AI Backend để được tư vấn linh kiện, phân tích tồn kho, kiểm tra tương thích hoặc đề xuất

phương án xử lý. AI Backend đồng thời giao tiếp hai chiều với Firebase để khai thác dữ liệu mô hình và phục vụ quá trình phân tích.

Smart Locker sử dụng ESP32 là khối thiết bị IoT. Khối này giao tiếp hai chiều với Firebase: ESP32 đọc lệnh điều khiển từ Firebase để mở tủ, đồng thời ghi ngược trạng thái tủ sau khi thao tác. Nhờ đó, trạng thái nhận hàng có thể được cập nhật và đồng bộ với giao diện quản trị cũng như giao diện người dùng.

Queue Display là màn hình web dùng để hiển thị thứ tự đơn hàng và trạng thái phục vụ. Khác với Smart Locker, khối này chỉ đọc dữ liệu từ Firebase Realtime Database để hiển thị thông tin theo thời gian thực, không tham gia ghi hoặc điều khiển dữ liệu.

Ngoài các khối chính, mô hình còn tích hợp dịch vụ thông báo Email và SMS. Email Service sử dụng EmailJS để gửi email xác nhận hoặc thông báo đến người dùng từ phía User Dashboard. SMS Service sử dụng eSMS để gửi tin nhắn thông báo từ phía Admin Dashboard. Hai dịch vụ này là các dịch vụ bên ngoài, không được kích hoạt trực tiếp bởi Firebase mà được gọi từ các dashboard tương ứng.

Nhìn chung, kiến trúc được tổ chức theo mô hình lấy Firebase làm trung tâm dữ liệu, trong đó các thành phần giao diện, AI Backend và thiết bị IoT cùng kết nối đến Firebase để đồng bộ dữ liệu. Cách tổ chức này giúp mô hình có khả năng cập nhật theo thời gian thực, dễ mở rộng và phù hợp với bài toán quản lý bán hàng linh kiện kết hợp AI và IoT.

3.2. Thiết kế giao diện

Frontend của AIoT Smart Retail được xây dựng bằng ReactJS nhằm cung cấp môi trường tương tác giữa người dùng và toàn bộ mô hình. Ngoài chức năng hiển thị dữ liệu, frontend còn đảm nhiệm việc kết nối với Firebase Realtime Database, Firebase Authentication, AI Backend, EmailJS và các thành phần phân cứng thông qua cơ sở dữ liệu thời gian thực.

Trong mô hình, frontend đóng vai trò là lớp giao tiếp trực tiếp với người dùng. Khách hàng có thể thực hiện các thao tác tìm kiếm sản phẩm, đặt hàng, theo dõi trạng thái đơn hàng và sử dụng AI Assistant. Quản trị viên có thể quản lý sản phẩm, tồn kho, đơn hàng, Smart Locker và Queue Display. Bên cạnh đó, mô hình còn xây dựng một màn hình hiển thị hàng chờ nhằm hỗ trợ quá trình giao nhận hàng hóa.

Các file frontend chính trong mô hình gồm:



Hình 3.2. Cấu trúc thư mục thư mục pages của mô hình

Bảng 3.1: Chức năng chi tiết của các file giao diện thuộc thư mục pages

File	Vai trò
UserHome.jsx	Trang giới thiệu và điều hướng đăng nhập
Login.jsx	Xử lý đăng nhập, đăng ký và phân quyền
UserDashboard.jsx	Giao diện khách hàng mua linh kiện
AdminDashboard.jsx	Giao diện quản trị kho, đơn hàng và AI
QueueDisplay.jsx	Màn hình hiển thị hàng chờ realtime

Cách tổ chức này giúp frontend được chia thành các nhóm giao diện rõ ràng. Mỗi file phụ trách một màn hình chức năng chính, giúp việc phát triển và bảo trì trở nên thuận tiện hơn.

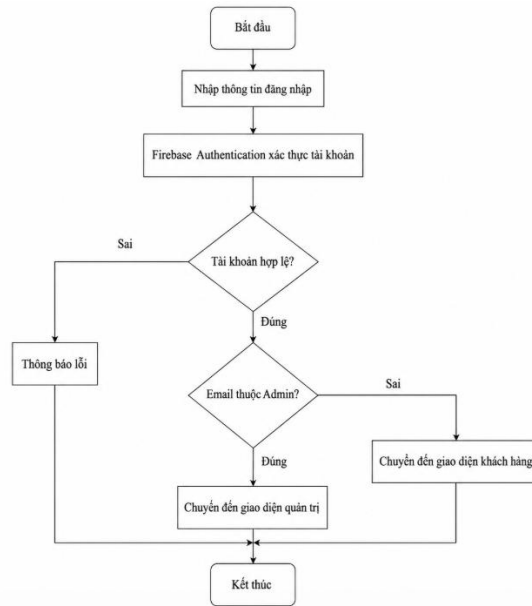
a. Thiết kế giao diện trang chủ và đăng nhập

Trang chủ UserHome là giao diện đầu tiên mà người dùng tiếp cận khi truy cập web. Giao diện này có nhiệm vụ giới thiệu AIoT Smart Retail, cung cấp các thông tin tổng quan về chức năng và điều hướng người dùng đến các phân hệ chính.

Các thành phần chính của trang chủ bao gồm thanh điều hướng, khu vực giới thiệu mô hình, khu vực mô tả công nghệ sử dụng và các nút truy cập nhanh đến giao diện đăng nhập hoặc giao diện khách hàng.

Sau khi người dùng lựa chọn đăng nhập, chuyển đến giao diện Login. Chức năng đăng nhập được xây dựng dựa trên Firebase Authentication nhằm đảm bảo tính bảo mật và khả năng quản lý người dùng tập trung.

Sau khi xác thực thành công, sẽ kiểm tra vai trò của tài khoản. Nếu tài khoản thuộc nhóm quản trị viên, người dùng sẽ được chuyển đến Admin Dashboard. Ngược lại, sẽ chuyển sang User Dashboard.



Hình 3.3. Lưu đồ chức năng đăng nhập và phân quyền người dùng

Việc phân quyền theo vai trò giúp mô hình tách biệt rõ ràng giữa người dùng thông thường và Admin. Người dùng chỉ được truy cập các chức năng mua hàng, xem sản phẩm và sử dụng AI tư vấn, trong khi Admin có quyền quản lý sản phẩm, đơn hàng, tồn kho, hàng đợi và Smart Locker.

b. Thiết kế giao diện khách hàng

Giao diện khách hàng được xây dựng nhằm hỗ trợ người dùng tra cứu linh kiện điện tử, quản lý giỏ hàng, đặt hàng trực tuyến và nhận hỗ trợ kỹ thuật từ AI Assistant.

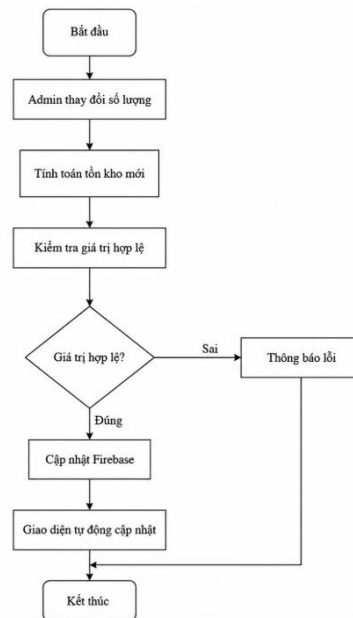
Dữ liệu sản phẩm được đọc trực tiếp từ node products trên Firebase Realtime Database. Thông qua cơ chế đồng bộ thời gian thực, mọi thay đổi về thông tin sản phẩm hoặc số lượng tồn kho đều được cập nhật ngay trên giao diện mà không cần tải lại trang.

Các chức năng chính của giao diện khách hàng bao gồm:

- Hiển thị danh sách sản phẩm.
- Tìm kiếm và lọc sản phẩm.
- Quản lý giỏ hàng.
- Đặt hàng trực tuyến.
- Theo dõi trạng thái đơn hàng.
- Tư vấn kỹ thuật bằng AI Assistant.

Khi người dùng thêm sản phẩm vào giỏ hàng, sẽ kiểm tra số lượng tồn kho trước khi cập nhật dữ liệu. Nếu số lượng tồn kho không đáp ứng yêu cầu, thì hiển thị thông báo tương ứng. Trong trường hợp hợp lệ, dữ liệu giỏ hàng sẽ được lưu lên Firebase để đảm bảo tính liên tục trong quá trình sử dụng.

Luồng xử lý thêm sản phẩm vào giỏ hàng được trình bày trong Hình 3.4.



Hình 3.4. Lưu đồ chức năng xử lý giỏ hàng

Ngoài chức năng mua sắm, User Dashboard còn tích hợp AI Assistant nhằm hỗ trợ khách hàng trong quá trình lựa chọn linh kiện và xây dựng dự án IoT. Khi người dùng gửi câu hỏi,

dữ liệu được chuyển đến AI Backend để thực hiện định tuyến Agent, truy xuất tri thức và sinh phản hồi.

Nhờ đó, giao diện khách hàng không chỉ đóng vai trò là một Website bán lẻ hiện đại mà còn là một nền tảng hỗ trợ tư vấn kỹ thuật thông minh.

c. Thiết kế giao diện quản trị

Giao diện quản trị được xây dựng nhằm hỗ trợ quản lý toàn bộ hoạt động của mô hình từ sản phẩm, tồn kho, đơn hàng đến Smart Locker và Queue Display.

Admin Dashboard được kết nối trực tiếp với Firebase Realtime Database để theo dõi và cập nhật dữ liệu theo thời gian thực. Mọi thay đổi phát sinh từ phía khách hàng hoặc phần cứng đều được đồng bộ ngay lập tức lên giao diện quản trị.

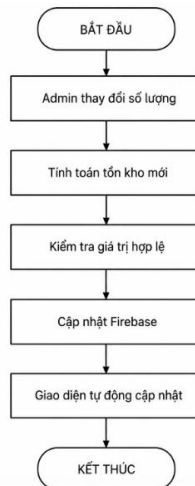
Các chức năng chính của Admin Dashboard bao gồm:

- Quản lý sản phẩm.
- Quản lý tồn kho.
- Theo dõi đơn hàng.
- Xử lý đơn hàng.
- Quản lý Smart Locker.
- Điều khiển Queue Display.
- Phân tích dữ liệu bằng AI Assistant.

Đối với quản lý tồn kho, Admin có thể thêm mới sản phẩm, cập nhật thông tin hoặc điều chỉnh số lượng tồn kho. Dữ liệu luôn được kiểm tra tính hợp lệ của dữ liệu nhằm đảm bảo sự nhất quán trong cơ sở dữ liệu.

Khi khách hàng hoàn tất đặt hàng, đơn hàng sẽ xuất hiện trong danh sách chờ xử lý. Admin có thể xác nhận đơn hàng, chuẩn bị sản phẩm và đưa hàng vào Smart Locker tương ứng để khách hàng nhận hàng.

Luồng xử lý đơn hàng được trình bày trong Hình 3.5.



Hình 3.5. Luồng xử lý đơn hàng

Ngoài các chức năng quản lý truyền thống, giao diện quản trị còn tích hợp AI Assistant phục vụ phân tích tồn kho và hỗ trợ ra quyết định. AI có khả năng đánh giá tình trạng kho hàng, xác định sản phẩm bán chạy hoặc cảnh báo các sản phẩm có nguy cơ hết hàng. Việc tích hợp AI giúp Admin khai thác hiệu quả dữ liệu vận hành thay vì chỉ thực hiện các thao tác quản lý thủ công.

d. Thiết kế màn hình Queue Display

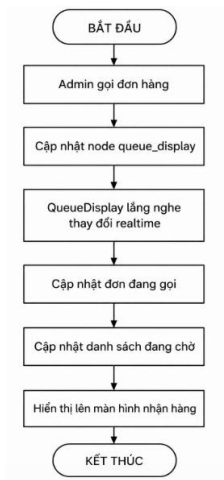
Queue Display được thiết kế nhằm hiển thị trạng thái xử lý đơn hàng tại khu vực nhận hàng. Thành phần này giúp khách hàng dễ dàng theo dõi tiến trình xử lý đơn hàng và biết thời điểm đến nhận sản phẩm.

Dữ liệu hiển thị được đồng bộ từ node `queue_display` trên Firebase Realtime Database. Khi Admin gọi đơn hàng hoặc thay đổi trạng thái xử lý, màn hình Queue Display sẽ tự động cập nhật mà không cần thao tác làm mới.

Dữ liệu Queue Display được tổ chức thành ba nhóm chính gồm:

- Đơn hàng đang được phục vụ.
- Danh sách đơn hàng đang chờ.
- Lịch sử các đơn hàng đã hoàn thành.

Luồng hoạt động của Queue Display được trình bày trong Hình 3.6.



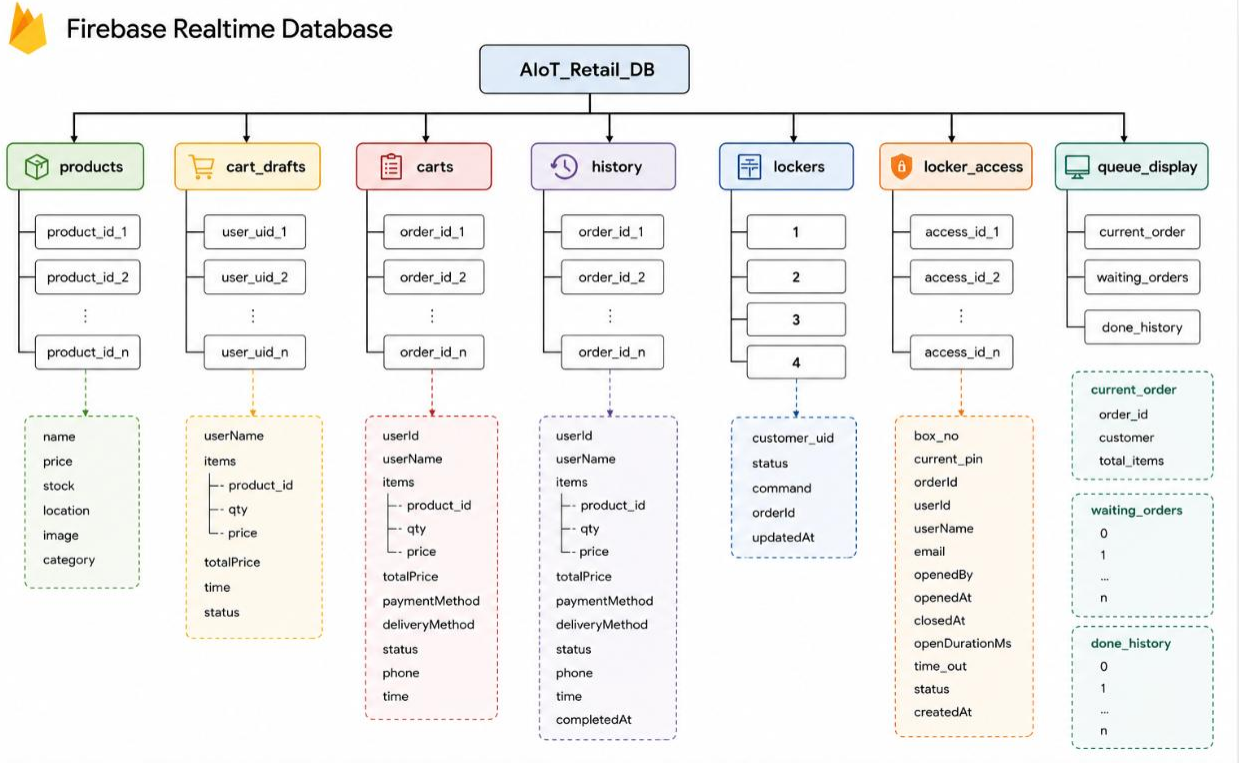
Hình 3.6. Luồng hoạt động của Queue Display

Thông qua cơ chế đồng bộ dữ liệu thời gian thực, Queue Display giúp quá trình giao nhận hàng trở nên rõ ràng và chuyên nghiệp hơn. Khách hàng có thể chủ động theo dõi trạng thái đơn hàng của mình, trong khi nhân viên dễ dàng quản lý thứ tự phục vụ.

Nhìn chung, thiết kế giao diện hướng đến việc phân chia rõ ràng chức năng giữa khách hàng và nhân viên, quản trị viên và khu vực giao nhận hàng. Việc sử dụng ReactJS kết hợp Firebase Realtime Database giúp đảm bảo khả năng cập nhật dữ liệu thời gian thực, đồng thời tạo nền tảng thuận lợi cho việc tích hợp AI Assistant và Smart Locker trong toàn bộ mô hình AIoT Smart Retail.

3.3. Thiết kế cấu trúc Firebase Realtime Database

Firebase Realtime Database được sử dụng làm trung tâm lưu trữ và đồng bộ dữ liệu thời gian thực giữa Website khách hàng, Website quản trị, Queue Display, AI Backend và Smart Locker ESP32. Dữ liệu được tổ chức theo cấu trúc phân cấp nhằm hỗ trợ quản lý sản phẩm, đơn hàng, giao nhận hàng và điều khiển thiết bị IoT.



Hình 3.7. Cấu trúc tổng thể Firebase Realtime Database

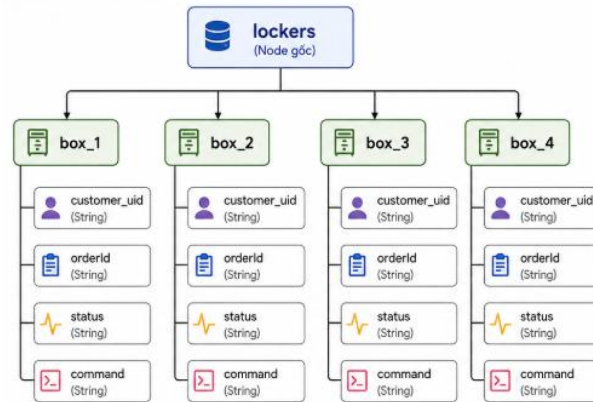
Node **products** lưu trữ thông tin sản phẩm bao gồm tên linh kiện, giá bán, số lượng tồn kho, hình ảnh và vị trí lưu trữ. Đây là nguồn dữ liệu chính phục vụ chức năng bán hàng và tư vấn sản phẩm.

Node **cart_drafts** được sử dụng để lưu trữ giỏ hàng tạm thời của khách hàng. Dữ liệu được lưu theo UID người dùng nhằm duy trì trạng thái giỏ hàng khi chuyển thiết bị hoặc đăng nhập lại.

Node **carts** quản lý các đơn hàng đang chờ xử lý. Sau khi khách hàng hoàn tất đặt hàng, thông tin đơn hàng sẽ được lưu tại node này để Admin theo dõi và xử lý.

Node **history** lưu trữ lịch sử các đơn hàng đã hoàn tất. Dữ liệu tại đây được sử dụng cho mục đích thống kê, báo cáo và phân tích hoạt động kinh doanh.

Node **lockers** đóng vai trò kết nối giữa Firebase và Smart Locker ESP32. Node này lưu trạng thái hoạt động của từng ngăn tủ, thông tin người nhận hàng và các lệnh điều khiển gửi đến thiết bị.



Hình 3.8. Cấu trúc dữ liệu node lockers

Node **locker_access** lưu trữ lịch sử truy cập và giao nhận hàng tại Smart Locker. Thông tin bao gồm mã PIN mở tủ, mã đơn hàng, người nhận, thời gian mở tủ và trạng thái giao nhận. Đây là cơ sở phục vụ truy vết và kiểm tra hoạt động của **locker_access**.



Hình 3.9. Cấu trúc dữ liệu node locker_access

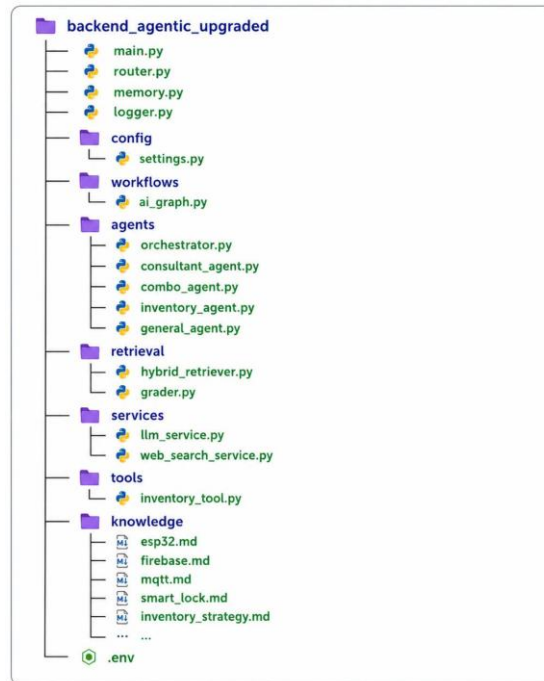
Node **queue_display** được sử dụng để đồng bộ dữ liệu cho màn hình Queue Display. Dữ liệu bao gồm đơn hàng đang được phục vụ, danh sách đơn hàng chờ và lịch sử các đơn hàng đã hoàn thành gần nhất.

Thông qua các node chức năng trên, Firebase Realtime Database đóng vai trò là trung tâm trao đổi dữ liệu giữa Website, AI Backend và Smart Locker. Việc tổ chức dữ liệu theo

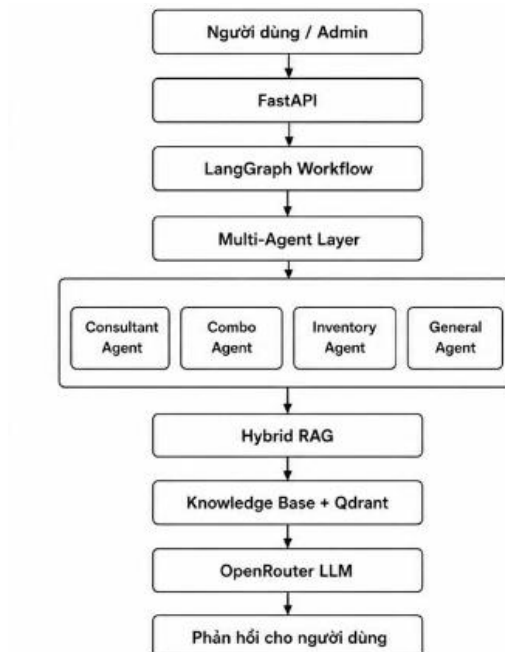
các node độc lập giúp quá trình quản lý trở nên hiệu quả hơn, giảm độ phức tạp khi đồng bộ dữ liệu và tạo điều kiện thuận lợi cho việc mở rộng chức năng trong tương lai.

3.4. Kiến trúc AI Backend

3.4.1. Kiến trúc tổng thể AI Backend



Hình 3.10. Sơ đồ cấu trúc thư mục và kiến trúc mã nguồn mô hình AI.



Hình 3.11. Sơ đồ kiến trúc mô hình AI.

AI Backend được xây dựng nhằm cung cấp khả năng xử lý trí tuệ nhân tạo cho mô hình AIoT Smart Retail. Thành phần này đóng vai trò trung tâm trong việc tiếp nhận câu hỏi từ người dùng hoặc quản trị viên, truy xuất tri thức liên quan từ kho dữ liệu nội bộ và sinh phản hồi thông qua mô hình ngôn ngữ lớn.

Khác với các chatbot truyền thống chỉ gửi trực tiếp câu hỏi đến mô hình AI, mô hình này được thiết kế theo kiến trúc nhiều tầng nhằm tăng khả năng mở rộng, nâng cao độ chính xác và giảm hiện tượng Hallucination. Kiến trúc được xây dựng dựa trên sự kết hợp giữa Supervisor-Based Multi-Agent, Hybrid RAG và LangGraph Workflow.

Trong kiến trúc này, FastAPI đóng vai trò là lớp giao tiếp với frontend. Mọi yêu cầu từ Website khách hàng hoặc Website quản trị đều được gửi đến AI Backend thông qua các API được xây dựng trên FastAPI.

Sau khi nhận câu hỏi, FastAPI chuyển dữ liệu đến LangGraph Workflow. Đây là thành phần điều phối trung tâm của mô hình, chịu trách nhiệm quản lý toàn bộ quá trình xử lý truy vấn.

Tại LangGraph Workflow, câu hỏi trước tiên được chuyển đến Supervisor Router để lựa chọn Agent phù hợp. Mô hình hiện tại bao gồm bốn Agent chuyên biệt:

- Consultant Agent.
- Combo Agent.
- Inventory Agent.
- General Agent.

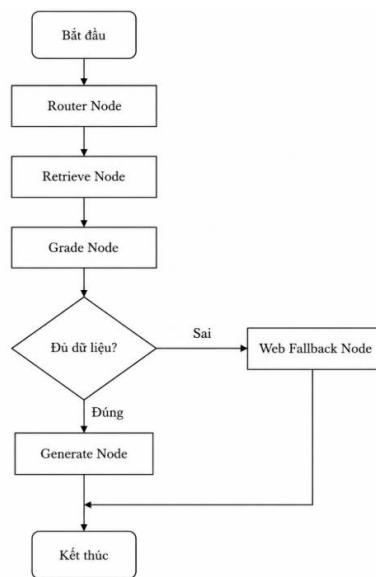
Sau khi Agent được xác định, Workflow tiếp tục gọi Hybrid RAG để truy xuất tri thức từ Knowledge Base và cơ sở dữ liệu vector Qdrant. Kết quả truy xuất được chuyển đến Truy xuất Grader để đánh giá mức độ liên quan.

Nếu dữ liệu truy xuất đạt yêu cầu, Workflow chuyển trực tiếp đến bước sinh phản hồi. Ngược lại, sẽ kích hoạt Web Fallback để tìm kiếm thêm thông tin từ Internet thông qua Tavily Search.

Cuối cùng, Prompt được xây dựng dựa trên Agent đã chọn, dữ liệu truy xuất và ngữ cảnh hội thoại trước khi gửi đến mô hình ngôn ngữ lớn thông qua OpenRouter API. Kết quả sinh ra được trả về FastAPI và gửi lại frontend cho người dùng.

Việc phân tách thành nhiều tầng xử lý giúp AI Backend dễ dàng mở rộng, bảo trì và bổ sung chức năng mới trong tương lai. Đồng thời, cách tiếp cận này giúp tận dụng được ưu điểm của cả Rule-Based Routing, Hybrid RAG và Large Language Model trong cùng một kiến trúc thống nhất.

3.4.2. Thiết kế LangGraph Workflow



Hình 3.12. Workflow xử lý truy vấn bằng LangGraph

Để điều phối toàn bộ quá trình xử lý truy vấn, mô hình sử dụng LangGraph nhằm xây dựng Workflow theo mô hình đồ thị trạng thái (State Graph). Mỗi bước xử lý được triển khai dưới dạng một Node độc lập, tạo thành chuỗi xử lý liên kết, trong đó dữ liệu được truyền tuần tự giữa các thành phần chức năng.

Trạng thái của Workflow được lưu trữ thông qua cấu trúc AIState.

```
class AIState(TypedDict):
    question: str
    context: str
    user_id: str
    agent: str
    knowledge: str
    sources: list
    retrieval_score: float
    final_answer: str
    is_web_fallback: bool
    force_web_fallback: bool
```

AIState đóng vai trò như bộ nhớ trung tâm của Workflow. Mỗi Node có thể đọc hoặc cập nhật dữ liệu trong trạng thái này trước khi chuyển sang bước xử lý tiếp theo.

Sau khi hoàn thiện cấu trúc trạng thái, các Node trong LangGraph được định nghĩa để thực hiện các bước xử lý tương ứng.

```
graph.add_node("router", router_node)
graph.add_node("retrieve", retrieve_node)
graph.add_node("grade", grade_node)
graph.add_node("web_fallback", web_fallback_node)
graph.add_node("generate", generate_node)
```

Trong đó:

Router Node thực hiện lựa chọn Agent.

Retrieve Node thực hiện Hybrid RAG Retrieval.

Grade Node đánh giá chất lượng retrieval.

Web Fallback Node truy xuất thông tin từ Internet.

Generate Node xây dựng Prompt và gọi mô hình ngôn ngữ lớn.

Luồng xử lý chính của Workflow được xây dựng thông qua các cạnh chuyển trạng thái.

```
graph.add_edge("router", "retrieve")
graph.add_edge("retrieve", "grade")
```

Sau khi hoàn tất quá trình retrieval, dữ liệu được chuyển đến Grade Node để đánh giá mức độ liên quan.

Tại đây, LangGraph sử dụng cơ chế điều hướng có điều kiện.

```
graph.add_conditional_edges(
    "grade",
    route_after_grade,
    {
        "generate": "generate",
        "web_fallback": "web_fallback"
    }
)
```

Nếu dữ liệu truy xuất đạt yêu cầu, Workflow chuyển trực tiếp đến Generate Node. Ngược lại, cơ chế Web Fallback được kích hoạt nhằm bổ sung thông tin từ Internet.

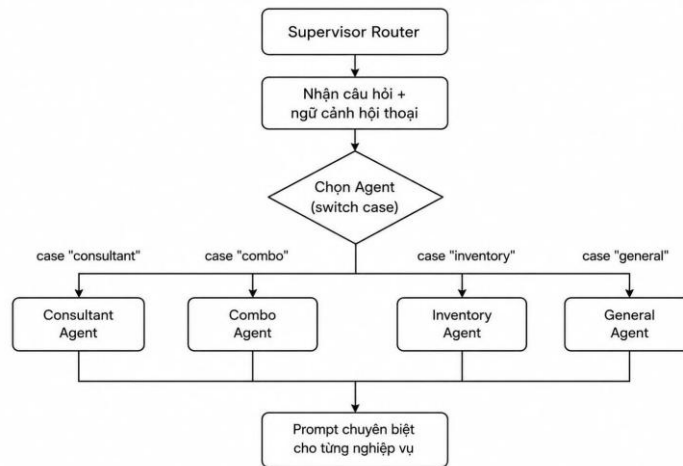
Sau khi Web Fallback hoàn tất, dữ liệu tiếp tục được chuyển đến Generate Node.

```
graph.add_edge(
    "web_fallback",
    "generate"
)
```

Generate Node là bước cuối cùng trong Workflow. Tại đây Prompt được tạo dựa trên Agent đã lựa chọn, sau đó được gửi đến OpenRouter để sinh phản hồi tương ứng.

Nhờ sử dụng LangGraph Workflow, toàn bộ quá trình xử lý được chia thành nhiều bước rõ ràng thay vì phụ thuộc hoàn toàn vào một lời gọi mô hình AI duy nhất. Điều này giúp dễ kiểm soát, dễ mở rộng và thuận lợi trong việc nâng cấp các thành phần riêng lẻ trong tương lai.

3.4.3. Thiết kế Supervisor-Based Multi-Agent



Hình 3.13. Kiến trúc Supervisor-Based Multi-Agent

Để nâng cao khả năng xử lý các nhóm truy vấn khác nhau, kiến trúc AI Backend được triển khai theo mô hình Supervisor-Based Multi-Agent. Thay vì sử dụng một Prompt chung cho mọi truy vấn, nhiều Agent chuyên biệt được xây dựng để xử lý các miền nghiệp vụ khác nhau, từ đó nâng cao độ chính xác và hiệu quả phản hồi.

Trong kiến trúc này, Supervisor Router đóng vai trò là thành phần điều phối trung tâm. Thành phần này tiếp nhận câu hỏi từ Workflow và chuyển đến bộ định tuyến để xác định Agent phù hợp.

Supervisor Router được triển khai trong file orchestrator.py.

```

def supervisor_router(
    question: str,
    context: str = ""
) -> str:

    combined = (
        f"{context}\n{question}"
        if context
        else question
    )

    return route_question(combined)

```

Đoạn mã trên cho thấy Supervisor Router không trực tiếp sinh phản hồi mà chỉ thực hiện nhiệm vụ kết hợp ngữ cảnh hội thoại với câu hỏi hiện tại trước khi chuyển đến bộ định tuyến.

Quá trình lựa chọn Agent được triển khai trong file router.py.

```

if has_any(q, inventory_keywords):
    return "inventory_agent"

if has_any(q, combo_keywords):
    return "combo_agent"

if has_any(q, consultant_keywords):
    return "consultant_agent"

return "general_agent"

```

Việc lựa chọn Agent được thực hiện thông qua cơ chế Rule-Based Routing, trong đó mỗi truy vấn được đối chiếu với tập từ khóa chuyên ngành tương ứng để xác định Agent xử lý phù hợp.

Trước khi thực hiện quá trình định tuyến, cơ chế kiểm tra bảo mật được áp dụng để phát hiện các truy vấn có dấu hiệu Prompt Injection hoặc các hành vi cố gắng truy cập trái phép vào dữ liệu nội bộ.

```
security_keywords = [  
    "ignore",  
    "system prompt",  
    "developer message",  
    "database",  
    "jailbreak"  
]
```

Nếu phát hiện các từ khóa nguy hiểm, truy vấn sẽ được chuyển đến General Agent nhằm hạn chế các rủi ro bảo mật và bảo vệ dữ liệu nội bộ.

Hiện tại, mô hình Multi-Agent được triển khai với bốn Agent chính:

- **Consultant Agent:** Xử lý các câu hỏi kỹ thuật liên quan đến vi điều khiển, cảm biến, giao tiếp phân cứng, IoT và các dự án nhúng.

```
def consultant_prompt(  
    question,  
    knowledge,  
    context,  
    memory  
):
```

- **Combo Agent:** Hỗ trợ đề xuất bộ linh kiện và các sản phẩm nên mua kèm cho từng dự án.


```
def combo_prompt(  
    question,  
    knowledge,  
    context  
):
```

- **Inventory Agent:** Phục vụ các truy vấn liên quan đến tồn kho, nhập hàng, doanh thu và các chức năng quản trị.

```
def inventory_prompt(  
    question,  
    knowledge,  
    context  
):
```

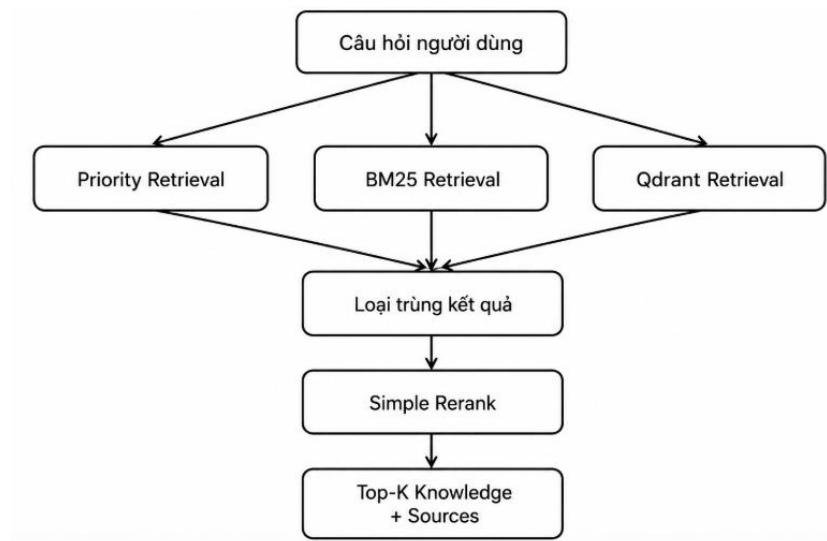
- **General Agent:** Là Agent mặc định của kiến trúc Multi-Agent, chịu trách nhiệm tiếp nhận và xử lý các câu hỏi mang tính tổng quát hoặc các truy vấn không thuộc phạm vi của các Agent chuyên biệt khác.

Khác với nhiều kiến trúc Multi-Agent sử dụng nhiều mô hình AI độc lập, các Agent trong mô hình không trực tiếp sinh phản hồi. Thay vào đó, mỗi Agent được xây dựng dưới dạng một Prompt Template chuyên biệt nhằm định hướng mô hình ngôn ngữ xử lý theo từng ngữ cảnh nghiệp vụ khác nhau.

Sau khi Agent được lựa chọn, thông tin này được lưu vào AIState và chuyển đến Hybrid RAG để thực hiện truy xuất tri thức trước khi sinh phản hồi cuối cùng.

Việc áp dụng kiến trúc Supervisor-Based Multi-Agent giúp tăng tính chuyên môn hóa của phản hồi, giảm hiện tượng trả lời sai ngữ cảnh và nâng cao chất lượng tương tác giữa người dùng với mô hình AIoT Smart Retail.

3.4.4. Thiết kế Hybrid RAG



Hình 3.14. Kiến trúc Hybrid RAG

Để nâng cao chất lượng truy xuất tri thức và hạn chế hiện tượng mô hình ngôn ngữ lớn sinh câu trả lời không có căn cứ, AI Backend sử dụng kiến trúc Hybrid RAG. Thành phần này được triển khai trong file `hybrid_retriever.py`, có nhiệm vụ tìm kiếm các tài liệu liên quan trong Knowledge Base và cơ sở dữ liệu vector Qdrant trước khi dữ liệu được đưa vào Prompt cho mô hình ngôn ngữ lớn.

Khác với RAG truyền thống chỉ sử dụng một phương pháp truy xuất duy nhất, Hybrid RAG trong mô hình kết hợp ba cơ chế gồm Priority Retrieval, BM25 Truy xuất và Qdrant Vector Retrieval. Sau khi thu thập kết quả từ các phương pháp trên, mô hình loại bỏ kết quả trùng lặp và thực hiện Reranking để chọn ra các tài liệu phù hợp nhất.

Trước khi thực hiện truy xuất, mô hình nạp toàn bộ tài liệu trong thư mục knowledge/ thông qua hàm `_load_documents()`.

```

def _load_documents() -> tuple[list[str], list[str]]:
    documents: list[str] = []
    sources: list[str] = []
    knowledge_dir = Path(KNOWLEDGE_DIR)

    if not knowledge_dir.exists():
        return documents, sources

    for file_path in sorted(knowledge_dir.glob("*.md")):
        try:
            text = file_path.read_text(encoding="utf-8")
        except Exception:
            continue

        documents.append(text)
        sources.append(file_path.name)

    return documents, sources

```

Hàm này đọc toàn bộ các file Markdown trong thư mục Knowledge Base, lưu nội dung vào danh sách DOCUMENTS và tên file nguồn vào danh sách SOURCES. Nhờ đó, mô hình có thể truy xuất tri thức từ nhiều tài liệu nội bộ khác nhau mà không cần nhập thủ công từng file.

Sau khi tài liệu được nạp, mô hình xây dựng chỉ mục BM25 bằng cách tách từ nội dung tài liệu. Trong trường hợp thư viện pyvi khả dụng, mô hình sử dụng ViTokenizer để hỗ trợ tách từ tiếng Việt. Nếu thư viện không khả dụng, mô hình tự động dùng phương án tách từ đơn giản nhằm đảm bảo chương trình vẫn hoạt động.

```
TOKENIZED_DOCS = [
    tokenize(doc).split()
    for doc in DOCUMENTS
] if DOCUMENTS else [[]]

BM25 = BM25Okapi(TOKENIZED_DOCS) \
    if (DOCUMENTS and BM25Okapi is not None) \
    else None
```

Đồng thời, HashingVectorizer được sử dụng để khởi tạo bộ vector hóa văn bản, tạo nền tảng cho các tác vụ xử lý ngôn ngữ tự nhiên.

```
vectorizer = HashingVectorizer(
    n_features=VECTOR_SIZE,
    alternate_sign=False,
    norm=None
)
```

Bộ vector hóa này được sử dụng trong hàm `embed_text()` để chuyển văn bản thành vector phục vụ truy vấn Qdrant.

```
def embed_text(text: str) -> list[float]:
    vec = vectorizer.transform([text])
    vec = normalize(vec)
    return vec.toarray()[0].tolist()
```

Priority Retrieval

Priority Truy xuất là lớp truy xuất đầu tiên của mô hình. Cơ chế này sử dụng tập luật ánh xạ giữa tên tài liệu và các từ khóa quan trọng. Nếu câu hỏi chứa một trong các từ khóa đã định nghĩa, nó sẽ ưu tiên lấy trực tiếp tài liệu tương ứng trong Knowledge Base.

```
PRIORITY_RULES: dict[str, list[str]] = {
    "rfid_rc522.md": ["rc522", "rfid", "thẻ từ", "mfr522"],
    "esp32.md": ["esp32", "wifi", "bluetooth", "devkit"],
    "esp32_s3.md": ["esp32-s3", "esp32 s3", "s3"],
    "esp8266.md": ["esp8266", "nodemcu"],
    "arduino_uno.md": ["arduino", "uno"],
    "stm32.md": ["stm32", "bluepill", "stm32f103"],
    "firebase.md": ["firebase", "realtime database"],
    "mqtt.md": ["mqtt"],
    "websocket.md": ["websocket"],
}
```

Trong source code thực tế, tập PRIORITY_RULES còn bao gồm nhiều tài liệu khác như relay.md, dht11.md, dht22.md, smart_home.md, smart_lock.md, combo_strategy.md, inventory_strategy.md và stock_analysis.md. Việc đưa nhiều tài liệu vào tập luật giúp quá trình truy xuất tri thức diễn ra nhanh chóng và chính xác hơn khi câu hỏi chứa các từ khóa đặc trưng.

Hàm priority_retrieve() được triển khai như sau:

```

def priority_retrieve(question: str) -> list[dict[str, Any]]:
    q = normalize_text(question)
    results = []

    for source, keywords in PRIORITY_RULES.items():
        if any(k in q for k in keywords):
            path = Path(KNOWLEDGE_DIR) / source
            if path.exists():
                results.append({
                    "text": path.read_text(encoding="utf-8"),
                    "source": source,
                    "score": 1.0,
                    "method": "priority",
                })

    return results

```

Kết quả của Priority Truy xuất luôn có score = 1.0 và method = "priority", thể hiện đây là dữ liệu được ưu tiên theo luật nghiệp vụ.

BM25 Retrieval

BM25 Truy xuất được sử dụng để tìm kiếm tài liệu dựa trên mức độ khớp từ khóa giữa câu hỏi và nội dung tài liệu. Phương pháp này phù hợp với các câu hỏi có chứa tên linh kiện, giao thức hoặc thuật ngữ kỹ thuật cụ thể.

```

def bm25_retrieve(
    question: str,
    top_k: int = TOP_K
) -> list[dict[str, Any]]:
    if not DOCUMENTS:
        return []

    tokenized_query = tokenize(question).split()

    if BM25 is not None:
        scores = BM25.get_scores(tokenized_query)
    else:
        q_terms = set(tokenized_query)
        scores = []

    for doc in DOCUMENTS:
        d_terms = set(tokenize(doc.lower()).split())
        scores.append(
            len(q_terms & d_terms) / max(len(q_terms), 1)
        )

```

Nếu thư viện `rank_bm25` không khả dụng, mô hình tự động chuyển sang cơ chế dự phòng bằng cách tính tỷ lệ từ khóa trùng khớp giữa câu hỏi và tài liệu. Điều này giúp mô hình vẫn có thể hoạt động ngay cả khi thiếu thư viện `BM25`.

Sau khi tính điểm, các tài liệu có điểm cao nhất được lựa chọn và tiến hành chuẩn hóa điểm số trước khi trả về kết quả.

```

top_idx = sorted(
    range(len(scores)),
    key=lambda i: scores[i],
    reverse=True
)[:top_k]

return [
    {
        "text": DOCUMENTS[i],
        "source": SOURCES[i],
        "score": float(scores[i] / max(max_score, 1e-9)),
        "method": "bm25" if BM25 is not None else "keyword_fallback",
    }
    for i in top_idx
    if scores[i] > 0
]

```

Qdrant Vector Retrieval

Bên cạnh tìm kiếm từ khóa, Qdrant được sử dụng để thực hiện tìm kiếm dựa trên vector. Trước tiên, câu hỏi của người dùng được chuyển thành vector thông qua hàm `embed_text()`.

```
query_vector = embed_text(question)
```

Sau đó, vector truy vấn được gửi đến Qdrant để tìm các điểm dữ liệu có độ tương đồng cao nhất.


```
points = client.query_points(  
    collection_name=QDRANT_COLLECTION,  
    query=query_vector,  
    limit=top_k,  
).points
```

Mỗi kết quả trả về từ Qdrant gồm nội dung văn bản, nguồn tài liệu, điểm tương đồng và phương pháp truy xuất.

```
results.append({  
    "text": payload.get("text", ""),  
    "source": payload.get("source", "qdrant"),  
    "score": float(getattr(p, "score", 0.0) or 0.0),  
    "method": "qdrant",  
})
```

Qdrant hỗ trợ tìm kiếm theo ngữ nghĩa, cho phép truy xuất các tài liệu liên quan ngay cả khi truy vấn không chứa chính xác các từ khóa xuất hiện trong Knowledge Base.

Reranking và hợp nhất kết quả

Sau khi có kết quả từ Priority Retrieval, BM25 Retrieval và Qdrant Retrieval, toàn bộ dữ liệu được hợp nhất thông qua hàm `hybrid_retrieve()`.

```
def hybrid_retrieve(
    question: str,
    top_k: int = TOP_K
) -> list[dict[str, Any]]:
    merged = []

    merged.extend(priority_retrieve(question))
    merged.extend(bm25_retrieve(question, top_k=top_k))
    merged.extend(qdrant_retrieve(question, top_k=top_k))
```

Do một tài liệu có thể được trả về bởi nhiều phương pháp khác nhau, các kết quả trùng lặp được loại bỏ dựa trên tên nguồn và một phần nội dung văn bản.

```
seen = set()
unique = []

for item in merged:
    key = (
        item.get("source"),
        item.get("text", "")[:120]
    )

    if key in seen:
        continue

    seen.add(key)
    unique.append(item)
```

Sau bước loại bỏ trùng lặp, quá trình Reranking được thực hiện thông qua hàm `simple_rerank()`.

```

return simple_rerank(
    question,
    unique,
    top_k=top_k
)

```

Hàm `simple_rerank()` tính toán điểm cuối cùng dựa trên điểm truy xuất ban đầu, tỷ lệ từ khóa trùng khớp và điểm thưởng theo phương pháp truy xuất.

```

method_bonus = {
    "priority": 0.2,
    "bm25": 0.1,
    "qdrant": 0.1
}.get(r.get("method"), 0)

r["final_score"] = (
    float(r.get("score", 0.0))
    + overlap
    + method_bonus
)

```

Trong đó, Priority Truy xuất được cộng điểm thưởng cao hơn vì đây là kết quả được xác định dựa trên luật nghiệp vụ. BM25 và Qdrant cũng được cộng điểm thưởng nhằm đảm bảo sự cân bằng giữa tìm kiếm từ khóa và tìm kiếm ngữ nghĩa.

Trả về tri thức và nguồn dữ liệu

Sau khi hoàn tất quá trình truy xuất và xếp hạng, mô hình sử dụng hàm `retrieve_knowledge_with_sources()` để trả về tri thức cuối cùng cho LangGraph Workflow.

```

def retrieve_knowledge_with_sources(
    question: str,
    top_k: int = TOP_K
) -> Tuple[str, List[str]]:
    results = hybrid_retrieve(question, top_k=top_k)

    knowledge = "\n\n".join(
        r["text"] for r in results
    )

    sources = []

    for r in results:
        if r["source"] not in sources:
            sources.append(r["source"])

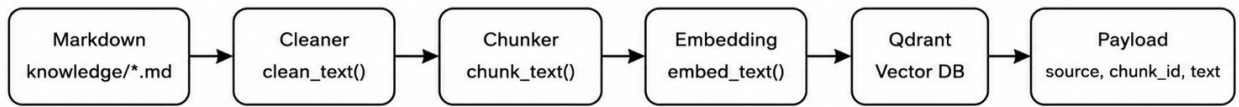
    return knowledge, sources

```

Kết quả trả về gồm hai thành phần. Thành phần knowledge là nội dung tri thức được đưa vào Prompt cho mô hình ngôn ngữ lớn. Thành phần sources chứa danh sách các tài liệu nguồn được sử dụng trong quá trình truy xuất, phục vụ việc theo dõi và truy vết dữ liệu được sử dụng trong quá trình sinh phản hồi.

Kiến trúc Hybrid RAG cho phép tích hợp nhiều cơ chế truy xuất khác nhau, bao gồm luật ưu tiên nghiệp vụ, tìm kiếm từ khóa bằng BM25 và tìm kiếm theo ngữ nghĩa dựa trên vector với Qdrant. Cách thiết kế này giúp AI Backend khai thác hiệu quả Knowledge Base nội bộ, tăng độ chính xác của dữ liệu truy xuất và hỗ trợ mô hình ngôn ngữ lớn tạo ra phản hồi phù hợp hơn với ngữ cảnh của mô hình AIoT Smart Retail.

3.4.5. Thiết kế Knowledge Base và Qdrant



Hình 3.15. Quy trình xây dựng Knowledge Base và Qdrant

Để cung cấp dữ liệu tri thức cho Hybrid RAG, một Knowledge Base nội bộ được xây dựng dưới dạng các tệp Markdown. Các tài liệu này được lưu trong thư mục knowledge/ và chứa thông tin về linh kiện điện tử, vi điều khiển, cảm biến, giao thức truyền thông, các mô hình ứng dụng IoT, chiến lược đề xuất combo và quản lý tồn kho.

Tuy nhiên, dữ liệu trong các tệp Markdown không thể được đưa trực tiếp vào cơ sở dữ liệu vector. Do đó, quy trình tiền xử lý dữ liệu được triển khai, bao gồm các bước: làm sạch văn bản, chia nhỏ tài liệu, sinh vector embedding và nạp dữ liệu vào Qdrant.

Làm sạch văn bản

Bước đầu tiên là làm sạch nội dung tài liệu. Chức năng này được triển khai trong file cleaner.py.

```
def clean_text(text: str) -> str:
    text = unicodedata.normalize("NFC", text or "")
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    text = re.sub(r"[ \t]+", " ", text)
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()
```

Hàm clean_text() thực hiện chuẩn hóa Unicode, chuẩn hóa ký tự xuống dòng, loại bỏ khoảng trắng dư thừa và rút gọn các dòng trống liên tiếp. Bước này giúp dữ liệu đầu vào có cấu trúc ổn định hơn trước khi chia nhỏ tài liệu và sinh vector.

Chia nhỏ tài liệu

Sau việc làm sạch, tài liệu được chia thành các đoạn nhỏ gọi là chunk. Chức năng này được triển khai trong file chunker.py.

Sau khi làm sạch, tài liệu được chia thành các đoạn nhỏ gọi là chunk. Chức năng này được triển khai trong file chunker.py.

```
def chunk_text(
    text: str,
    chunk_size: int = CHUNK_SIZE,
    chunk_overlap: int = CHUNK_OVERLAP
) -> list[str]:
```

Hàm `chunk_text()` đọc nội dung tài liệu theo từng dòng và ghép các dòng lại thành từng chunk sao cho độ dài không vượt quá `CHUNK_SIZE`. Khi một chunk mới được tạo, một phần nội dung ở cuối chunk trước được giữ lại thông qua tham số `CHUNK_OVERLAP`.

Cơ chế overlap giúp hạn chế mất ngữ cảnh ở ranh giới giữa hai chunk liên tiếp. Điều này đặc biệt quan trọng đối với các tài liệu kỹ thuật, vì một ý nghĩa có thể trải dài qua nhiều dòng hoặc nhiều đoạn văn.

Sinh vector embedding

Sau khi tài liệu được chia thành các chunk, mỗi chunk được chuyển thành vector thông qua hàm `embed_text()` trong file `hybrid_retriever.py`.

```
def embed_text(text: str) -> list[float]:
    vec = vectorizer.transform([text])
    vec = normalize(vec)
    return vec.toarray()[0].tolist()
```

Trong mô hình, vector được tạo bằng `HashingVectorizer` của thư viện `Scikit-learn`. Sau khi văn bản được chuyển thành vector, mô hình chuẩn hóa vector trước khi lưu trữ. Việc chuẩn hóa giúp quá trình so sánh độ tương đồng giữa các vector ổn định hơn.

Nạp dữ liệu vào Qdrant

Quá trình nạp tri thức vào Qdrant được triển khai trong file `ingest_knowledge.py`.

Đầu tiên, mô hình kết nối đến Qdrant và tạo lại collection dùng để lưu vector.

```

client = QdrantClient(url=QDRANT_URL)

client.recreate_collection(
    collection_name=QDRANT_COLLECTION,
    vectors_config=VectorParams(
        size=VECTOR_SIZE,
        distance=Distance.COSINE
    ),
)

```

Trong đó, QDRANT_COLLECTION là tên collection dùng để lưu dữ liệu tri thức, VECTOR_SIZE là kích thước vector và Distance.COSINE là độ đo tương đồng được sử dụng trong quá trình tìm kiếm vector.

Sau khi tạo collection, toàn bộ các tệp Markdown trong thư mục knowledge/ được duyệt và xử lý.

```

for file_path in sorted(knowledge_dir.glob("*.md")):
    text = clean_text(
        file_path.read_text(encoding="utf-8")
    )

    chunks = chunk_text(text)

```

Mỗi file được đọc, làm sạch và chia thành các chunk nhỏ. Với mỗi chunk, mô hình tạo một điểm dữ liệu PointStruct để lưu vào Qdrant.

```

PointStruct(
    id=point_id,
    vector=embed_text(chunk),
    payload={
        "source": file_path.name,
        "chunk_id": idx,
        "text": chunk,
    },
)

```

Mỗi điểm dữ liệu gồm ba phần chính. Trường id là mã định danh của chunk. Trường vector chứa vector embedding của đoạn văn bản. Trường payload lưu thông tin bổ sung gồm tên tài liệu nguồn, số thứ tự chunk và nội dung văn bản gốc.

Sau khi tạo xong danh sách các điểm dữ liệu, toàn bộ dữ liệu được ghi vào Qdrant thông qua lệnh:

```

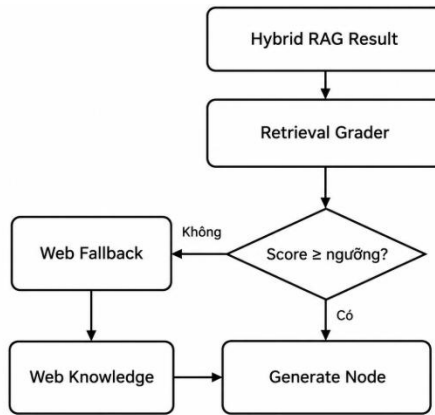
client.upsert(
    collection_name=QDRANT_COLLECTION,
    points=points
)

```

Nhờ quy trình này, các tài liệu Markdown trong Knowledge Base được chuyển đổi thành dữ liệu vector có thể tìm kiếm được. Khi người dùng đặt câu hỏi, Hybrid RAG có thể truy vấn Qdrant để tìm các chunk có độ tương đồng cao nhất với nội dung câu hỏi.

Việc thiết kế Knowledge Base kết hợp Qdrant giúp mở rộng tri thức một cách linh hoạt mà không cần huấn luyện lại mô hình ngôn ngữ lớn. Khi cần bổ sung dữ liệu mới, người phát triển chỉ cần thêm hoặc chỉnh sửa file Markdown trong thư mục knowledge/, sau đó chạy lại tiến trình ingestion để cập nhật dữ liệu vào Qdrant.

3.4.6. Thiết kế Truy xuất Grader và Web Fallback



Hình 3.16. Cơ chế Truy xuất Grader và Web Fallback

Mặc dù Hybrid RAG có khả năng truy xuất tri thức từ Knowledge Base và cơ sở dữ liệu vector Qdrant, tuy nhiên không phải mọi kết quả truy xuất đều có chất lượng đủ tốt để sử dụng trực tiếp cho quá trình sinh phản hồi. Trong thực tế, hệ thống có thể truy xuất nhầm tài liệu hoặc chỉ tìm thấy những tài liệu có mức độ liên quan thấp đến câu hỏi của người dùng.

Để giải quyết vấn đề này, đề tài xây dựng cơ chế Truy xuất Grader nhằm đánh giá chất lượng dữ liệu truy xuất trước khi chuyển sang bước sinh phản hồi. Thành phần này được triển khai trong file grader.py.

Khác với nhiều hệ thống RAG sử dụng mô hình ngôn ngữ lớn để đánh giá tài liệu, Truy xuất Grader của đề tài được xây dựng theo phương pháp Rule-Based. Cách tiếp cận này giúp giảm chi phí tính toán, tăng tốc độ xử lý và dễ dàng kiểm soát tiêu chí đánh giá.

Chuẩn hóa dữ liệu đánh giá

Trước khi thực hiện chấm điểm, hệ thống chuẩn hóa nội dung câu hỏi và dữ liệu retrieval.

```
def normalize(text: str) -> str:
    text = text.lower()
    text = unicodedata.normalize("NFC", text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

Việc chuẩn hóa giúp loại bỏ sự khác biệt về chữ hoa, chữ thường và khoảng trắng dư thừa, từ đó tăng độ chính xác trong quá trình so sánh từ khóa.

Trích xuất từ khóa quan trọng

Sau khi chuẩn hóa dữ liệu, hệ thống loại bỏ các từ dừng tiếng Việt và giữ lại các từ khóa mang ý nghĩa chính của câu hỏi.

```
q_words = [
    w for w in normalize(question).split()
    if w not in VI_STOPWORDS
]
```

Các từ khóa này đóng vai trò là tập từ đại diện cho nội dung truy vấn của người dùng.

Tính điểm Retrieval

Tiếp theo, hệ thống kiểm tra số lượng từ khóa xuất hiện trong dữ liệu retrieval.

```
matched = sum(
    1
    for w in q_words
    if w in knowledge_text
)
```

Điểm truy xuất được tính theo tỷ lệ từ khóa xuất hiện trong dữ liệu truy xuất.

```
score = matched / max(  
    len(q_words),  
    1  
)
```

Giá trị này phản ánh mức độ liên quan giữa dữ liệu truy xuất và câu hỏi của người dùng. Điểm số càng cao thì khả năng dữ liệu phù hợp với truy vấn càng lớn.

Ngoài việc tính toán tỷ lệ từ khóa trùng khớp, hệ thống còn áp dụng thêm cơ chế giảm điểm khi tỷ lệ xuất hiện của các từ khóa chính quá thấp.

```
if keyword_match_ratio < 0.3:  
    score *= 0.3
```

Cơ chế này giúp hạn chế trường hợp dữ liệu truy xuất chứa nhiều nội dung không liên quan nhưng vẫn đạt điểm cao do trùng khớp một số từ khóa phổ biến.

Sau khi hoàn tất quá trình đánh giá, điểm số truy xuất được lưu vào AIState để phục vụ bước điều hướng Workflow.

Điều hướng Workflow sau khi đánh giá

Kết quả Truy xuất Grader được sử dụng trong hàm quyết định luồng xử lý của LangGraph.

```
if retrieval_score >= threshold:  
    return "generate"  
  
return "web_fallback"
```

Nếu điểm số truy xuất vượt ngưỡng cấu hình, Workflow chuyển trực tiếp đến Generate Node để sinh phản hồi.

Ngược lại, nếu dữ liệu truy xuất không đủ chất lượng, hệ thống kích hoạt Web Fallback nhằm tìm kiếm tri thức bổ sung từ Internet.

Thiết kế Web Fallback

Web Fallback là cơ chế dự phòng được xây dựng để xử lý các câu hỏi nằm ngoài phạm vi Knowledge Base hoặc các trường hợp Truy xuất Grader đánh giá dữ liệu truy xuất chưa đủ tin cậy.

Thành phần này được triển khai trong file `web_search_service.py` và sử dụng Tavily Search API để tìm kiếm thông tin từ Web.

```
client = TavilyClient(  
    api_key=TAVILY_API_KEY  
)
```

Khi được kích hoạt, hệ thống gửi câu hỏi của người dùng đến Tavily Search.

```
response = client.search(  
    query=query,  
    search_depth="basic",  
    max_results=3  
)
```

Trong đó:

- `query` là câu hỏi của người dùng.
- `search_depth` xác định độ sâu tìm kiếm.
- `max_results` quy định số lượng kết quả tối đa được trả về.

Sau khi nhận được dữ liệu từ Tavily, hệ thống tổng hợp nội dung các kết quả tìm kiếm thành một khối tri thức duy nhất.

```
knowledge = "\n\n".join(  
    result["content"]  
    for result in results  
)
```

Khối tri thức này được đưa trở lại Workflow và thay thế cho dữ liệu truy xuất trước đó.

Sau đó hệ thống tiếp tục chuyển dữ liệu đến Generate Node để xây dựng Prompt và sinh phản hồi thông qua mô hình ngôn ngữ lớn.

Vai trò của Truy xuất Grader và Web Fallback

Sự kết hợp giữa Truy xuất Grader và Web Fallback tạo thành một cơ chế tự đánh giá và bổ sung tri thức trước khi gọi mô hình ngôn ngữ lớn.

Đối với các câu hỏi thuộc phạm vi Knowledge Base, hệ thống sử dụng dữ liệu nội bộ để đảm bảo tính chính xác và nhất quán. Đối với các câu hỏi mới hoặc chưa xuất hiện trong kho tri thức, Web Fallback giúp hệ thống mở rộng khả năng truy cập thông tin từ Internet.

Nhờ đó, AI Backend vừa duy trì được độ tin cậy của dữ liệu nội bộ, vừa đảm bảo khả năng xử lý linh hoạt đối với nhiều loại truy vấn khác nhau.

3.4.7. Thiết kế OpenRouter LLM Service

Sau khi hoàn tất các bước lựa chọn Agent, truy xuất tri thức và đánh giá chất lượng dữ liệu, hệ thống tiến hành sinh phản hồi thông qua mô hình ngôn ngữ lớn (Large Language Model - LLM). Thành phần này được triển khai trong file `llm_service.py` và đóng vai trò là tầng xử lý cuối cùng của AI Backend.

Thay vì kết nối trực tiếp đến một mô hình AI cố định, đề tài sử dụng OpenRouter API làm lớp trung gian giữa hệ thống và các mô hình ngôn ngữ lớn. Cách tiếp cận này giúp hệ thống có thể linh hoạt thay đổi mô hình AI mà không cần sửa đổi kiến trúc hoặc logic xử lý bên trong.

Cấu hình kết nối OpenRouter

Thông tin kết nối đến OpenRouter được cấu hình trong file `settings.py`.

```
OPENROUTER_API_KEY
OPENROUTER_BASE_URL
MODEL_NAME
```

Trong đó

- OPENROUTER_API_KEY dùng để xác thực với OpenRouter.
- OPENROUTER_BASE_URL là địa chỉ API Gateway.
- MODEL_NAME xác định mô hình AI được sử dụng.

Nhờ tách riêng phần cấu hình, hệ thống có thể dễ dàng chuyển đổi giữa các mô hình khác nhau mà không cần sửa mã nguồn nghiệp vụ.

Xây dựng hàm sinh phản hồi

Chức năng sinh phản hồi được triển khai thông qua hàm `call_ai()`.

```
def call_ai(
    prompt: str,
    temperature: float = 0.4,
    max_tokens: int = 1200
):
```

Hàm này nhận Prompt đã được xây dựng từ Workflow và gửi yêu cầu đến OpenRouter.

```
payload = {  
    "model": MODEL_NAME,  
    "messages": [  
        {  
            "role": "user",  
            "content": prompt  
        }  
    ],  
    "temperature": temperature,  
    "max_tokens": max_tokens  
}
```

Trong đó:

model xác định mô hình AI sử dụng.

messages chứa nội dung Prompt.

temperature điều khiển mức độ sáng tạo của câu trả lời.

max_tokens giới hạn độ dài phản hồi.

Sau khi xây dựng payload, hệ thống gửi yêu cầu đến OpenRouter bằng giao thức HTTP.

```
response = requests.post(  
    OPENROUTER_BASE_URL,  
    headers=headers,  
    json=payload,  
    timeout=60  
)
```

Kết quả trả về được phân tích và trích xuất nội dung phản hồi.

```
answer = response.json()[
    "choices"
][0]["message"]["content"]
```

Nội dung này chính là câu trả lời cuối cùng được gửi về cho người dùng.

Xây dựng Prompt theo Agent

Một điểm quan trọng trong hệ thống là Prompt không được tạo cố định mà phụ thuộc vào Agent đã được lựa chọn bởi Supervisor Router.

Ví dụ:

```
if agent == "consultant_agent":
    prompt = consultant_prompt(...)

elif agent == "combo_agent":
    prompt = combo_prompt(...)

elif agent == "inventory_agent":
    prompt = inventory_prompt(...)

else:
    prompt = general_prompt(...)
```

Nhờ cơ chế này, cùng một mô hình AI nhưng có thể xử lý nhiều nhóm nghiệp vụ khác nhau thông qua các Prompt chuyên biệt.

- Consultant Agent tập trung vào tư vấn kỹ thuật.
- Combo Agent tập trung vào đề xuất bộ linh kiện.
- Inventory Agent tập trung vào phân tích tồn kho.
- General Agent xử lý hội thoại tổng quát.

Điều này giúp tăng mức độ chuyên môn hóa của phản hồi mà không cần triển khai nhiều mô hình AI riêng biệt.

Hỗ trợ Streaming Response

Bên cạnh chế độ phản hồi thông thường, hệ thống còn hỗ trợ Streaming Response nhằm cải thiện trải nghiệm người dùng.

```
def stream_ai(  
    prompt: str  
):
```

Ở chế độ này, mô hình AI gửi dữ liệu từng phần thay vì đợi sinh hoàn chỉnh toàn bộ câu trả lời.

Frontend có thể hiển thị nội dung ngay khi nhận được dữ liệu mới, giúp giảm thời gian chờ và tạo cảm giác phản hồi theo thời gian thực.

Vai trò của OpenRouter trong hệ thống

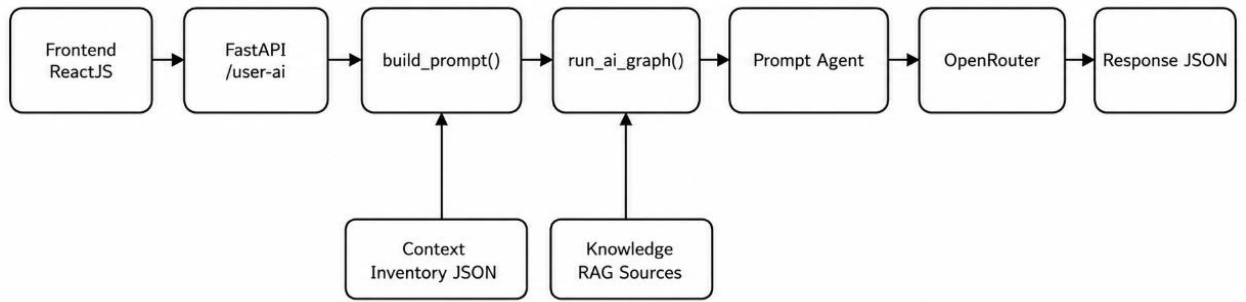
OpenRouter đóng vai trò là tầng truy cập mô hình ngôn ngữ lớn của AI Backend. Thành phần này nhận Prompt đã được tối ưu hóa từ LangGraph Workflow và trả về phản hồi cuối cùng cho người dùng.

Việc sử dụng OpenRouter mang lại nhiều lợi ích:

- Dễ dàng thay đổi mô hình AI.
- Không phụ thuộc vào một nhà cung cấp duy nhất.
- Hỗ trợ nhiều mô hình hiện đại.
- Giảm chi phí bảo trì hệ thống.
- Tăng khả năng mở rộng trong tương lai.

Nhờ đó, AI Backend có thể tận dụng năng lực xử lý ngôn ngữ tự nhiên của các mô hình hiện đại đồng thời vẫn duy trì được tính linh hoạt trong triển khai và nâng cấp hệ thống.

3.4.8. Thiết kế FastAPI AI Backend



Hình 3.17. Luồng xử lý yêu cầu trong FastAPI AI Backend

Để cung cấp giao diện lập trình ứng dụng cho toàn bộ hệ thống AI, đề tài sử dụng FastAPI làm nền tảng xây dựng AI Backend. FastAPI được lựa chọn nhờ khả năng xử lý hiệu năng cao, hỗ trợ bất đồng bộ và tích hợp tốt với các ứng dụng trí tuệ nhân tạo.

FastAPI đóng vai trò là lớp giao tiếp giữa frontend ReactJS và các thành phần AI Backend. Mọi yêu cầu từ người dùng đều được gửi đến FastAPI trước khi chuyển vào LangGraph Workflow để xử lý.

Khởi tạo ứng dụng FastAPI

Ứng dụng được khởi tạo trong file main.py.

```
app = FastAPI(
    title="AIoT Smart Retail AI Backend",
    version="2.0.0"
)
```

Đoạn mã trên tạo một máy chủ API chịu trách nhiệm tiếp nhận và xử lý các yêu cầu từ phía người dùng.

Để cho phép frontend và backend giao tiếp với nhau, hệ thống cấu hình CORS Middleware.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"]
)
```

Cấu hình này cho phép frontend ReactJS truy cập các API của AI Backend từ nhiều nguồn khác nhau trong quá trình phát triển và triển khai.

Xây dựng mô hình dữ liệu đầu vào

Dữ liệu đầu vào của hệ thống được chuẩn hóa bằng lớp AskRequest.

```
class AskRequest(BaseModel):
    question: str
    context: Optional[Any] = None
    user_id: Optional[str] = "guest"
    history: Optional[
        List[Dict[str, str]]
    ] = []
```

Trong đó:

- question chứa câu hỏi của người dùng.
- context chứa dữ liệu ngữ cảnh bổ sung.
- user_id định danh người dùng.
- history lưu lịch sử hội thoại.

Việc chuẩn hóa dữ liệu giúp hệ thống dễ dàng kiểm tra và xử lý yêu cầu trước khi đưa vào Workflow.

Thiết kế các API xử lý AI

Hệ thống cung cấp nhiều API khác nhau nhằm phục vụ các nhóm người dùng và các chế độ hoạt động riêng biệt.

API dành cho người dùng:

```
@app.post("/user-ai")
```

API dành cho quản trị viên:

```
@app.post("/admin-ai")
```

API tổng quát:

```
@app.post("/ask-ai")
```

API hỗ trợ Streaming Response:

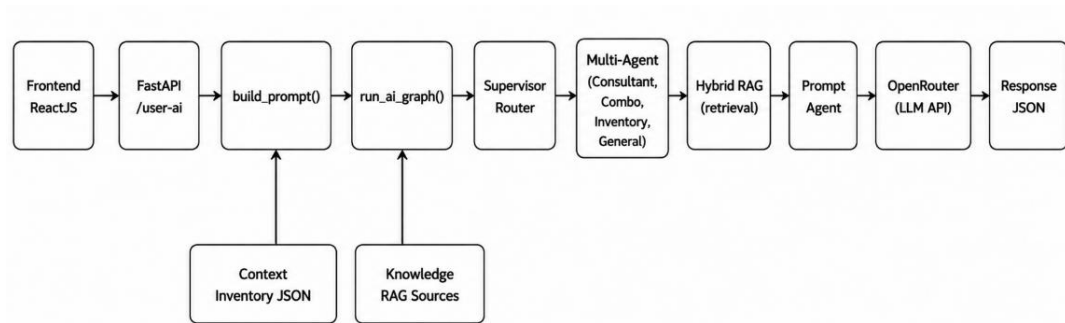
```
@app.post("/user-ai-stream")
```

Các API này đều tiếp nhận dữ liệu từ frontend và chuyển câu hỏi vào LangGraph Workflow để xử lý.

Luồng xử lý yêu cầu

Khi nhận được yêu cầu từ frontend, FastAPI thực hiện quy trình xử lý như sau:

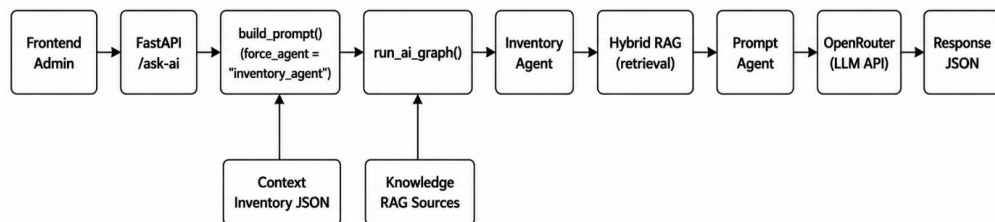
Luồng User AI (/user-ai)



Hình 3.18. Luồng xử lý User AI

Người dùng gửi câu hỏi từ giao diện Web đến API /user-ai. Hệ thống xây dựng ngữ cảnh thông qua build_prompt(), sau đó chuyển truy vấn đến run_ai_graph(). Supervisor Router phân tích nội dung câu hỏi và tự động lựa chọn Agent phù hợp. Tiếp theo, Hybrid RAG truy xuất tri thức từ Knowledge Base và Qdrant để xây dựng Prompt. Prompt được gửi đến OpenRouter để sinh phản hồi và trả kết quả về frontend dưới dạng JSON.

Luồng Admin AI (/admin-ai)



Hình 3.19. Luồng xử lý Admin AI

Quản trị viên gửi câu hỏi đến API /admin-ai. Hệ thống xây dựng ngữ cảnh và sử dụng cơ chế force_agent="inventory_agent" để chuyển trực tiếp đến Inventory Agent. Sau

đó, Hybrid RAG truy xuất dữ liệu liên quan, xây dựng Prompt và gửi đến OpenRouter để sinh phản hồi. Kết quả được trả về giao diện quản trị dưới dạng JSON.

Luồng Ask AI (/ask-ai)

API /ask-ai được triển khai bằng cách gọi trực tiếp API /admin-ai, do đó luồng xử lý hoàn toàn tương tự Admin AI. Mô hình sử dụng Inventory Agent, truy xuất dữ liệu thông qua Hybrid RAG, xây dựng Prompt và gửi đến OpenRouter để tạo phản hồi cuối cùng.

Vai trò của FastAPI trong hệ thống

FastAPI là thành phần kết nối toàn bộ kiến trúc AI Backend với các hệ thống bên ngoài. Nhờ sử dụng FastAPI, hệ thống có thể:

- Dễ dàng tích hợp với ReactJS.
- Hỗ trợ giao tiếp thời gian thực.
- Quản lý API tập trung.
- Dễ dàng mở rộng dịch vụ trong tương lai.
- Hỗ trợ triển khai trên Docker và Cloud.

Việc xây dựng AI Backend trên nền tảng FastAPI giúp hệ thống đạt được hiệu năng cao, dễ bảo trì và đáp ứng tốt các yêu cầu của ứng dụng AIoT Smart Retail.

Ngoài dữ liệu truy xuất từ Hybrid RAG, hệ thống còn hỗ trợ tiếp nhận dữ liệu tồn kho theo thời gian thực từ frontend thông qua trường context.

Để hỗ trợ các truy vấn liên quan đến quản lý kho, AI Backend sử dụng hàm `build_inventory_context()` trong file `inventory_tool.py` nhằm phân tích dữ liệu JSON được gửi từ giao diện quản trị.

```
inventory_context = build_inventory_context(  
    data.context  
)
```

Hàm này thực hiện việc đọc danh sách sản phẩm, kiểm tra số lượng tồn kho và phát hiện các mặt hàng có số lượng thấp hơn ngưỡng cho phép.

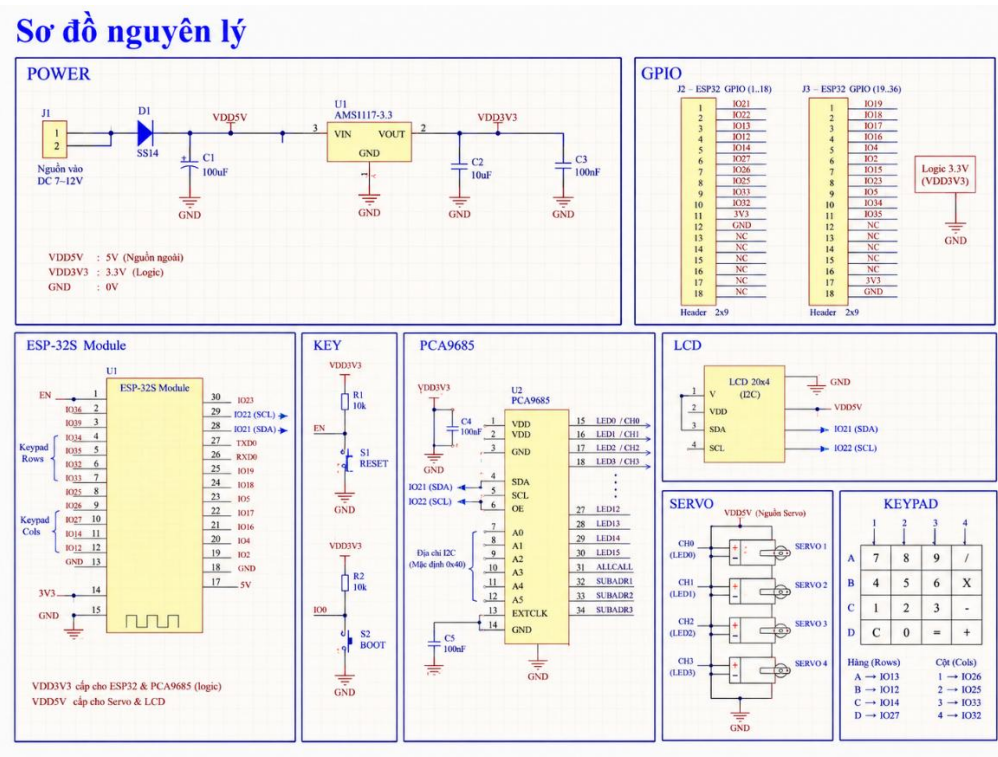
Kết quả phân tích được chuyển thành văn bản và bổ sung vào Prompt trước khi gửi đến mô hình ngôn ngữ lớn.

```
knowledge += (
    "\n\nPHÂN TÍCH NHANH DỮ LIỆU KHO:\n"
    + inventory_context
)
```

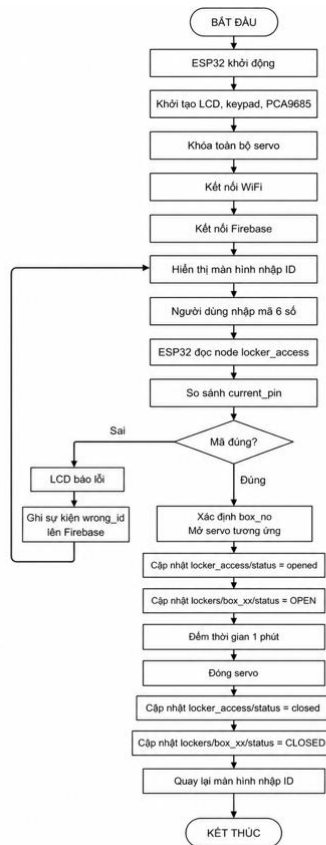
Nhờ cơ chế này, AI có thể đưa ra các nhận xét và khuyến nghị dựa trên dữ liệu tồn kho hiện tại của hệ thống mà không cần lưu trữ dữ liệu động trong Knowledge Base hoặc Qdrant.

3.5. Kiến trúc tổng thể phần cứng

Kiến trúc phần cứng Smart Locker được thiết kế theo mô hình sau:



Hình 3.20. Sơ đồ nguyên lý phần cứng



Hình 3.21. Lưu đồ luồng hoạt động của hệ thống khóa tủ

Lưu đồ mô tả quy trình hoạt động của hệ thống khóa tủ thông minh sử dụng vi điều khiển ESP32 kết hợp Firebase Realtime Database. Khi hệ thống được cấp nguồn, ESP32 tiến hành khởi động và thực hiện quá trình khởi tạo các thiết bị ngoại vi bao gồm màn hình LCD, bàn phím keypad và module điều khiển servo PCA9685. Sau khi hoàn tất khởi tạo, hệ thống đưa toàn bộ servo về trạng thái khóa nhằm đảm bảo các ngăn tủ luôn ở trạng thái an toàn trước khi vận hành.

Tiếp theo, ESP32 thực hiện kết nối WiFi để truy cập Internet và thiết lập kết nối với Firebase nhằm đồng bộ dữ liệu thời gian thực giữa phần cứng và hệ thống quản lý. Khi kết nối thành công, màn hình LCD hiển thị giao diện yêu cầu người dùng nhập mã ID gồm 6 chữ số để truy cập tủ khóa.

Sau khi người dùng nhập mã, ESP32 tiến hành đọc dữ liệu từ node locker_access trên Firebase và thực hiện so sánh mã vừa nhập với giá trị current_pin được lưu trong cơ sở dữ

liệu. Quá trình kiểm tra này quyết định quyền truy cập của người dùng đối với hệ thống tủ khóa.

Trong trường hợp mã nhập không chính xác, hệ thống sẽ hiển thị thông báo lỗi trên màn hình LCD, đồng thời ghi nhận sự kiện đăng nhập sai (`wrong_id`) lên Firebase để phục vụ việc theo dõi và quản lý lịch sử truy cập. Sau đó, hệ thống quay trở lại màn hình nhập ID để người dùng thực hiện nhập lại mã truy cập.

Ngược lại, nếu mã nhập hợp lệ, ESP32 sẽ xác định số thứ tự ngăn tủ (`box_no`) tương ứng và điều khiển servo mở đúng ngăn chứa hàng. Đồng thời, trạng thái truy cập được cập nhật trên Firebase thông qua việc thay đổi giá trị `locker_access/status` thành `opened` và cập nhật trạng thái của ngăn tủ tại node `lockers/box_xx/status` thành `OPEN`.

Sau khi mở tủ thành công, hệ thống bắt đầu đếm thời gian sử dụng trong vòng 1 phút. Khi hết thời gian quy định, ESP32 tự động điều khiển servo đóng lại ngăn tủ nhằm đảm bảo an toàn cho hàng hóa. Cuối cùng, trạng thái hệ thống tiếp tục được cập nhật trên Firebase với giá trị `closed` và `CLOSED`, sau đó giao diện quay trở lại màn hình nhập ID để chờ lượt truy cập tiếp theo.

Các thành phần chính của phần cứng:

Bảng 3.2. Các thành phần chính của phần cứng

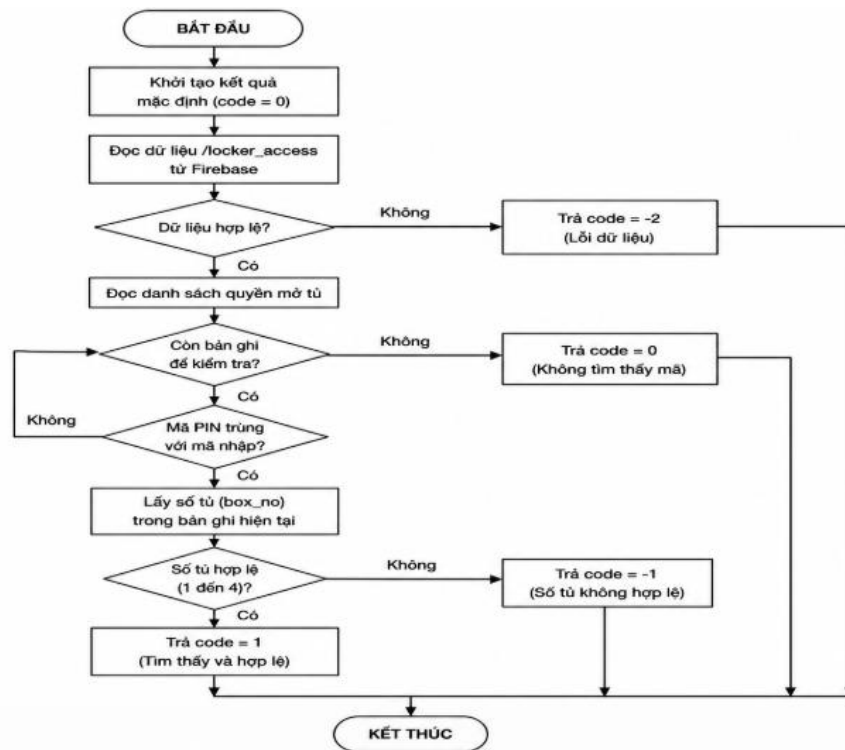
Thành phần	Vai trò
ESP32 DevKit	Bộ điều khiển trung tâm, kết nối WiFi và xử lý logic
Keypad 4x4	Nhập mã PIN hoặc ID nhận hàng
LCD I2C	Hiển thị trạng thái hệ thống
PCA9685	Mở rộng kênh PWM để điều khiển nhiều servo
Servo	Cơ cấu khóa/mở từng ngăn tủ
Nguồn 5V 3A	Cấp nguồn riêng cho servo
Firebase RTDB	Lưu mã mở tủ, trạng thái tủ và lịch sử truy cập

Trong thiết kế này, ESP32 không điều khiển servo trực tiếp mà thông qua module PCA9685. Cách thiết kế này giúp hệ thống có thể mở rộng số lượng tủ vì PCA9685 hỗ trợ

nhiều kênh PWM. LCD và PCA9685 cùng sử dụng giao tiếp I2C nên giảm số lượng chân kết nối trên ESP32.

3.5.1. Thiết kế giải thuật tìm mã PIN trên Firebase

Khi người dùng nhập đủ 6 số, ESP32 gọi hàm checkID() để kiểm tra mã. Hàm này tiếp tục gọi findAccessByID() để tìm mã trong node locker_access.



Hình 3.22. lưu đồ kiểm tra quyền mở tủ

Lưu đồ giải thuật Hình 3.32 mô tả quá trình kiểm tra quyền mở tủ dựa trên mã PIN được người dùng nhập từ bàn phím. Ban đầu, hệ thống khởi tạo cấu trúc kết quả mặc định với giá trị code = 0, đồng thời thiết lập các thông tin như số tủ và đường dẫn bản ghi ở trạng thái rỗng. Sau đó, ESP32 thực hiện đọc dữ liệu từ node /locker_access trên Firebase Realtime Database để lấy danh sách các quyền truy cập tủ hiện có trong hệ thống.

Sau khi đọc dữ liệu, hệ thống tiến hành kiểm tra tính hợp lệ của dữ liệu nhận được. Nếu dữ liệu rỗng, không tồn tại hoặc xảy ra lỗi trong quá trình đọc và phân tích dữ liệu JSON, chương trình sẽ trả về mã lỗi code = -2, biểu thị trạng thái lỗi dữ liệu Firebase. Trong trường

hợp dữ liệu hợp lệ, chương trình bắt đầu duyệt tuần tự từng bản ghi trong danh sách quyền truy cập.

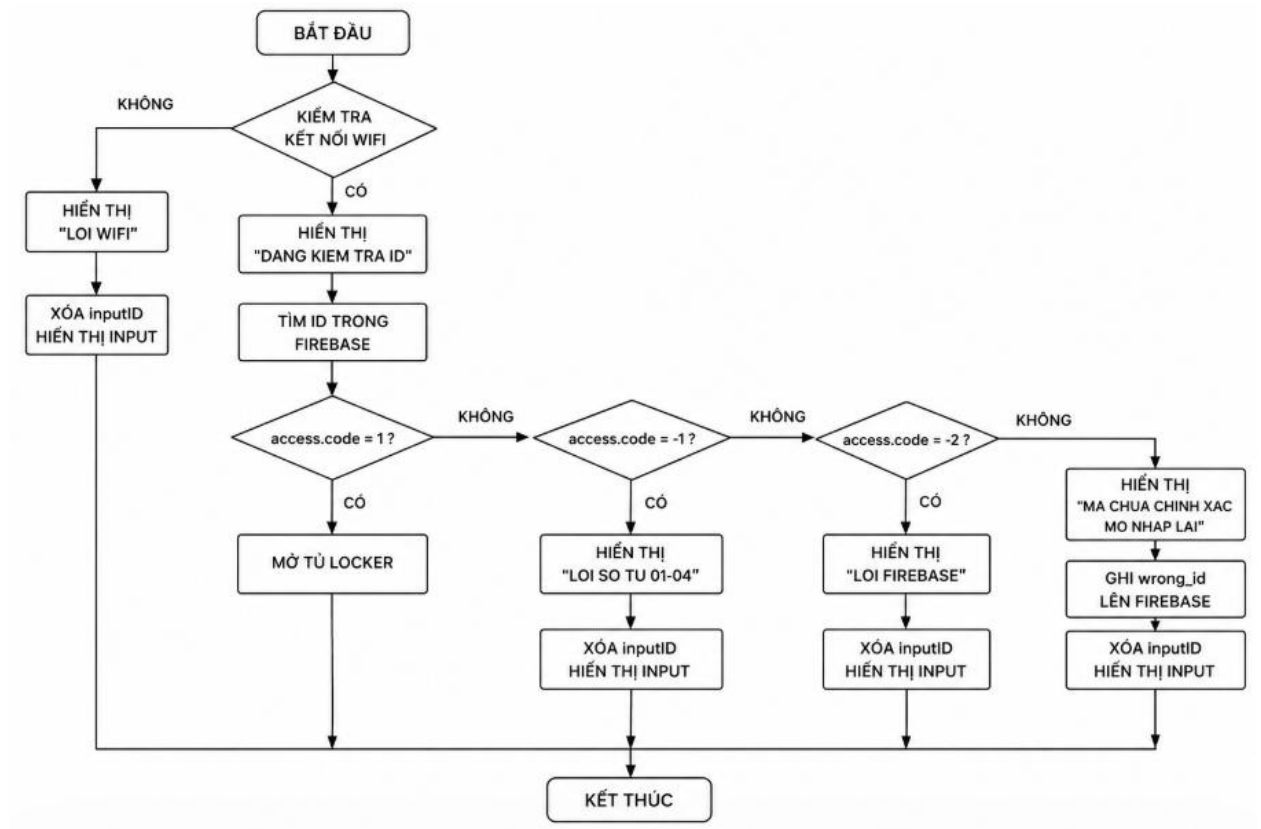
Đối với mỗi bản ghi, hệ thống kiểm tra bản ghi đó có đúng định dạng đối tượng dữ liệu hay không. Nếu không hợp lệ, chương trình bỏ qua bản ghi hiện tại và tiếp tục duyệt sang bản ghi kế tiếp. Nếu hợp lệ, chương trình lấy trường `current_pin` và thực hiện so sánh với mã PIN mà người dùng đã nhập.

Nếu mã PIN không trùng khớp, hệ thống tiếp tục quay lại vòng lặp để kiểm tra bản ghi tiếp theo trong Firebase. Quá trình này được lặp lại cho đến khi tìm thấy mã PIN phù hợp hoặc duyệt hết toàn bộ danh sách. Khi phát hiện mã PIN trùng khớp, chương trình lấy giá trị `box_no` tương ứng để xác định số tủ cần mở, sau đó kiểm tra phạm vi hợp lệ của số tủ từ 1 đến 4.

Nếu số tủ không hợp lệ, hệ thống trả về `code = -1`, biểu thị lỗi dữ liệu số tủ. Ngược lại, nếu số tủ hợp lệ, chương trình cập nhật các thông tin kết quả bao gồm số tủ (`boxNo`), đường dẫn bản ghi (`recordPath`) và khóa bản ghi (`recordKey`), sau đó trả về `code = 1` để xác nhận rằng người dùng có quyền mở tủ.

Trong trường hợp chương trình duyệt hết toàn bộ danh sách mà không tìm thấy mã PIN phù hợp, hệ thống sẽ giữ nguyên giá trị mặc định và trả về `code = 0`, biểu thị trạng thái không tìm thấy quyền truy cập tương ứng với mã PIN đã nhập.

3.5.2. Thiết kế giải thuật xử lý mã nhập từ người dùng



Hình 3.23. Lưu đồ quá trình kiểm tra mã ID

Lưu đồ này mô tả quá trình kiểm tra mã ID để mở tủ locker thông qua kết nối WiFi và dữ liệu trên Firebase.

Đầu tiên, hệ thống bắt đầu bằng việc kiểm tra trạng thái kết nối WiFi. Nếu thiết bị chưa kết nối WiFi thì hệ thống sẽ hiển thị thông báo “LOI WIFI”, sau đó xóa dữ liệu inputID, hiển thị lại màn hình nhập ID và đi đến kết thúc.

Nếu WiFi đã kết nối thành công, hệ thống sẽ hiển thị thông báo “ĐANG KIỂM TRA ID” để báo cho người dùng biết đang xử lý dữ liệu. Sau đó chương trình tiến hành tìm kiếm mã ID trong Firebase bằng hàm findAccessByID(inputID).

Sau khi kiểm tra, chương trình sẽ xét giá trị access.code.

- Nếu access.code = 1 nghĩa là mã ID hợp lệ, hệ thống sẽ thực hiện mở tủ locker tương ứng bằng hàm openLocker().

- Nếu `access.code = -1` nghĩa là số tử không hợp lệ (ngoài phạm vi từ 01 đến 04), hệ thống sẽ hiển thị thông báo “LOI SO TU 01-04”, sau đó xóa `inputID` và quay lại màn hình nhập.
- Nếu `access.code = -2` nghĩa là xảy ra lỗi khi truy cập Firebase, hệ thống sẽ hiển thị thông báo “LOI FIREBASE”, rồi xóa `inputID` và hiển thị lại màn hình nhập ID.
- Nếu không thuộc các trường hợp trên thì hệ thống xác định người dùng nhập sai mã ID. Khi đó màn hình sẽ hiển thị thông báo “MA CHUA CHINH XAC - MOI NHAP LAI”. Đồng thời hệ thống ghi trạng thái `wrong_id` cùng thời gian xảy ra lỗi lên Firebase bằng hàm `firebasePatchRaw()`. Sau đó dữ liệu `inputID` được xóa và giao diện nhập ID được hiển thị lại.

Cuối cùng, tất cả các nhánh xử lý đều hội tụ về một điểm kết thúc duy nhất của chương trình.

3.6. Thiết kế dịch vụ thông báo Email và SMS

Để nâng cao khả năng tương tác với người dùng, hệ thống AIoT Smart Retail được tích hợp các dịch vụ gửi Email và SMS nhằm tự động thông báo trạng thái đơn hàng, xác nhận giao dịch và hỗ trợ quản trị hệ thống. Hai dịch vụ này hoạt động độc lập với Firebase Realtime Database và được kích hoạt trực tiếp từ các giao diện Web Dashboard thông qua các API dịch vụ bên ngoài.

a. Dịch vụ Email

Hệ thống sử dụng nền tảng EmailJS để gửi email xác nhận đơn hàng và các thông báo liên quan đến quá trình mua hàng. Khi người dùng hoàn tất đơn hàng trên User Dashboard, hệ thống sẽ thu thập các thông tin cần thiết như mã đơn hàng, danh sách sản phẩm, tổng giá trị đơn hàng và địa chỉ email người nhận.

Sau khi dữ liệu được kiểm tra hợp lệ, User Dashboard gửi yêu cầu đến EmailJS thông qua API. EmailJS sẽ thực hiện quá trình tạo nội dung email theo mẫu đã được cấu hình và gửi trực tiếp đến hộp thư của khách hàng.

Kiến trúc này giúp hệ thống không cần xây dựng mail server riêng, giảm chi phí triển khai và đơn giản hóa quá trình phát triển hệ thống.

Luồng xử lý Email:

Người dùng đặt hàng → User Dashboard → EmailJS → Hộp thư người dùng

b. Dịch vụ SMS

Bên cạnh Email, hệ thống còn tích hợp dịch vụ SMS thông qua nền tảng eSMS nhằm hỗ trợ gửi thông báo nhanh đến người dùng hoặc quản trị viên.

Khi phát sinh các sự kiện quan trọng như đơn hàng mới Admin Dashboard có thể gửi yêu cầu đến hệ thống eSMS thông qua API. Nội dung tin nhắn sẽ được tạo tự động dựa trên dữ liệu hiện có trong hệ thống.

Sau khi tiếp nhận yêu cầu, eSMS thực hiện gửi tin nhắn đến số điện thoại được chỉ định và phản hồi trạng thái gửi về hệ thống.

Luồng xử lý SMS:

Admin Dashboard → eSMS API → Điện thoại người nhận

Hai dịch vụ này được triển khai dưới dạng dịch vụ bên ngoài (Third-Party Services), không tham gia lưu trữ dữ liệu nghiệp vụ mà chỉ đảm nhiệm chức năng truyền tải thông tin đến người dùng cuối.

CHƯƠNG 4: KẾT QUẢ TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1. Kết quả triển khai Frontend ReactJS

4.1.1. Giao diện trang chủ User Home

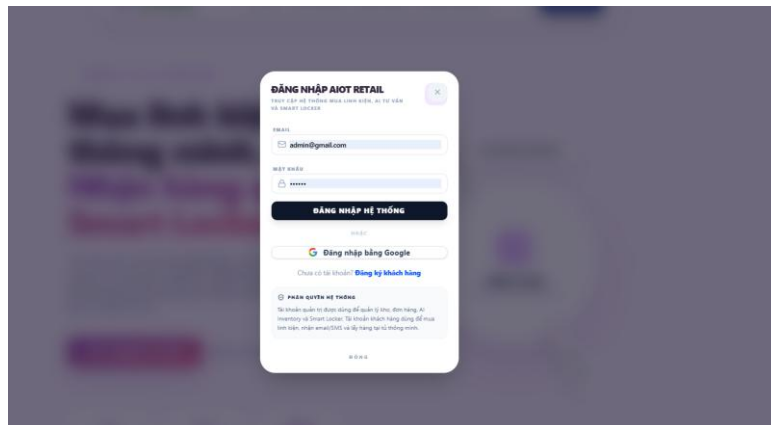


Hình 4.1. Giao diện trang chủ User Home

Hình trên thể hiện giao diện trang chủ của hệ thống AIoT Smart Retail sau khi triển khai hoàn chỉnh Frontend Web. Giao diện được thiết kế theo phong cách hiện đại nhằm giới thiệu mô hình mua linh kiện thông minh kết hợp AI và Smart Locker.

Kết quả cho thấy hệ thống đã tích hợp các chức năng chính như AI tự vận linh kiện, quản lý kho realtime và nhận hàng thông qua Smart Locker ESP32. Các thành phần AI, IoT và Firebase Realtime Database cũng được hiển thị trực quan trên giao diện.

Ngoài ra, thanh điều hướng và các nút chức năng đã hoạt động ổn định, giúp người dùng dễ dàng truy cập các module trong hệ thống. Điều này chứng minh Frontend đã được xây dựng hoàn chỉnh và sẵn sàng kết nối với Backend AIoT của đề tài.



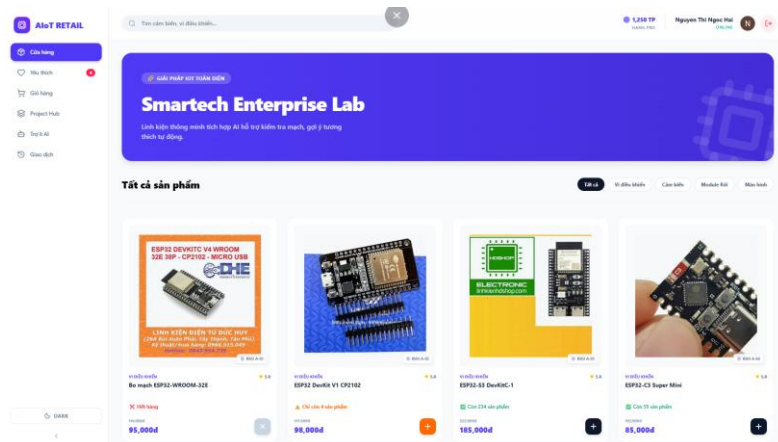
Hình 4.2. Giao diện đăng nhập và xác thực người dùng của hệ thống AIoT Retail

Hình trên thể hiện giao diện đăng nhập của hệ thống AIoT Retail sau khi triển khai chức năng xác thực người dùng. Hệ thống hỗ trợ đăng nhập bằng tài khoản email/mật khẩu và Google Authentication nhằm tăng tính thuận tiện và bảo mật cho người dùng.

Kết quả cho thấy giao diện đã tích hợp đầy đủ các chức năng như đăng nhập, đăng ký khách hàng và phân quyền hệ thống. Phần mô tả bên dưới cũng thể hiện sự phân chia quyền giữa quản trị viên và khách hàng trong quá trình sử dụng hệ thống Smart Retail và Smart Locker.

Ngoài ra, giao diện popup đăng nhập hoạt động ổn định với bố cục trực quan và dễ sử dụng, chứng minh hệ thống Frontend đã kết nối thành công với cơ chế xác thực người dùng của Backend/Firebase Authentication.

4.1.2. Giao diện User Dashboard

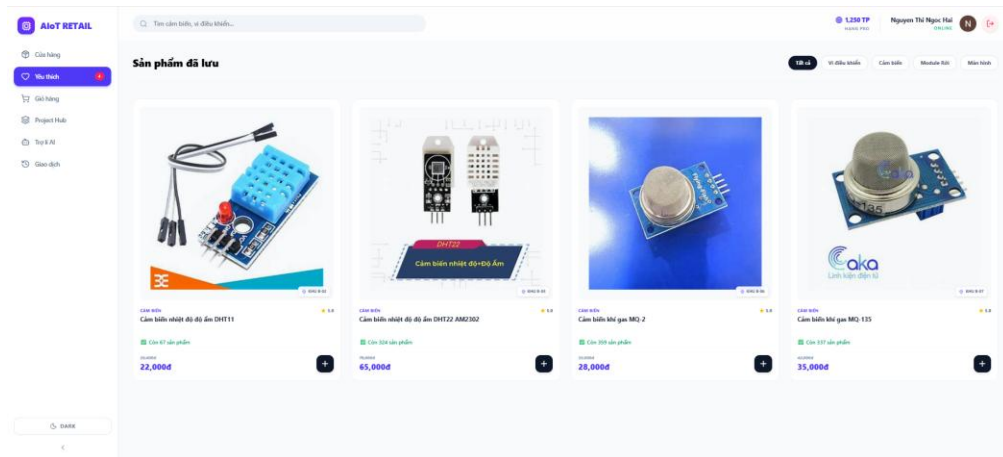


Hình 4.3. Giao diện hiển thị danh sách sản phẩm của hệ thống AIoT Retail

Hình trên thể hiện kết quả triển khai giao diện hiển thị sản phẩm của hệ thống AIoT Retail. Các linh kiện điện tử được hiển thị dưới dạng card gồm hình ảnh, tên sản phẩm, vị trí kho và giá bán nhằm hỗ trợ người dùng dễ dàng tìm kiếm và lựa chọn linh kiện.

Kết quả cho thấy hệ thống đã phân loại sản phẩm theo từng danh mục như vi điều khiển, cảm biến và module rời. Ngoài ra, chức năng tìm kiếm và lọc sản phẩm cũng hoạt động ổn định, giúp tăng khả năng quản lý và truy xuất linh kiện trong hệ thống Smart Retail.

Giao diện được xây dựng trực quan, đồng bộ màu sắc và hỗ trợ hiển thị dữ liệu động từ cơ sở dữ liệu thời gian thực, chứng minh Frontend ReactJS đã kết nối thành công với Backend và Firebase.

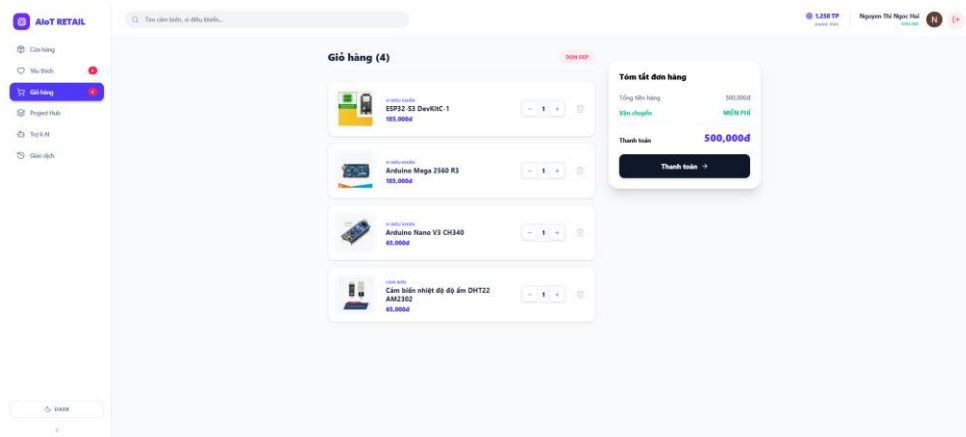


Hình 4.4. Giao diện danh sách sản phẩm yêu thích của hệ thống AIoT Retail

Hình trên thể hiện giao diện quản lý sản phẩm yêu thích của hệ thống AIoT Retail. Người dùng có thể lưu các linh kiện thường sử dụng để hỗ trợ tìm kiếm và mua hàng nhanh hơn.

Kết quả cho thấy hệ thống đã hiển thị thành công thông tin sản phẩm gồm hình ảnh, tên linh kiện, giá bán, số lượng tồn kho và vị trí lưu trữ trong kho. Các sản phẩm cảm biến như DHT11, DHT22 và MQ-2 được hiển thị trực quan dưới dạng card.

Ngoài ra, chức năng lưu sản phẩm yêu thích và đồng bộ dữ liệu theo thời gian thực đã hoạt động ổn định, chứng minh hệ thống Frontend đã kết nối thành công với cơ sở dữ liệu Firebase/Backend.

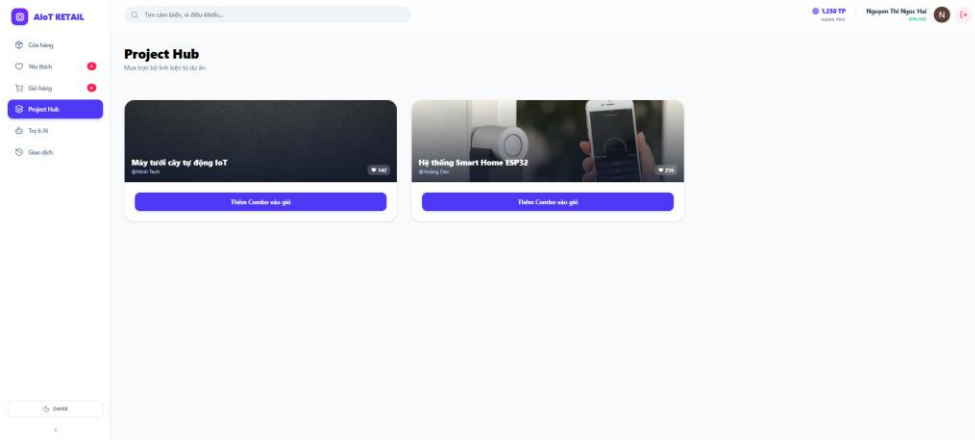


Hình 4.5. Giao diện giỏ hàng và thanh toán của hệ thống AIoT Retail

Hình trên thể hiện giao diện giỏ hàng của hệ thống AIoT Retail sau khi triển khai chức năng mua linh kiện trực tuyến. Người dùng có thể xem danh sách sản phẩm đã chọn, điều chỉnh số lượng hoặc xóa sản phẩm khỏi giỏ hàng.

Kết quả cho thấy hệ thống đã hiển thị đầy đủ thông tin gồm tên linh kiện, giá bán, số lượng và tổng giá trị đơn hàng. Chức năng cập nhật số lượng và tính toán tổng tiền hoạt động ổn định theo thời gian thực.

Ngoài ra, module thanh toán đã được tích hợp trực quan trên giao diện, giúp người dùng dễ dàng thực hiện quy trình đặt hàng trong hệ thống Smart Retail.

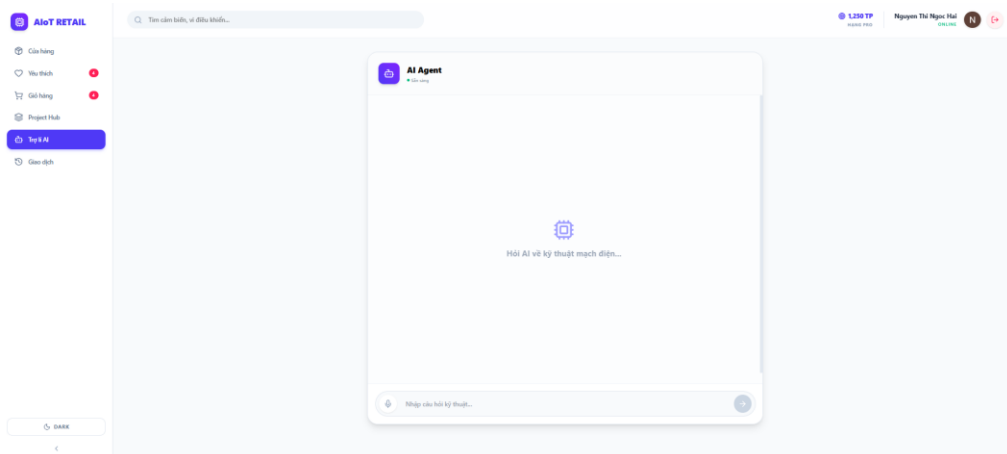


Hình 4.6. Giao diện Project Hub của hệ thống AIoT Retail

Hình trên thể hiện giao diện Project Hub của hệ thống AIoT Retail, cho phép người dùng lựa chọn nhanh các bộ linh kiện theo từng dự án IoT thực tế như hệ thống tưới cây tự động hoặc Smart Home ESP32.

Kết quả cho thấy hệ thống đã hỗ trợ hiển thị các project dưới dạng card trực quan kèm nút thêm combo vào giỏ hàng. Chức năng này giúp người dùng dễ dàng lựa chọn đầy đủ linh kiện cần thiết cho từng mô hình mà không cần tìm kiếm thủ công từng thiết bị.

Ngoài ra, giao diện Project Hub hoạt động ổn định và hỗ trợ liên kết với hệ thống giỏ hàng, góp phần nâng cao trải nghiệm người dùng trong hệ thống Smart Retail.

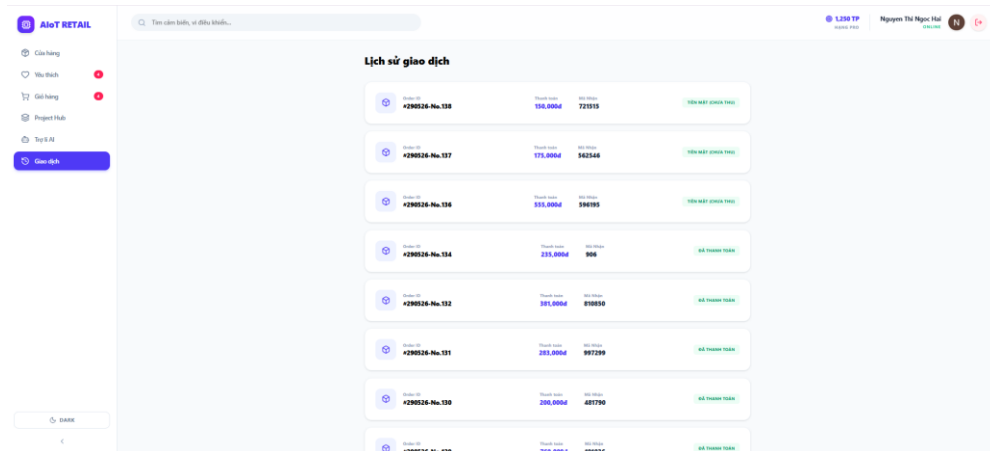


Hình 4.7. Giao diện trợ lý AI của hệ thống AIoT Retail

Hình trên thể hiện giao diện trợ lý AI được tích hợp trong hệ thống AIoT Retail nhằm hỗ trợ người dùng tư vấn kỹ thuật và lựa chọn linh kiện điện tử. Người dùng có thể nhập câu hỏi trực tiếp để trao đổi với AI Agent.

Kết quả cho thấy hệ thống đã triển khai thành công giao diện chatbot AI với khả năng tương tác thời gian thực. Module AI được thiết kế trực quan, hỗ trợ nhập văn bản và xử lý câu hỏi liên quan đến mạch điện, IoT và linh kiện điện tử.

Ngoài ra, giao diện AI Agent hoạt động ổn định và sẵn sàng kết nối với Backend AI sử dụng LangGraph, RAG và Qdrant trong hệ thống Smart Retail.



The screenshot displays the 'AIoT RETAIL' web application. On the left is a sidebar menu with options: 'Cửa hàng', 'Yêu thích', 'Giỏ hàng', 'Project Hub', 'Trợ lý AI', and 'Giao dịch'. The main content area is titled 'Lịch sử giao dịch' (Transaction History) and contains a table with 8 rows of transaction data. Each row includes a transaction ID, a date, a transaction amount, a balance, and a status indicator.

Mã đơn hàng	Ngày giao dịch	Giá trị thanh toán	Số dư	Trạng thái
#290526-Nhà.138	20/05/2024	100.000đ	727015	Đã thanh toán
#290526-Nhà.137	20/05/2024	175.000đ	552546	Đã thanh toán
#290526-Nhà.136	20/05/2024	335.000đ	217546	Đã thanh toán
#290526-Nhà.134	20/05/2024	135.000đ	800	Đã thanh toán
#290526-Nhà.133	20/05/2024	195.000đ	810050	Đã thanh toán
#290526-Nhà.131	20/05/2024	205.000đ	907299	Đã thanh toán
#290526-Nhà.130	20/05/2024	200.000đ	407799	Đã thanh toán
#290526-Nhà.129	20/05/2024	700.000đ	101034	Đã thanh toán

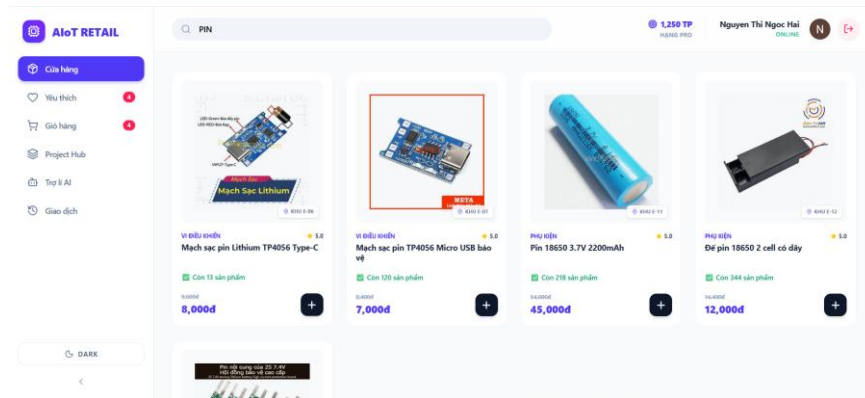
Hình 4.8. Giao diện lịch sử giao dịch của hệ thống AIoT Retail

Hình trên thể hiện giao diện lịch sử giao dịch của hệ thống AIoT Retail sau khi triển khai chức năng quản lý đơn hàng. Hệ thống hiển thị danh sách các giao dịch đã thực hiện gồm mã đơn hàng, giá trị thanh toán và trạng thái xử lý.

Kết quả cho thấy dữ liệu giao dịch được đồng bộ và cập nhật thành công theo thời gian thực. Người dùng có thể dễ dàng theo dõi trạng thái đơn hàng và quá trình nhận hàng trong hệ thống Smart Locker.

Ngoài ra, giao diện được thiết kế trực quan và hỗ trợ quản lý giao dịch ổn định, góp phần nâng cao khả năng theo dõi và quản lý đơn hàng trong hệ thống Smart Retail.

KẾT QUẢ BỘ LỌC DANH MỤC LINH KIỆN



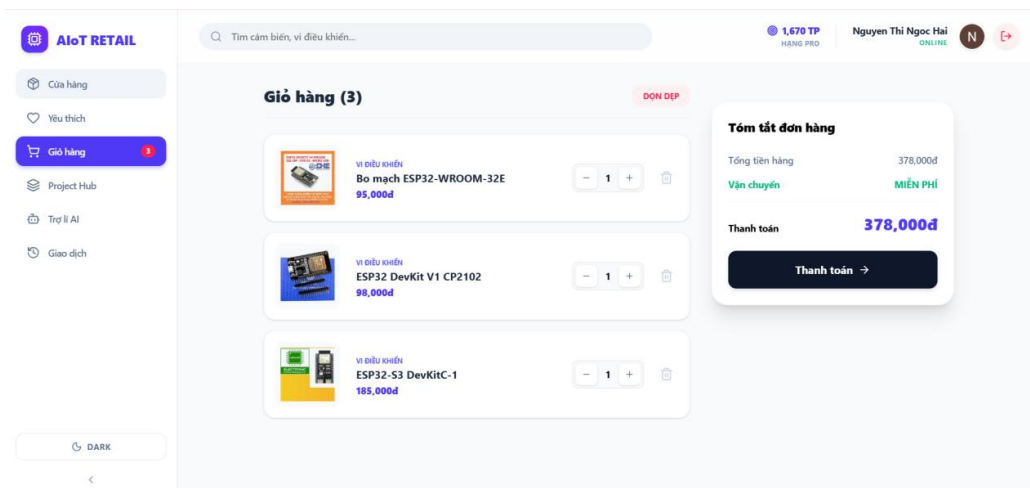
Hình 4.9. Kết quả tìm kiếm linh kiện theo từ khóa trong hệ thống AIoT Retail

Hình trên thể hiện kết quả chức năng tìm kiếm linh kiện theo từ khóa trong hệ thống AIoT Retail. Khi người dùng nhập từ khóa “PIN”, hệ thống tự động truy xuất và hiển thị các sản phẩm liên quan như pin 18650, mạch sạc TP4056 và đế pin.

Kết quả cho thấy chức năng tìm kiếm hoạt động ổn định với khả năng lọc dữ liệu nhanh và hiển thị sản phẩm theo thời gian thực. Ngoài ra, thông tin về giá bán, số lượng tồn kho và vị trí lưu trữ cũng được cập nhật đầy đủ trên giao diện.

Chức năng này giúp người dùng dễ dàng tìm kiếm linh kiện cần thiết, góp phần nâng cao trải nghiệm sử dụng và tối ưu khả năng quản lý dữ liệu sản phẩm trong hệ thống Smart Retail.

KẾT QUẢ THÊM SẢN PHẨM VÀO GIỎ HÀNG

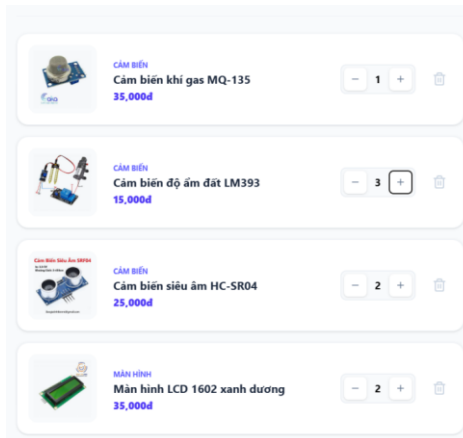


Hình 4.10. Kết quả thêm sản phẩm vào giỏ hàng trong hệ thống AIoT Retail

Khi click vào nút bấm dấu cộng (+) tại các thẻ component sản phẩm, Frontend ReactJS kích hoạt hàm quản lý trạng thái addToCart. Hệ thống tiến hành bóc tách mã định danh sản phẩm (id) để đối chiếu với mảng dữ liệu hiện hành trong giỏ hàng (cart).

Nếu sản phẩm chưa tồn tại trong mảng, hệ thống khởi tạo một đối tượng mới với cấu trúc dữ liệu gồm: tên linh kiện, ảnh, đơn giá, vị trí kho và gán thuộc tính số lượng mặc định qty: 1. Song song với tiến trình này, biến trạng thái chỉ mục giỏ hàng trên thanh Header tự động tính toán lại tổng số lượng phần tử bằng hàm .length để cập nhật số hiển thị trực quan, giúp người dùng nhận biết trạng thái thêm hàng mà không cần kích hoạt tiến trình tải lại trang.

KẾT QUẢ CẬP NHẬT SỐ LƯỢNG SẢN PHẨM

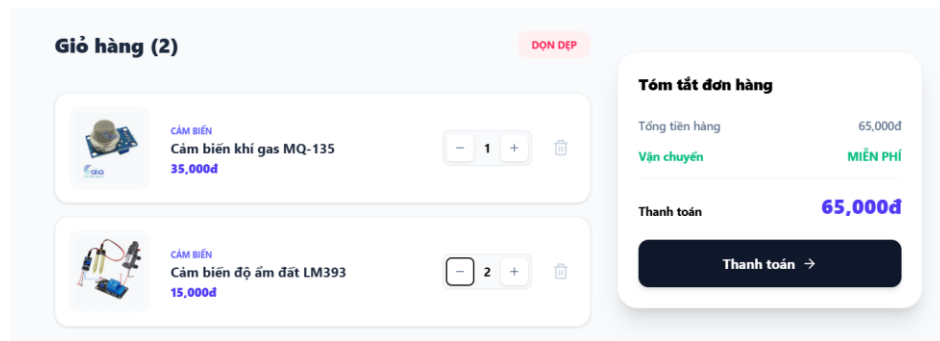


Hình 4.11. Kết quả cập nhật số lượng sản phẩm trong giỏ hàng

Tại giao diện quản lý giỏ hàng, việc thay đổi số lượng được xử lý qua hàm updateCartQuantity(productId, newQty). Luồng logic của hàm thiết lập hai điều kiện kiểm tra giới hạn biên:

- Chặn dưới: Nếu người dùng bấm giảm số lượng khi biến trạng thái số lượng đang ở mức tối thiểu (qty == 1), nút giảm sẽ kích hoạt hàm xóa sản phẩm khỏi mảng, ngăn chặn lỗi số lượng bằng 0 hoặc số âm.
- Chặn trên: Hệ thống so khớp biến newQty với trường dữ liệu tồn kho stock được lấy từ Firebase Realtime Database (Ví dụ: đối chiếu với thông số tồn kho thực tế của mạch sạc). Nếu vượt quá số lượng trong kho, hệ thống tự động khóa lệnh tăng và giữ nguyên giá trị biên trên.

KẾT QUẢ TÍNH TOÁN TỔNG TIỀN TỰ ĐỘNG



Hình 4.12. Kết quả tính toán tổng tiền tự động trong giỏ hàng

Hình 4.12 thể hiện kết quả kiểm thử chức năng tính toán tổng tiền được thực thi tự động phía Frontend khi có sự thay đổi về số lượng (qty) hoặc số chủng loại sản phẩm trong giỏ hàng.

Cảm biến khí gas MQ-135: price=35,000đ, qty=1 → Tích số=35,000đ.

Cảm biến độ ẩm đất LM393: price=15,000đ, qty=2 → Tích số=30,000đ.

Hệ thống thực hiện cộng dồn giá trị của các phần tử trong mảng:

Tổng số tiền = 35,000đ + 30,000đ = 65,000đ

Giá trị này sau đó được Frontend chuyển đổi qua hàm định dạng `.toLocaleString('vi-VN')` để chèn dấu phân tách hàng nghìn và hiển thị chuỗi ký tự 65,000đ đồng bộ tại hai trường Tổng tiền hàng và Thanh toán ở khối tóm tắt đơn hàng bên phải. Kết quả kiểm thử cho thấy bộ tính toán xử lý chính xác tuyệt đối, dữ liệu cập nhật theo thời gian thực và không xảy ra lỗi xô lệch bố cục giao diện.

KẾT QUẢ TẠO ĐƠN HÀNG THỜI GIAN THỰC

✓ Xác nhận đặt hàng thành công
Cần xin báo để mua sản tại SmartLocker

CHI TIẾT ĐƠN HÀNG
Mã đơn hàng: 010626-No.141 Ngày đặt: 14/10/2023 15:02:23
Khách hàng: Nguyễn Thị Ngọc Hài

DANH SÁCH SẢN PHẨM

Sản phẩm	SL	Giá	Thành tiền
Mành nắp pin TP4056 Micro USB bảo vệ	1	7.000đ	7.000đ
Mành nắp pin Lithium TP4056 Type-C	1	8.000đ	8.000đ
Tổng sản phẩm:			15.000đ
MST (VAT):			
Tổng cộng:			15.000đ

PHƯƠNG THỨC THANH TOÁN
Thanh toán tiền mặt
(Tiền mặt (Chưa thu))

MÃ LẤY HÀNG (NHẬN TẠI QUẦY)
Mã của bạn: 317
Vui lòng giữ email này để tra cứu thông tin đơn hàng của bạn.

THÔNG TIN KHÁCH HÀNG
Tên: Nguyễn Thị Ngọc Hài Email: haitn@vnn.vn

Cần gọi 0971 646 846 hỗ trợ: support@vnn.vn
© 2020 SmartLocker V.T. Thôn - Cần xin báo để mua đặt

Đặt hàng thành công

Đặt hàng thành công!
Cảm ơn bạn đã tin tưởng AIoT Retail

SỐ THỨ TỰ CỦA BẠN
#317

THANH TOÁN TIỀN MẶT
Vui lòng thanh toán tại quầy khi đọc số thứ tự nhận hàng.

MÃ ĐƠN
010626-No.141

HOÀN TẤT & ĐÓNG

Hình 4.13. Kết quả đặt hàng và tạo mã nhận hàng trong hệ thống AIoT Retail

Hình trên thể hiện kết quả sau khi người dùng hoàn tất quá trình đặt hàng trong hệ thống AIoT Retail. Hệ thống tự động tạo hóa đơn điện tử và cấp mã nhận hàng để sử dụng tại Smart Locker hoặc quầy nhận hàng.

Kết quả cho thấy thông tin đơn hàng gồm mã đơn, danh sách sản phẩm, tổng thanh toán và phương thức thanh toán được hiển thị đầy đủ. Ngoài ra, hệ thống cũng sinh mã nhận hàng tự động giúp người dùng dễ dàng nhận linh kiện tại tủ Smart Locker.

Chức năng này hoạt động ổn định và góp phần tự động hóa quy trình mua hàng, thanh toán và nhận hàng trong hệ thống Smart Retail.

KẾT QUẢ LỰA CHỌN HÌNH THỨC NHẬN HÀNG

Thông tin thanh toán

1 Thông tin liên lạc

Số điện thoại (Nhận SMS/MMS từ)

Nhập SĐT (Ví dụ: 0344881004)

Email (Nhận hóa đơn chi tiết)

22161249@student.hcmute.edu.vn

* Hệ thống chấp nhận SĐT, Email hoặc số fax để thông báo.

2 Thanh toán

☐ Chuyển khoản ☐ Tiền mặt

3 Hình thức nhận hàng

☐ Tủ Smart Locker
Lấy hàng từ tủ thông minh và/hoặc mã PIN

☐ Nhận tại quầy
Lấy trực tiếp tại quầy hàng và/hoặc từ tủ

☐ Giao hàng tận nơi
Shipper giao an toàn đến địa chỉ của bạn

Cần thanh toán: **235,000 đ**

XÁC NHẬN ĐẶT HÀNG

Hình 4.14. Giao diện thanh toán và lựa chọn hình thức nhận hàng của hệ thống AIoT Retail

Hình 4.14 thể hiện kết quả kiểm thử phân hệ rẽ nhánh kịch bản điều phối đơn hàng thông qua biến trạng thái lựa chọn phương thức giao nhận (deliveryMethod). Các giá trị cấu hình từ giao diện Client được bóc tách và xử lý ngầm theo 3 luồng logic phần cứng và phần mềm độc lập:

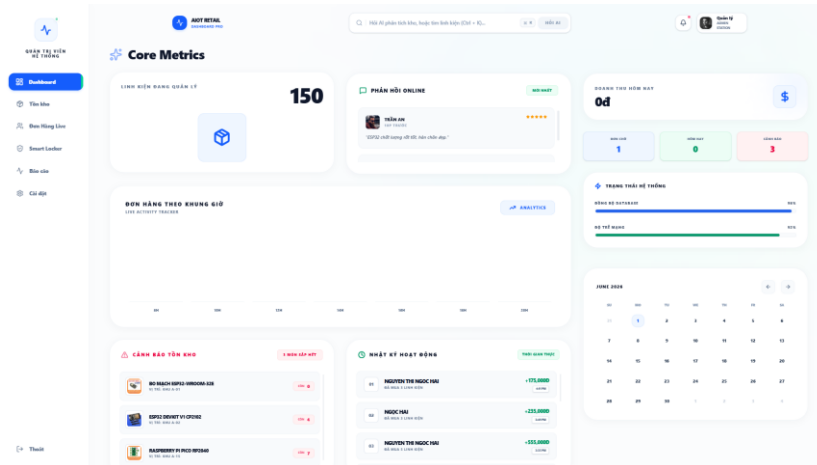
Luồng Tủ Smart Locker (method: "locker"): Khi người dùng tick chọn hình thức này, hệ thống sẽ liên kết đơn hàng với phân hệ quản lý tủ đồ tự động. Frontend kích hoạt luồng sinh mã PIN bảo mật ngẫu nhiên, đồng thời gửi lệnh ghi trạng thái xuống node lockers/ trên Firebase Realtime Database nhằm điều khiển mạch ESP32 kích hoạt rơ-le mở chốt khóa Solenoid khi người dùng nhập mã xác thực tại tủ.

Luồng Nhận tại quầy (method: "counter"): Khi hình thức nhận tại quầy được kích hoạt, luồng xử lý sẽ định tuyến thông tin đơn hàng sang node queue_display/. Sau khi hoàn tất thanh toán, hệ thống tự động cấp phát số thứ tự (Token) để đẩy vào hàng đợi hiển thị trên bảng LED Kiosk công cộng.

Luồng Giao hàng tận nơi (method: "delivery"): Đây là tùy chọn vận chuyển ngoại vi truyền thống, hệ thống sẽ đóng gói thông tin Số điện thoại và Email liên lạc trích xuất từ phân mục số 1 để đẩy sang giao diện quản trị của nhân viên giao hàng mà không can thiệp vào các kịch bản điều khiển phần cứng trực tiếp.

Kết quả thực nghiệm chứng minh các thành phần Radio Button tương tác đúng mục tiêu, bộ liên kết dữ liệu bắt giữ chính xác thông tin đầu vào của khách hàng và cập nhật số tiền 235,000đ đồng bộ trước khi lệnh XÁC NHẬN ĐẶT HÀNG được gửi đi.

4.1.3. Giao diện Admin Dashboard



Hình 4.15. Giao diện Dashboard quản lý hệ thống IoT Retail

Hình 4.15 cho kết quả kết xuất giao diện giám sát trung tâm của quản trị viên khi điều kiện `activeTab === 'dashboard'` thỏa mãn. Các thành phần dữ liệu bóc tách từ cơ sở dữ liệu và biến trạng thái nội bộ được phân rã thành 4 phân khu chức năng:

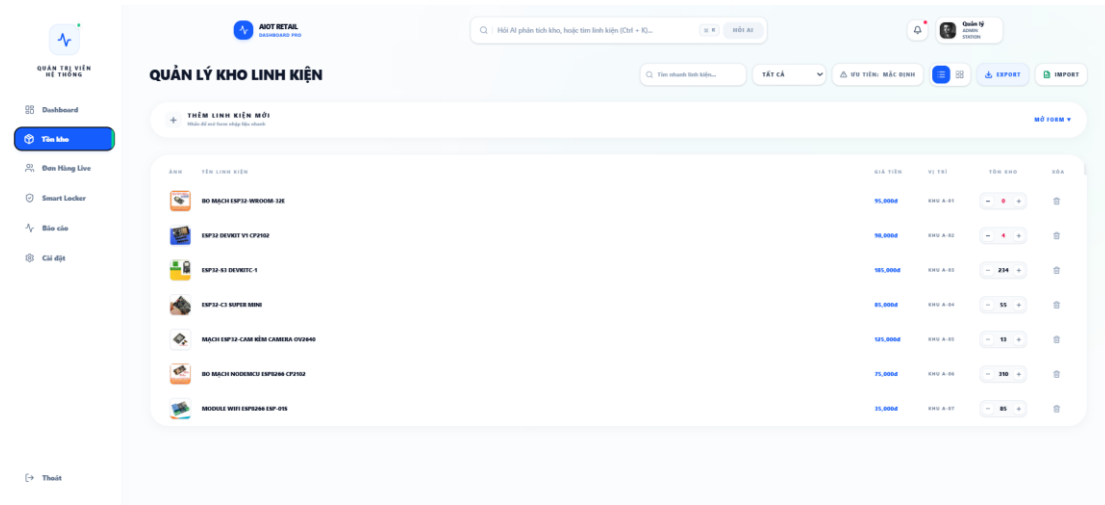
Khối thống kê kho và đơn hàng: Thẻ thành phần Linh kiện đang quản lý thực hiện đo độ dài mảng dữ liệu `products.length` và kết xuất chính xác giá trị số 150 lên UI. Khối đếm trạng thái đơn hàng trích xuất giá trị từ mảng `waitingCarts` để hiển thị số lượng đơn chờ xử lý là 3, trong khi biến tổng doanh thu tích lũy trong ngày `dailyRevenue` khởi tạo giá trị số hiển thị là 0đ.

Khối giám sát hiệu năng hệ thống: Khung trạng thái hệ thống (TRẠNG THÁI HỆ THỐNG) bóc tách hai trường dữ liệu thông số và render thông qua cấu trúc component `motion.div` của thư viện Framer Motion. Kết quả ghi nhận thanh tiến trình đồ họa hiển thị tỷ lệ đồng bộ Database đạt mức 98% và độ trễ mạng đạt 92%.

Khối tương tác khách hàng và Nhật ký (Logs): Ô Phản hồi Online bóc tách mảng dữ liệu review tĩnh và hiển thị bản ghi mới nhất của tài khoản "Trần An" với nội dung chuỗi ký tự: "ESP32 chất lượng rất tốt, hàn chân đẹp.". Phân khu cảnh báo tồn kho vật lý giữ trạng thái trống (Kho hàng an toàn) do số lượng phần tử của mảng lọc `lowStockProducts` chưa vượt ngưỡng cấu hình biên dưới.

Phần hệ dòng lệnh System Terminal Log: Đặt cố định dưới chân trang giao diện, thực hiện chức năng bắt giữ luồng ký tự nhật ký từ biến mảng systemLogs. Phần khu này hiển thị dòng thông báo hệ thống đang hoạt động ổn định và sẵn sàng bắt tay tín hiệu thời gian thực thông qua giao thức WebSockets với phần cứng.

Nhìn chung, các component tuân thủ cấu trúc Grid Layout, dữ liệu số và lịch biểu (CalendarCard) hiển thị đồng bộ, căn biên theo đúng lề cấu trúc và không xảy ra lỗi xô lệch ký tự hay xung đột render đồ họa trên trình duyệt Client.

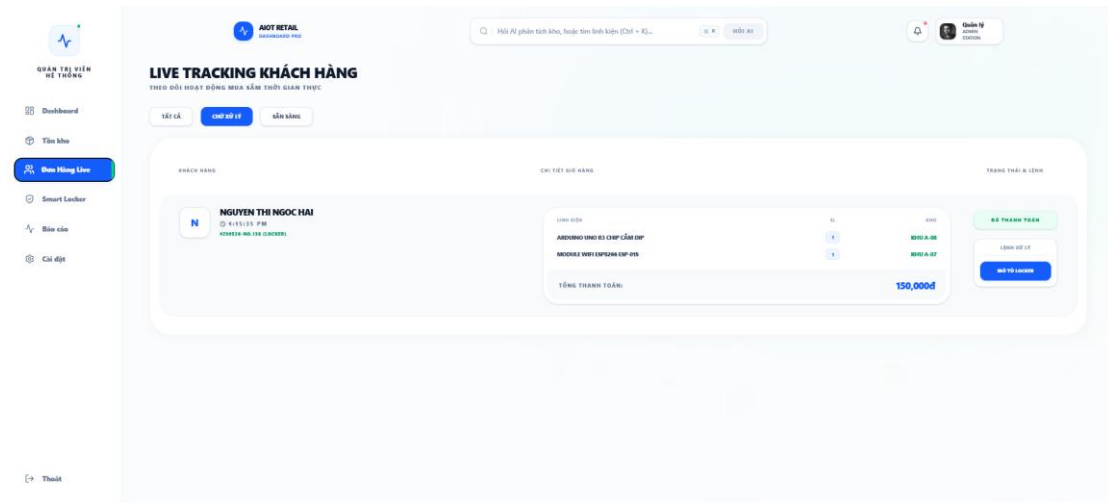


Hình 4.16. Giao diện quản lý kho linh kiện của hệ thống AIoT Retail

Hình trên thể hiện giao diện quản lý kho linh kiện của hệ thống AIoT Retail dành cho quản trị viên. Hệ thống hỗ trợ theo dõi danh sách linh kiện, vị trí lưu trữ, giá bán và số lượng tồn kho theo thời gian thực.

Kết quả cho thấy chức năng thêm, cập nhật và quản lý linh kiện đã hoạt động ổn định. Ngoài ra, hệ thống còn hỗ trợ tìm kiếm nhanh, import/export dữ liệu và điều chỉnh số lượng tồn kho trực tiếp trên giao diện.

Giao diện quản lý kho được thiết kế trực quan, giúp quản trị viên dễ dàng theo dõi và kiểm soát trạng thái linh kiện trong hệ thống Smart Retail và Smart Locker.

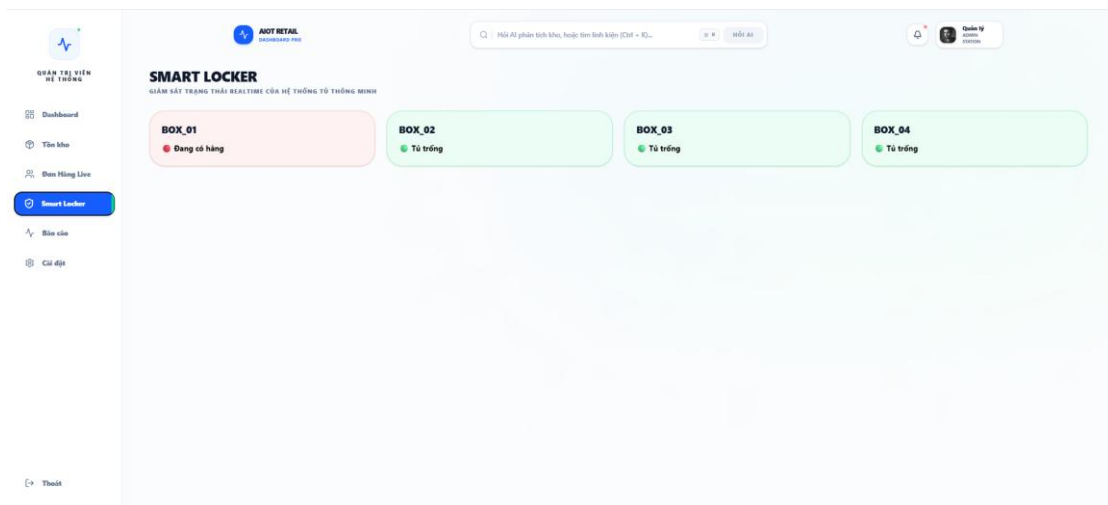


Hình 4.17. Giao diện theo dõi đơn hàng thời gian thực của hệ thống AIOt Retail

Hình trên thể hiện giao diện Live Tracking dùng để theo dõi hoạt động mua hàng theo thời gian thực trong hệ thống AIOt Retail. Hệ thống hiển thị thông tin khách hàng, danh sách linh kiện đã mua, vị trí kho và tổng giá trị đơn hàng.

Kết quả cho thấy trạng thái đơn hàng và dữ liệu giỏ hàng đã được cập nhật thành công theo thời gian thực. Ngoài ra, quản trị viên có thể thực hiện lệnh xử lý như mở tủ Smart Locker trực tiếp trên giao diện.

Giao diện hoạt động ổn định và hỗ trợ giám sát quá trình mua hàng, xử lý đơn và nhận hàng trong hệ thống Smart Retail.

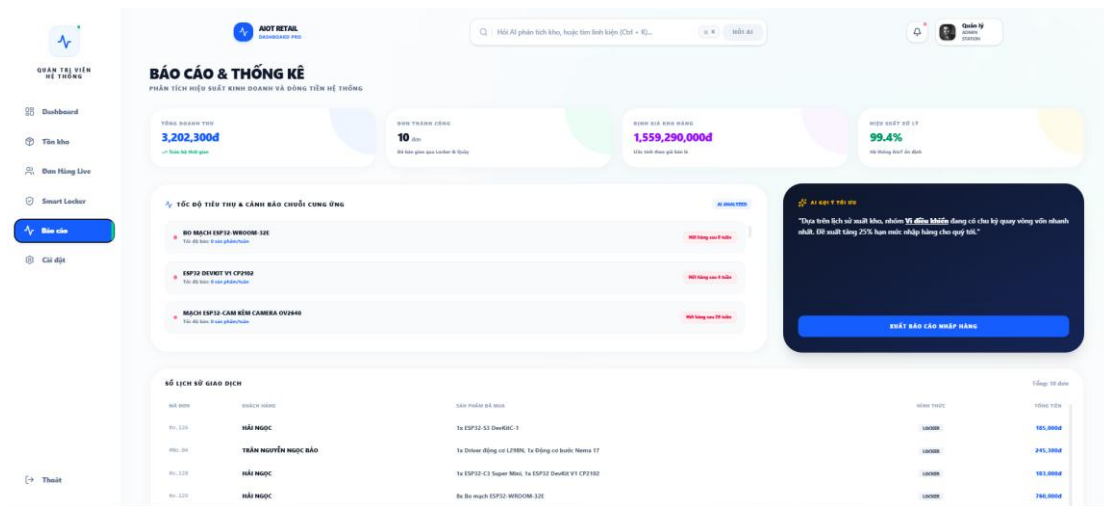


Hình 4.18. Giao diện giám sát Smart Locker của hệ thống AIoT Retail

Hình trên thể hiện giao diện giám sát trạng thái Smart Locker trong hệ thống AIoT Retail. Hệ thống hỗ trợ theo dõi trạng thái các ngăn tủ theo thời gian thực nhằm phục vụ quá trình nhận hàng tự động.

Kết quả cho thấy trạng thái từng ngăn tủ như BOX_01, BOX_02, BOX_03 và BOX_04 được cập nhật trực quan với các trạng thái “Đang có hàng” hoặc “Tủ trống”. Điều này giúp quản trị viên dễ dàng theo dõi tình trạng hoạt động của hệ thống Smart Locker.

Ngoài ra, giao diện hoạt động ổn định và hỗ trợ đồng bộ dữ liệu realtime giữa Web, Firebase và thiết bị IoT ESP32 trong hệ thống Smart Retail.

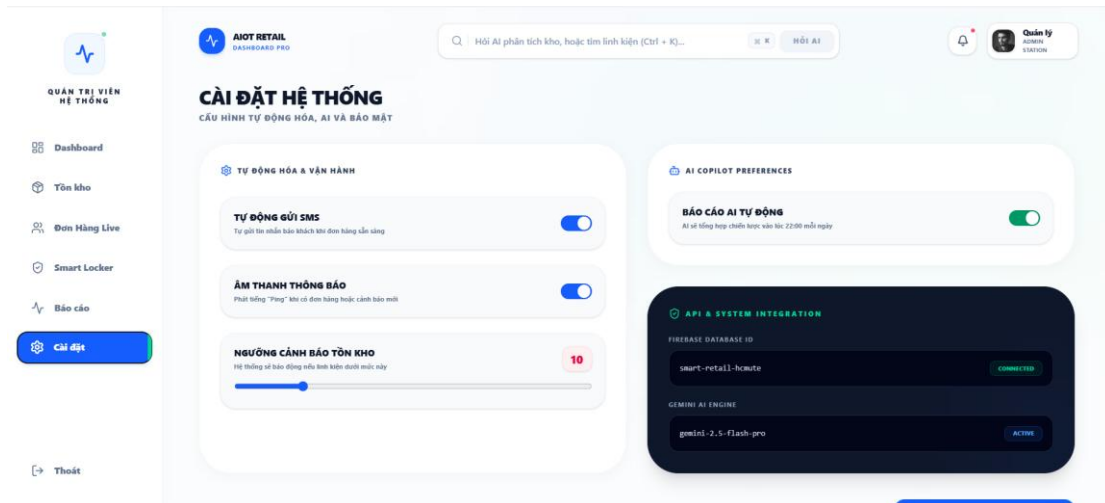


Hình 4.19. Giao diện báo cáo và thống kê của hệ thống AIoT Retail

Hình trên thể hiện giao diện báo cáo và thống kê của hệ thống AIoT Retail sau khi triển khai chức năng phân tích dữ liệu hoạt động. Hệ thống hiển thị các thông tin như tổng doanh thu, số đơn hàng thành công, hiệu suất xử lý và lịch sử giao dịch.

Kết quả cho thấy hệ thống đã hỗ trợ theo dõi tốc độ tiêu thụ linh kiện và cảnh báo chuỗi cung ứng theo thời gian thực. Ngoài ra, AI còn đưa ra gợi ý nhập hàng dựa trên dữ liệu bán hàng và tồn kho nhằm hỗ trợ quản lý hiệu quả hơn.

Giao diện báo cáo hoạt động ổn định, giúp quản trị viên dễ dàng theo dõi tình hình kinh doanh và đánh giá hiệu suất hoạt động của hệ thống Smart Retail.



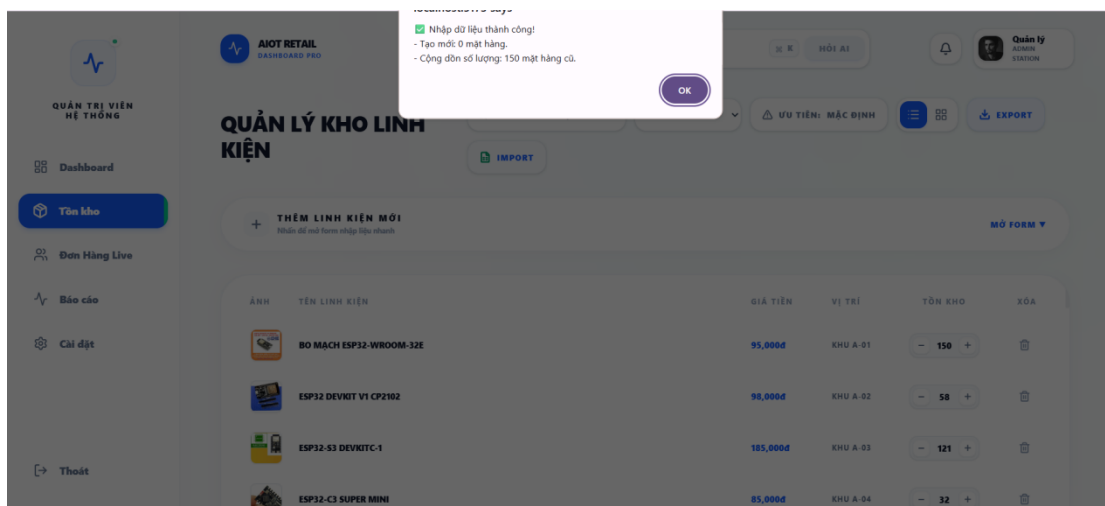
Hình 4.20. Giao diện cài đặt hệ thống của AIoT Retail

Hình trên thể hiện giao diện cài đặt và cấu hình hệ thống của AIoT Retail. Hệ thống hỗ trợ quản trị viên thiết lập các chức năng tự động hóa như gửi SMS, âm thanh thông báo và ngưỡng cảnh báo tồn kho.

Kết quả cho thấy các module AI và Firebase đã được kết nối thành công, đồng thời hỗ trợ cấu hình báo cáo AI tự động và lưu cấu hình trực tiếp trên giao diện quản trị.

Ngoài ra, giao diện hoạt động ổn định và hỗ trợ quản lý cấu hình hệ thống Smart Retail theo thời gian thực, giúp nâng cao khả năng vận hành và giám sát toàn bộ hệ thống.

KẾT QUẢ IMPORT CSV



Hình 4.21. Kết quả cập nhật tồn kho linh kiện và thông báo realtime

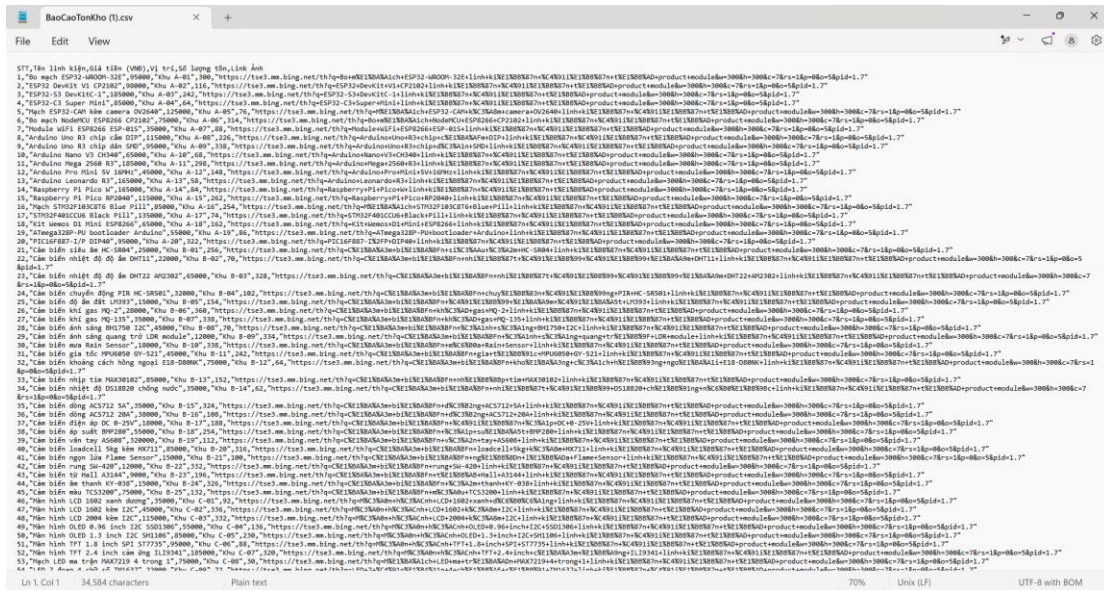
Hình trên thể hiện chức năng quản lý tồn kho linh kiện trong hệ thống AIoT Smart Retail. Khi quản trị viên thực hiện thao tác nhập thêm linh kiện, hệ thống tự động cập nhật số lượng

tồn kho và hiển thị thông báo realtime trên giao diện người dùng. Trong ví dụ minh họa, hệ thống đã cập nhật thành công số lượng của mặt hàng và hiển thị thông báo “Nhập dữ liệu thành công”, đồng thời tăng tổng số lượng linh kiện thêm 150 sản phẩm.

Kết quả này cho thấy hệ thống Web ReactJS đã kết nối và đồng bộ dữ liệu thành công với Firebase Realtime Database. Sau khi dữ liệu được cập nhật từ phía Backend/Firebase, giao diện quản lý kho tự động render lại danh sách linh kiện mà không cần tải lại trang. Điều này giúp đảm bảo tính realtime và hỗ trợ quản lý kho hiệu quả hơn.

Ngoài ra, hệ thống còn hỗ trợ thao tác tăng/giảm số lượng trực tiếp trên giao diện thông qua các nút điều khiển, giúp người quản trị dễ dàng theo dõi và cập nhật trạng thái tồn kho của từng linh kiện trong kho hàng.

KẾT QUẢ EXPORT CSV



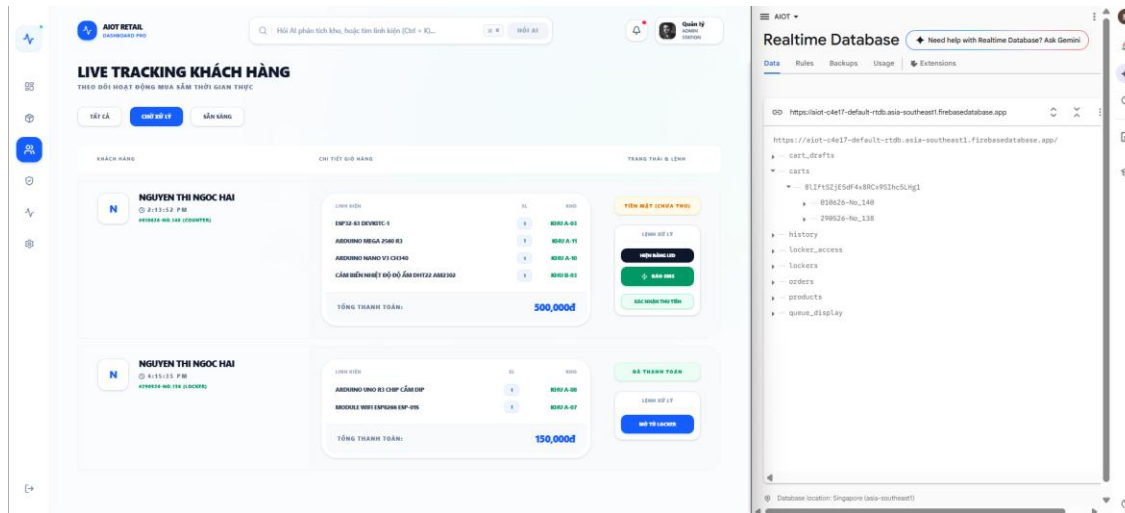
Hình 4.22. Kết quả xuất báo cáo tồn kho ra tệp CSV

Hình 4.22 thể hiện kết quả hệ thống xuất báo cáo tồn kho linh kiện thành công dưới định dạng BaoCaoTonKho.csv. Sau khi người quản trị thực hiện chức năng Export trên giao diện Web, hệ thống tự động tổng hợp dữ liệu tồn kho hiện tại và tạo tệp báo cáo lưu về máy tính cục bộ.

Nội dung tệp CSV bao gồm các thông tin quan trọng như tên linh kiện, giá tiền, số lượng tồn kho, vị trí lưu trữ, mô tả và đường dẫn hình ảnh sản phẩm. Dữ liệu được lưu theo cấu trúc bảng chuẩn CSV nên có thể dễ dàng mở bằng Notepad, Excel hoặc các công cụ xử lý dữ liệu khác.

Kết quả này chứng minh hệ thống đã thực hiện thành công quá trình truy xuất dữ liệu từ Firebase và chuyển đổi sang định dạng báo cáo phục vụ quản lý kho. Chức năng Export giúp hỗ trợ kiểm kê, lưu trữ dữ liệu và thuận tiện cho việc thống kê hoặc phân tích tồn kho trong thực tế.

4.2. Kết quả triển khai tương tác giữa frontend và firebase



Hình 4.23. Kết quả tương tác giữa Frontend và Firebase Realtime Database

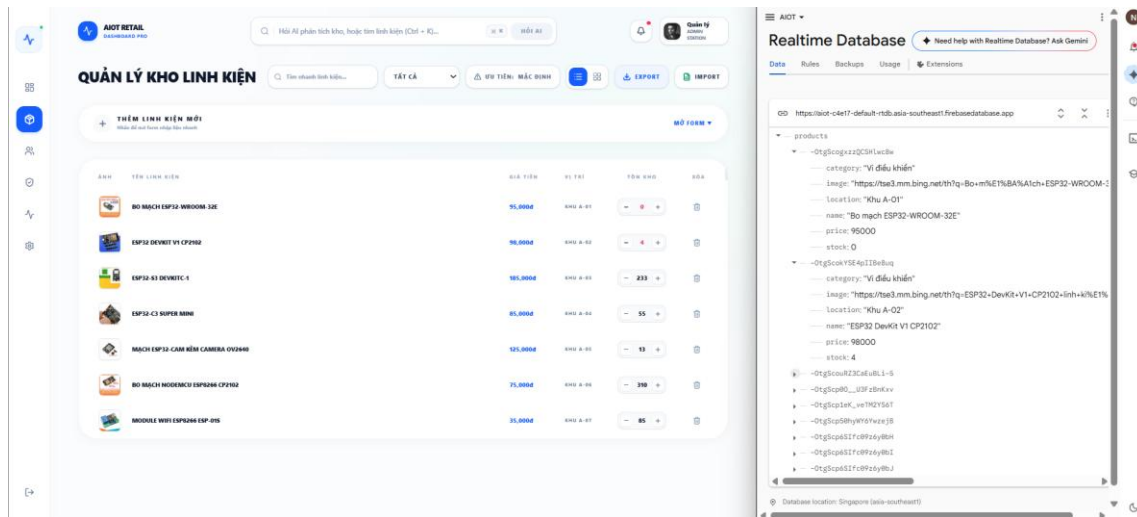
Hình trên cho thấy giao diện Live Tracking Khách Hàng ở phía Frontend đang đồng bộ trực tiếp với dữ liệu trong Firebase Realtime Database. Ở Firebase, dữ liệu đơn hàng được lưu trong nhánh carts, gồm các mã đơn như 010626-No_140 và 290526-No_138. Tương ứng với dữ liệu này, giao diện Web hiển thị hai đơn hàng đang chờ xử lý.

Khi Firebase có dữ liệu mới trong nhánh carts, Frontend ReactJS sử dụng cơ chế lắng nghe realtime để tự động đọc dữ liệu và cập nhật danh sách đơn hàng lên màn hình. Vì vậy, quản trị viên có thể theo dõi ngay khách hàng, danh sách linh kiện, số lượng, vị trí kho, tổng tiền và trạng thái thanh toán mà không cần tải lại trang.

Ngoài hiển thị dữ liệu, Frontend còn gửi lệnh ngược lại Firebase thông qua các nút xử lý như Hiện bảng LED, Báo SMS, Xác nhận thu tiền và Mở tủ Locker. Khi quản trị viên bấm các nút này, trạng thái đơn hàng hoặc dữ liệu điều khiển sẽ được cập nhật vào Firebase, sau đó các phân hệ khác như bảng LED, Smart Locker hoặc hệ thống thông báo có thể đọc dữ liệu để thực hiện hành động tương ứng.

Kết quả này chứng minh hệ thống đã đạt được khả năng tương tác hai chiều giữa Frontend Web và Firebase: Firebase đóng vai trò lưu trữ và đồng bộ dữ liệu thời gian thực,

còn Frontend đóng vai trò hiển thị, giám sát và gửi lệnh điều khiển cho quá trình xử lý đơn hàng.



Hình 4.24. Kết quả đồng bộ dữ liệu tồn kho giữa Frontend và Firebase

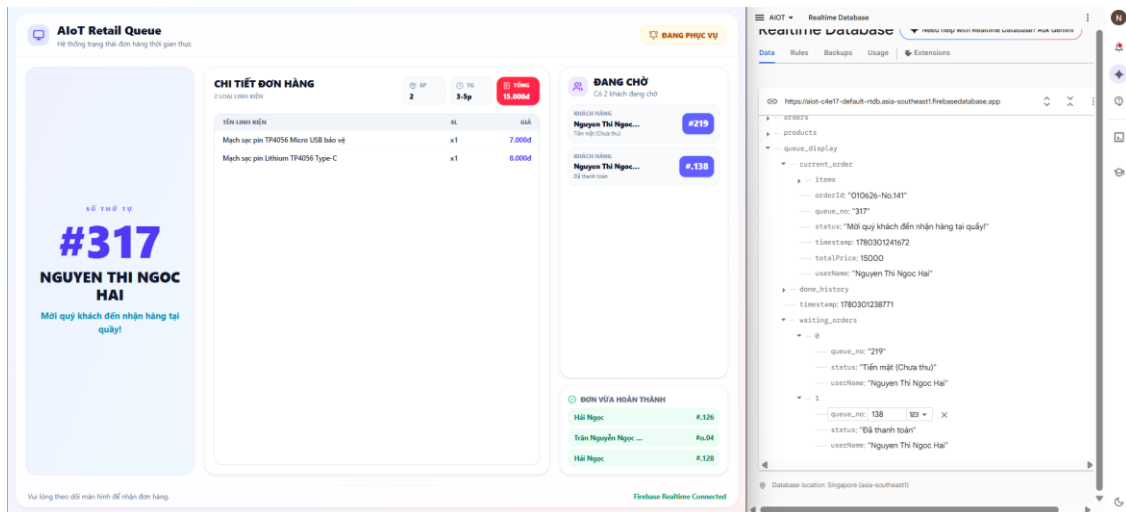
Hình trên thể hiện quá trình đồng bộ dữ liệu giữa giao diện quản lý kho linh kiện ở Frontend và Firebase Realtime Database ở Backend. Bên trái là giao diện Web ReactJS hiển thị danh sách linh kiện, còn bên phải là dữ liệu tương ứng được lưu trong nhánh products của Firebase.

Mỗi linh kiện trong Firebase được lưu dưới dạng một node riêng với các thuộc tính như name, price, stock, location, category và image. Frontend sử dụng cơ chế lắng nghe realtime (onValue) để đọc trực tiếp dữ liệu từ Firebase và hiển thị lên giao diện quản lý kho theo thời gian thực.

Khi quản trị viên thực hiện thao tác tăng hoặc giảm số lượng tồn kho bằng các nút + và -, dữ liệu stock trong Firebase sẽ được cập nhật ngay lập tức. Sau khi dữ liệu thay đổi, giao diện Web tự động render lại mà không cần tải lại trang. Ví dụ trong hình, linh kiện “ESP32 DevKit V1 CP2102” đang có tồn kho bằng 4, giá 98.000đ và vị trí KHU A-02, toàn bộ thông tin này được đồng bộ trực tiếp từ Firebase sang Frontend.

Ngoài chức năng hiển thị dữ liệu, hệ thống còn hỗ trợ tìm kiếm, lọc danh mục, import/export dữ liệu CSV và cập nhật tồn kho realtime. Điều này cho thấy Firebase không chỉ đóng vai trò lưu trữ dữ liệu mà còn là trung tâm đồng bộ giữa các phân hệ trong hệ thống AIoT Smart Retail.

Kết quả thực nghiệm chứng minh hệ thống đã xây dựng thành công cơ chế tương tác hai chiều giữa Frontend và Firebase, giúp việc quản lý kho linh kiện diễn ra nhanh chóng, trực quan và ổn định trong thời gian thực.



Hình 4.25. Kết quả đồng bộ dữ liệu Queue Display giữa Frontend và Firebase Realtime Database

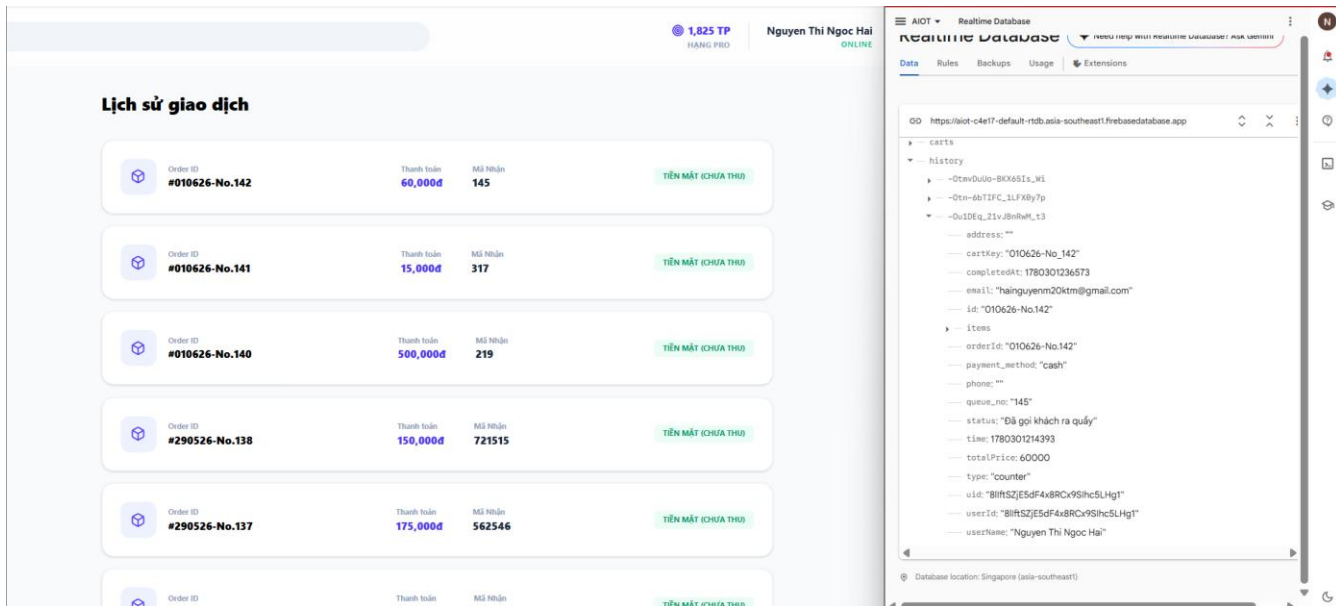
Hình trên thể hiện cơ chế đồng bộ realtime giữa giao diện bảng hiển thị hàng chờ (AIoT Retail Queue) và Firebase Realtime Database. Bên trái là giao diện Frontend hiển thị thông tin khách hàng đang được phục vụ, trong khi bên phải là dữ liệu tương ứng được lưu trong nhánh queue_display của Firebase.

Trong Firebase, hệ thống lưu dữ liệu thành các nhóm gồm current_order, waiting_orders và done_history. Nhánh current_order chứa thông tin đơn hàng đang được gọi hiện tại như mã đơn (orderId), số thứ tự (queue_no), tên khách hàng (userName), trạng thái (status) và tổng tiền (totalPrice). Các dữ liệu này được Frontend đọc realtime để hiển thị trực tiếp lên màn hình hàng chờ.

Ví dụ trong hình, Firebase đang lưu số thứ tự 317 cùng trạng thái “Mời quý khách đến nhận hàng tại quầy!”, và giao diện Queue phía Frontend cũng đồng thời hiển thị đúng thông tin này cho khách hàng. Điều này cho thấy dữ liệu đã được đồng bộ thành công giữa Backend và giao diện hiển thị.

Ngoài đơn hàng hiện tại, hệ thống còn đồng bộ danh sách khách hàng đang chờ (waiting_orders) và lịch sử đơn đã hoàn thành (done_history). Khi quản trị viên thực hiện

thao tác gọi khách trên giao diện quản trị, dữ liệu sẽ được cập nhật vào Firebase, sau đó bảng Queue tự động nhận dữ liệu mới và cập nhật giao diện mà không cần tải lại trang. Kết quả thực nghiệm chứng minh hệ thống đã triển khai thành công cơ chế giao tiếp realtime giữa Frontend và Firebase, giúp bảng hàng chờ hoạt động giống hệ thống lấy số tự động trong thực tế. Điều này hỗ trợ nâng cao khả năng quản lý đơn hàng, giảm thời gian chờ và tăng tính trực quan cho khách hàng tại quầy.



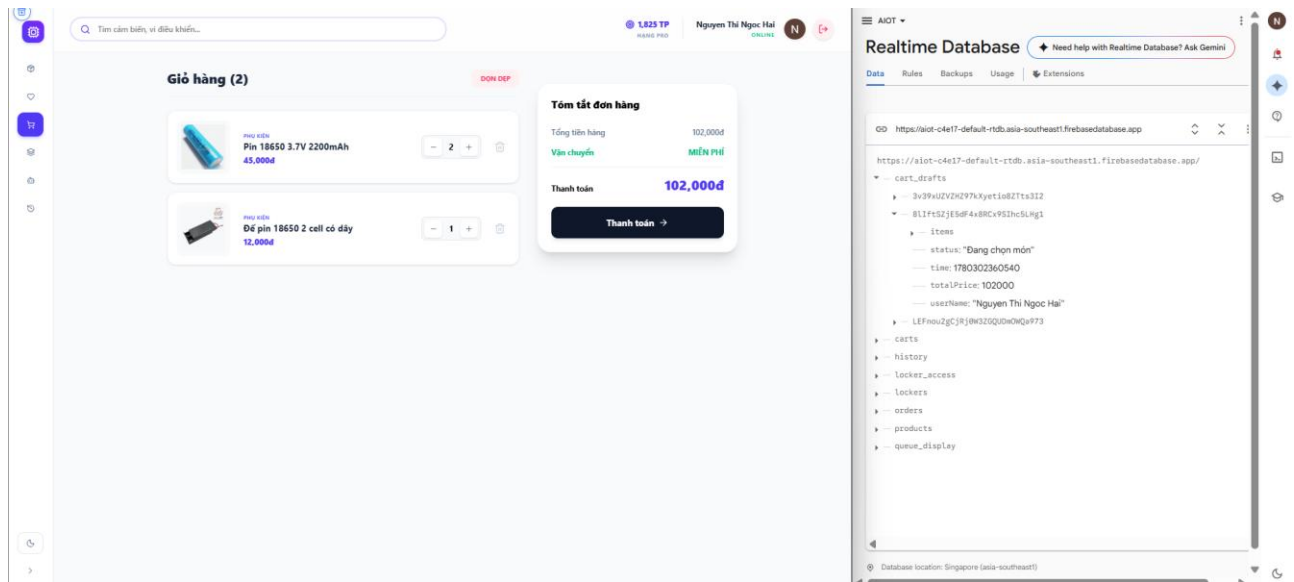
Hình 4.26. Kết quả đồng bộ lịch sử giao dịch giữa Frontend và Firebase

Hình trên thể hiện chức năng quản lý và lưu trữ lịch sử giao dịch của hệ thống AIoT Smart Retail. Bên trái là giao diện Frontend hiển thị danh sách các đơn hàng đã được xử lý, trong khi bên phải là dữ liệu tương ứng được lưu trong nhánh history của Firebase Realtime Database.

Sau khi một đơn hàng được hoàn tất hoặc khách hàng được gọi ra quầy nhận hàng, hệ thống sẽ tự động chuyển dữ liệu từ nhánh carts sang nhánh history. Trong Firebase, mỗi giao dịch được lưu với đầy đủ thông tin như orderId, queue_no, payment_method, status, totalPrice, completedAt, userName và danh sách sản phẩm (items). Frontend sử dụng cơ chế realtime để đọc dữ liệu này và hiển thị trực tiếp lên giao diện lịch sử giao dịch.

Ví dụ trong hình, đơn hàng 010626-No.142 có tổng thanh toán 60.000đ, mã nhận 145 và trạng thái “Đã gọi khách ra quầy”. Các thông tin này được hiển thị đồng thời trên giao diện Web và Firebase, chứng minh dữ liệu đã được đồng bộ thành công giữa hai phía.

Kết quả thực nghiệm cho thấy hệ thống đã xây dựng thành công cơ chế quản lý lịch sử giao dịch realtime giữa Frontend và Firebase, đảm bảo dữ liệu đơn hàng được lưu trữ đầy đủ, đồng bộ và ổn định trong quá trình vận hành hệ thống AIoT Smart Retail.

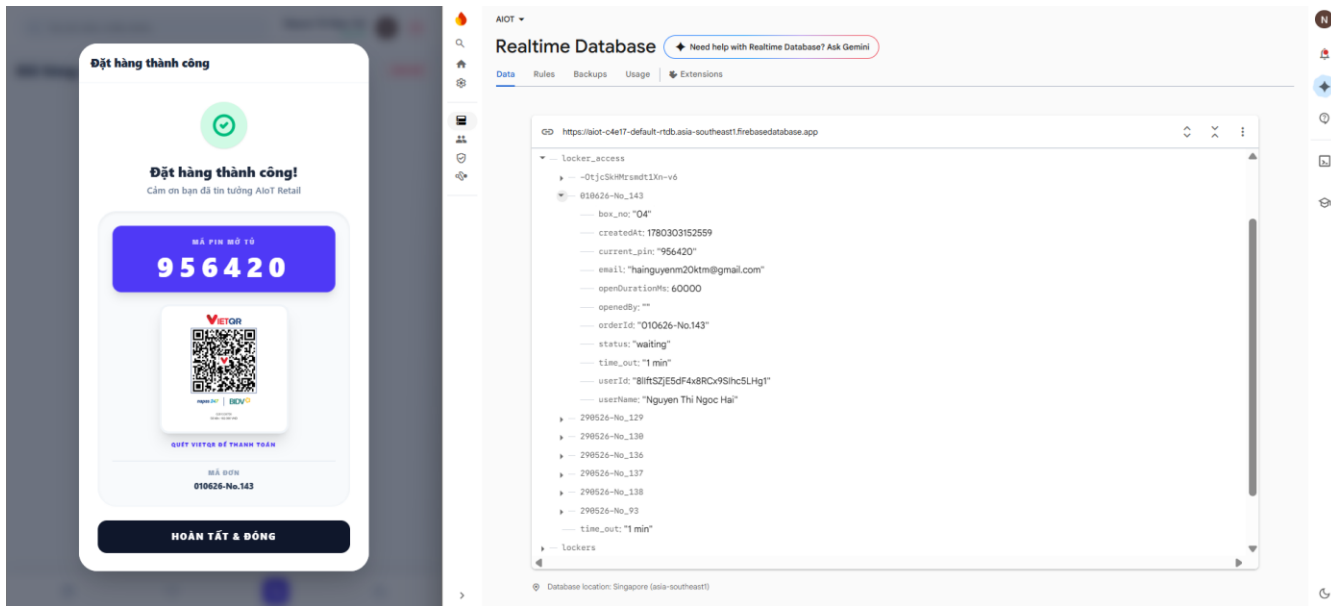


Hình trên thể hiện chức năng quản lý giỏ hàng nháp của hệ thống AIoT Smart Retail. Bên trái là giao diện giỏ hàng phía Frontend, còn bên phải là dữ liệu được lưu trong nhánh cart drafts của Firebase Realtime Database.

Trong ví dụ ở hình minh họa, người dùng đang chọn hai sản phẩm gồm pin 18650 và đế pin 2 cell, với tổng thanh toán 102.000đ. Dữ liệu này đồng thời được hiển thị trên giao diện Frontend và lưu trong Firebase với trạng thái “Đang chọn món”. Điều này cho thấy cơ chế đồng bộ realtime giữa giao diện người dùng và cơ sở dữ liệu đã hoạt động thành công.

Ngoài chức năng lưu tạm giỏ hàng, hệ thống còn hỗ trợ cập nhật số lượng sản phẩm trực tiếp trên giao diện thông qua các nút tăng/giảm số lượng. Sau khi dữ liệu thay đổi, Firebase sẽ tự động cập nhật và Frontend render lại thông tin giỏ hàng mà không cần tải lại trang.

Kết quả thực nghiệm chứng minh hệ thống đã xây dựng thành công cơ chế lưu trữ đơn hàng nháp realtime trên Firebase, giúp người dùng có thể tiếp tục quá trình mua sắm mà không bị mất dữ liệu khi chuyển trang hoặc thao tác nhiều lần trên hệ thống.



Hình 4.28. Kết quả đồng bộ hệ thống Smart Locker giữa Frontend và Firebase

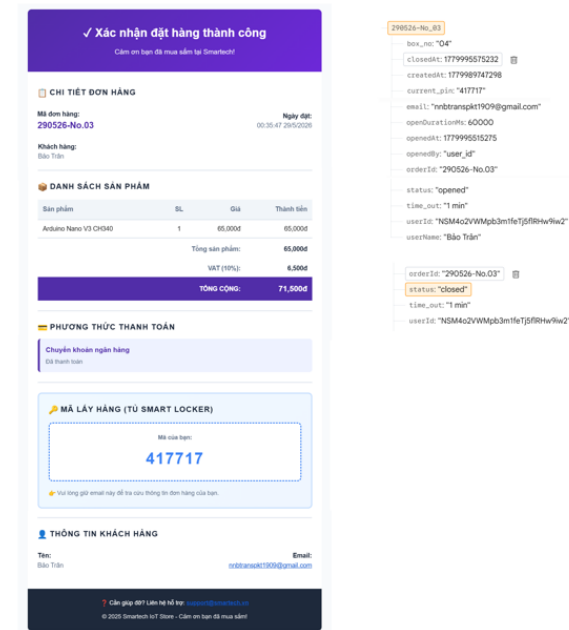
Hình trên thể hiện chức năng Smart Locker của hệ thống AIoT Smart Retail sau khi khách hàng đặt hàng thành công. Bên trái là giao diện Frontend hiển thị mã PIN mở tủ và mã QR thanh toán, trong khi bên phải là dữ liệu tương ứng được lưu trong nhánh locker_access của Firebase Realtime Database.

Sau khi người dùng hoàn tất đơn hàng, hệ thống tự động tạo một phiên truy cập Smart Locker và lưu dữ liệu vào Firebase. Các thông tin được lưu bao gồm box_no (số tủ), current_pin (mã PIN mở tủ), orderId, userName, email, status, thời gian tạo (createdAt) và thời gian hết hạn (time_out). Frontend đồng thời đọc dữ liệu này để hiển thị trực tiếp cho khách hàng trên giao diện xác nhận đơn hàng.

Trong ví dụ minh họa, hệ thống đã tạo thành công mã PIN 956420 cho đơn hàng 010626-No.143 tại tủ số 04. Dữ liệu này được hiển thị đồng thời trên giao diện Web và Firebase, cho thấy cơ chế đồng bộ realtime đã hoạt động chính xác.

Ngoài việc hiển thị mã PIN, hệ thống còn sinh mã QR VietQR để hỗ trợ thanh toán trực tuyến. Sau khi khách hàng thanh toán hoặc đến nhận hàng, trạng thái trong Firebase có thể được cập nhật để phục vụ cho việc điều khiển Smart Locker và quản lý truy cập tủ hàng.

Kết quả thực nghiệm chứng minh hệ thống đã triển khai thành công cơ chế kết nối giữa Frontend và Firebase trong phân hệ Smart Locker. Firebase đóng vai trò trung tâm lưu trữ và đồng bộ dữ liệu truy cập tủ, trong khi Frontend đảm nhiệm hiển thị thông tin và hỗ trợ tương tác trực tiếp với người dùng theo thời gian thực.



Hình 4.29. Kết quả xác nhận đơn hàng và cấp mã lấy hàng Smart Locker

Hình trên thể hiện giao diện xác nhận đơn hàng thành công của hệ thống AIoT Smart Retail sau khi khách hàng hoàn tất quá trình thanh toán. Bên trái là giao diện hóa đơn điện tử hiển thị thông tin đơn hàng, còn bên phải là dữ liệu được lưu trong Firebase Realtime Database tại nhánh locker_access.

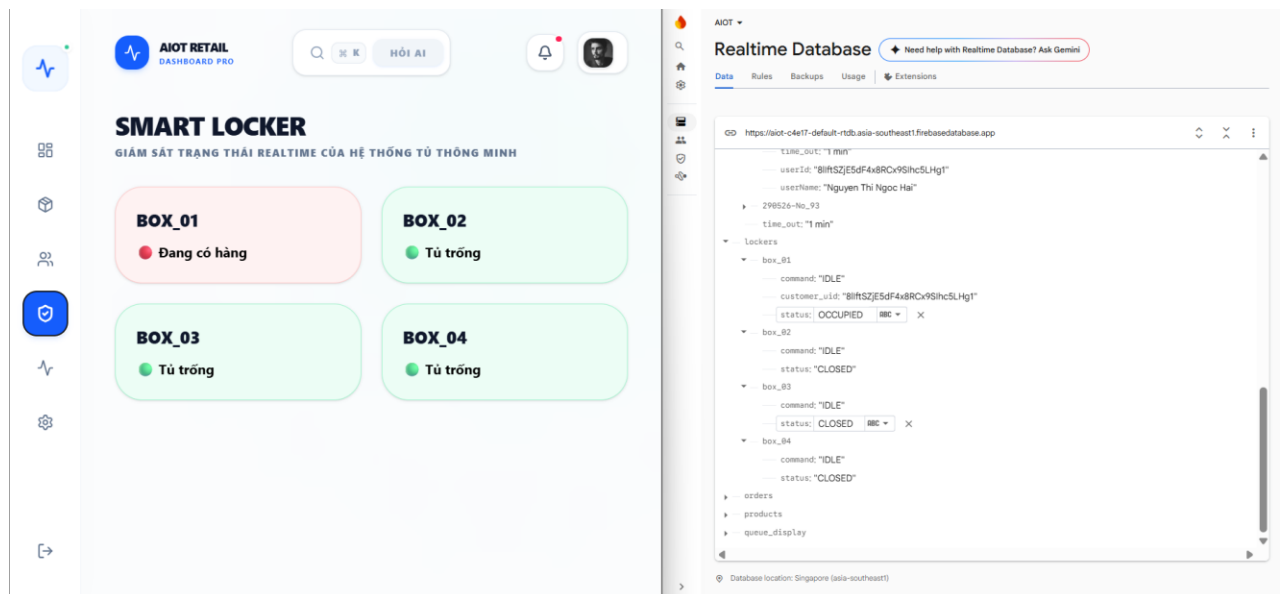
Sau khi đơn hàng được xác nhận, hệ thống tự động tạo mã lấy hàng dành cho Smart Locker và lưu vào Firebase. Các thông tin bao gồm orderId, current_pin, status, openedAt, closedAt, userName, email và thời gian mở tủ (openDurationMs). Frontend đồng thời đọc dữ liệu này để hiển thị trực tiếp cho khách hàng dưới dạng mã PIN lấy hàng.

Trong ví dụ minh họa, đơn hàng 290526-No.03 đã được thanh toán thành công với tổng giá trị 71.500đ, đồng thời hệ thống tạo mã lấy hàng 417717 cho khách hàng “Bảo Trân”. Trạng

thái trong Firebase được cập nhật là opened, cho thấy Smart Locker đã sẵn sàng phục vụ khách hàng nhận hàng.

Ngoài chức năng cấp mã PIN, giao diện còn hiển thị đầy đủ thông tin hóa đơn gồm danh sách sản phẩm, phương thức thanh toán, thông tin khách hàng và tổng tiền thanh toán. Điều này giúp người dùng dễ dàng kiểm tra lại đơn hàng trước khi đến nhận hàng tại Smart Locker.

Kết quả thực nghiệm cho thấy hệ thống đã xây dựng thành công cơ chế đồng bộ giữa giao diện xác nhận đơn hàng và Firebase Realtime Database. Firebase đóng vai trò lưu trữ trạng thái truy cập Smart Locker theo thời gian thực, trong khi Frontend hiển thị thông tin và hỗ trợ khách hàng thực hiện quá trình nhận hàng một cách trực quan và thuận tiện.



Hình 4.30. Kết quả giám sát trạng thái Smart Locker realtime trên Frontend và Firebase

Hình trên thể hiện chức năng giám sát trạng thái Smart Locker theo thời gian thực của hệ thống AIoT Smart Retail. Bên trái là giao diện Frontend hiển thị trạng thái các ngăn tủ thông minh, trong khi bên phải là dữ liệu tương ứng được lưu trong Firebase Realtime Database tại nhánh lockers.

Mỗi ngăn tủ trong Firebase được quản lý dưới dạng một node riêng như box_01, box_02, box_03 và box_04. Các thuộc tính lưu trữ gồm status, command và customer_uid, cho phép hệ thống theo dõi trạng thái hoạt động của từng tủ theo thời gian thực.

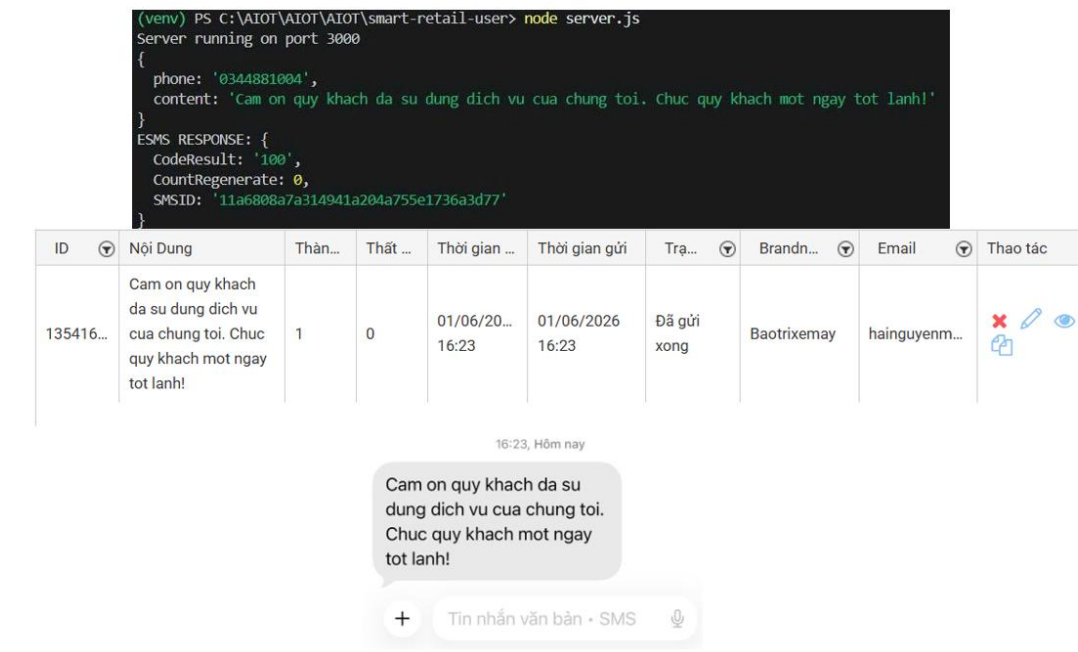
Trong ví dụ minh họa, BOX_01 đang ở trạng thái OCCUPIED, tương ứng với giao diện Frontend hiển thị “Đang có hàng” cùng màu đỏ cảnh báo. Các tủ còn lại ở trạng thái

CLOSED, đồng thời giao diện hiển thị “Tủ trống” với màu xanh lá. Điều này cho thấy dữ liệu trạng thái giữa Firebase và Frontend đã được đồng bộ chính xác.

Khi ESP32 hoặc hệ thống Website cập nhật trạng thái tủ trên Firebase, giao diện Smart Locker sẽ tự động thay đổi mà không cần tải lại trang. Cơ chế này giúp quản trị viên có thể theo dõi nhanh tình trạng các ngăn tủ, biết được tủ nào đang chứa hàng và tủ nào còn trống để phục vụ khách hàng.

Kết quả thực nghiệm chứng minh hệ thống đã triển khai thành công cơ chế giám sát Smart Locker realtime sử dụng Firebase Realtime Database. Firebase đóng vai trò trung tâm đồng bộ dữ liệu giữa Website, ESP32 và Frontend quản trị, giúp hệ thống hoạt động trực quan và ổn định trong thời gian thực.

4.3. Kết quả gửi thông báo về Email và SMS



The image displays a terminal window and a table of SMS messages. The terminal shows the execution of a Node.js server script, which sends an SMS and returns a response. The table below lists the details of the sent SMS.

```
(venv) PS C:\AIOT\AIOT\AIOT\smart-retail-user> node server.js
Server running on port 3000
{
  phone: '0344881004',
  content: 'Cam on quy khách đã sử dụng dịch vụ của chúng tôi. Chúc quý khách một ngày tốt lành!'
}
ESMS RESPONSE: {
  CodeResult: '100',
  CountRegenerate: 0,
  SMSID: '11a6808a7a314941a204a755e1736a3d77'
}
```

ID	Nội Dung	Thành...	Thất ...	Thời gian ...	Thời gian gửi	Trạ...	Brandn...	Email	Thao tác
135416...	Cam on quy khách đã sử dụng dịch vụ của chúng tôi. Chúc quý khách một ngày tốt lành!	1	0	01/06/20...	01/06/2026	Đã gửi xong	Baotrixemay	hainguyenm...	

16:23, Hôm nay

Cam on quy khách đã sử dụng dịch vụ của chúng tôi. Chúc quý khách một ngày tốt lành!

+ Tin nhắn văn bản • SMS

Hình 4.31. Kết quả triển khai hệ thống gửi SMS thông báo khách hàng

Hình trên thể hiện kết quả triển khai và kiểm thử chức năng gửi SMS trong hệ thống AIoT Smart Retail. Hệ thống sử dụng Backend NodeJS kết hợp API ESMS để gửi tin nhắn thông báo trực tiếp đến khách hàng sau khi đơn hàng được xử lý thành công.

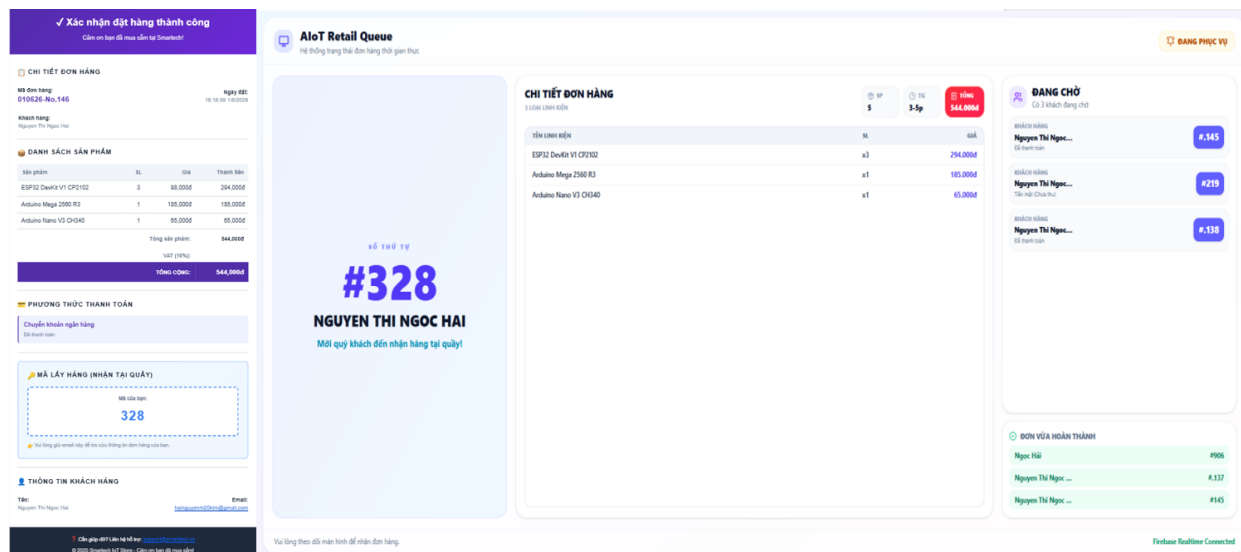
Ở hình thứ nhất, giao diện quản lý SMS cho thấy trạng thái gửi tin nhắn đã thành công với số lượng gửi thành công bằng 1, không có lỗi thất bại (0) và trạng thái “Đã gửi xong”.

Nội dung tin nhắn được lưu lại trên hệ thống quản lý SMS nhằm hỗ trợ theo dõi và kiểm tra lịch sử gửi thông báo.

Hình thứ hai thể hiện tin nhắn thực tế đã được gửi đến điện thoại khách hàng. Nội dung SMS hiển thị đầy đủ thông báo cảm ơn khách hàng sau khi sử dụng dịch vụ của hệ thống Smart Retail. Điều này chứng minh chức năng gửi SMS đã hoạt động thành công trên thiết bị thực tế.

Ở hình thứ ba, cửa sổ terminal Backend NodeJS hiển thị quá trình xử lý yêu cầu gửi SMS. Sau khi Frontend gửi request tới API /send-sms, server thực hiện gọi API ESMS và nhận phản hồi thành công với CodeResult: 100. Đồng thời hệ thống cũng sinh ra SMSID dùng để xác thực và quản lý tin nhắn trên hệ thống ESMS.

Kết quả thực nghiệm cho thấy hệ thống đã triển khai thành công cơ chế giao tiếp giữa Frontend ReactJS, Backend NodeJS và dịch vụ SMS ESMS. Chức năng này giúp tự động gửi thông báo cho khách hàng trong quá trình xử lý đơn hàng, góp phần nâng cao trải nghiệm người dùng và khả năng tương tác realtime của hệ thống AIoT Smart Retail.



Hình 4.32. Kết quả gửi Email xác nhận đơn hàng và đồng bộ Queue Realtime

Hình trên thể hiện quá trình gửi Email xác nhận đơn hàng kết hợp đồng bộ dữ liệu hàng chờ realtime trong hệ thống AIoT Smart Retail. Bên trái là giao diện Email/Hóa đơn điện tử được gửi đến khách hàng sau khi đặt hàng thành công, trong khi bên phải là giao diện Queue Display dùng để hiển thị trạng thái phục vụ tại quầy.

Sau khi khách hàng hoàn tất thanh toán, hệ thống tự động tạo hóa đơn điện tử và gửi Email xác nhận chứa đầy đủ thông tin đơn hàng gồm mã đơn (010626-No.146), danh sách sản phẩm, số lượng, giá tiền, tổng thanh toán và mã lấy hàng tại quầy. Email này giúp khách hàng dễ dàng kiểm tra lại thông tin trước khi đến nhận hàng.

Đồng thời, hệ thống cũng cập nhật dữ liệu vào Firebase Realtime Database để đồng bộ với màn hình Queue Display. Trong ví dụ minh họa, khách hàng “Nguyen Thi Ngoc Hai” được cấp số thứ tự #328 và trạng thái “Mời quý khách đến nhận hàng tại quầy!”. Queue Display sẽ tự động hiển thị thông tin này theo thời gian thực mà không cần tải lại giao diện.

Ngoài hiển thị đơn hàng hiện tại, hệ thống còn quản lý danh sách khách hàng đang chờ và lịch sử các đơn đã hoàn thành. Điều này giúp nhân viên dễ dàng theo dõi trạng thái xử lý đơn hàng cũng như tối ưu quy trình phục vụ tại quầy.

4.4. Kết quả triển khai và đánh giá AI Backend

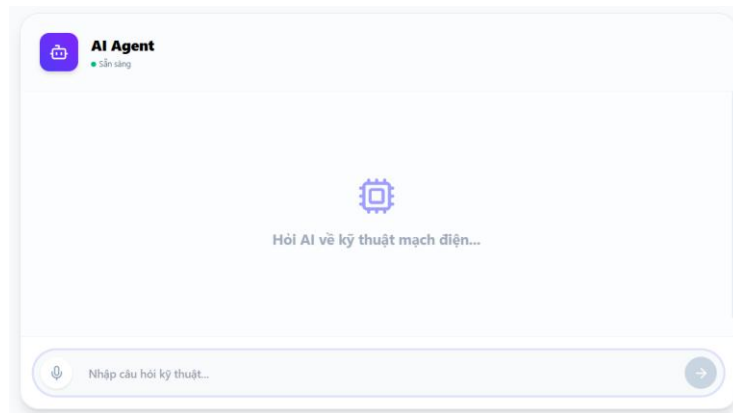
4.4.1. Kết quả kết nối Web Chatbot với AI Backend

Hệ thống Web Chatbot được xây dựng bằng ReactJS nhằm cho phép người dùng giao tiếp trực tiếp với AI Backend.

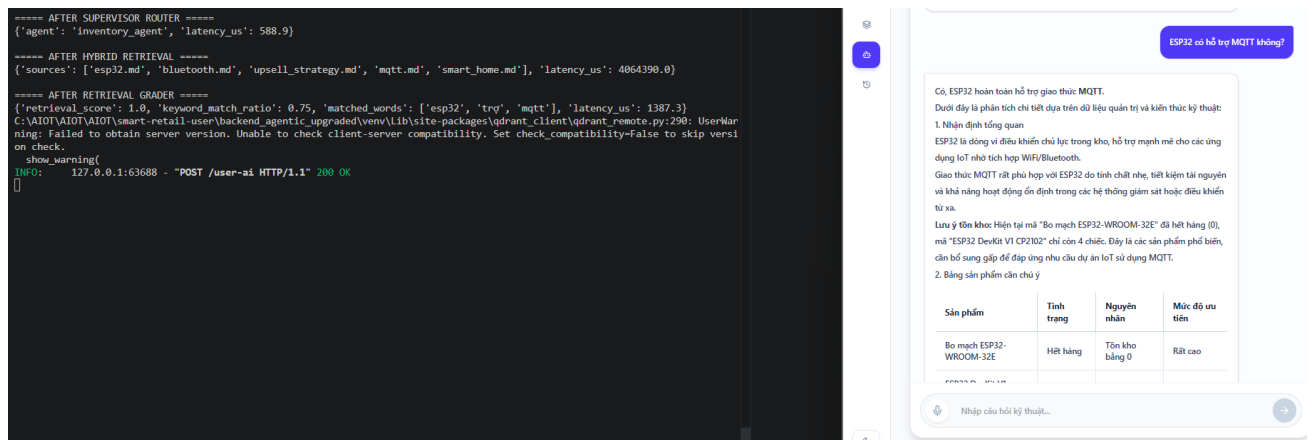
Người dùng có thể:

- hỏi thông tin kỹ thuật,
- kiểm tra tồn kho,
- gợi ý combo linh kiện,
- truy vấn dữ liệu Internet,
- hỏi thông tin Smart Locker.

Dữ liệu từ chatbot được gửi đến FastAPI Backend thông qua REST API.



Hình 4.33. Giao diện Web Chatbot kết nối AI Backend



Hình 4.34. Kết quả phản hồi của AI trên Web Chatbot

Kết quả triển khai cho thấy Web Chatbot có khả năng:

- gửi query đến AI Backend,
- nhận phản hồi AI realtime,
- hiển thị nội dung phản hồi trực tiếp trên giao diện web.

Các truy vấn kỹ thuật như:

- “ESP32 có hỗ trợ MQTT không?”
- “RFID RC522 dùng giao tiếp gì?”

được AI phản hồi chính xác dựa trên truy xuất dữ liệu nội bộ.

Ngoài ra chatbot còn có khả năng:

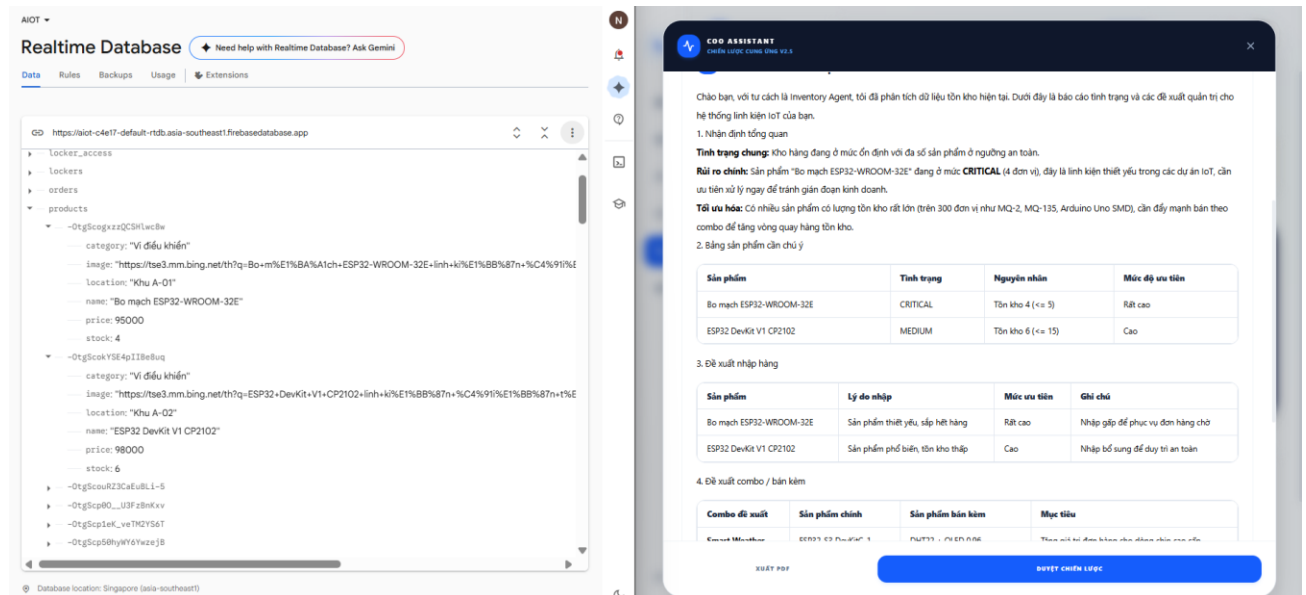
- hiển thị phản hồi tồn kho,
- phản hồi combo linh kiện,
- hỗ trợ Smart Locker.

Kết quả này cho thấy hệ thống chatbot đã có khả năng hoạt động như một AI assistant trong môi trường Smart Retail và AIoT.

4.4.2. Kết quả đọc dữ liệu từ Firebase Realtime Database cho phản hồi AI

Firebase Realtime Database được sử dụng nhằm:

- lưu dữ liệu tồn kho,
- lưu trạng thái Smart Locker,
- lưu trạng thái đơn hàng,
- đồng bộ dữ liệu realtime giữa frontend và backend.



Hình 4.35. Kết quả AI truy xuất dữ liệu tồn kho từ Firebase Realtime Database

Hình bên trái thể hiện cấu trúc dữ liệu tồn kho được lưu trữ trong Firebase Realtime Database. Trong node products, hệ thống lưu các thông tin:

- tên sản phẩm,

- giá bán,
- trạng thái tồn kho,
- vị trí lưu trữ,
- hình ảnh sản phẩm.

Khi người dùng gửi truy vấn từ Web Chatbot, AI Backend sẽ truy xuất dữ liệu realtime từ Firebase để phân tích tồn kho và sinh phản hồi phù hợp.

Hình bên phải thể hiện kết quả phản hồi của AI khi thực hiện truy vấn tồn kho. Hệ thống AI đã:

- phân tích dữ liệu tồn kho,
- đánh giá mức độ ưu tiên nhập hàng,
- phát hiện sản phẩm sắp hết hàng,
- đề xuất phương án xử lý.

Kết quả cho thấy hệ thống có khả năng:

- truy xuất dữ liệu realtime từ Firebase,
- kết hợp dữ liệu tồn kho với AI reasoning,
- sinh phản hồi phân tích thông minh thay vì chỉ trả về dữ liệu thô.

Ngoài ra AI còn có khả năng:

- đánh giá mức độ ưu tiên nhập hàng,
- phân tích sản phẩm bán chạy,
- gợi ý combo bán kèm,
- hỗ trợ quản lý kho trong môi trường Smart Retail.

Điều này cho thấy sự kết hợp giữa:

- Firebase Realtime Database,
- Qdrant Retrieval,
- LangGraph Workflow,
- Multi-Agent AI

đã giúp hệ thống hoạt động như một trợ lý quản lý kho thông minh trong môi trường AIoT Smart Retail.

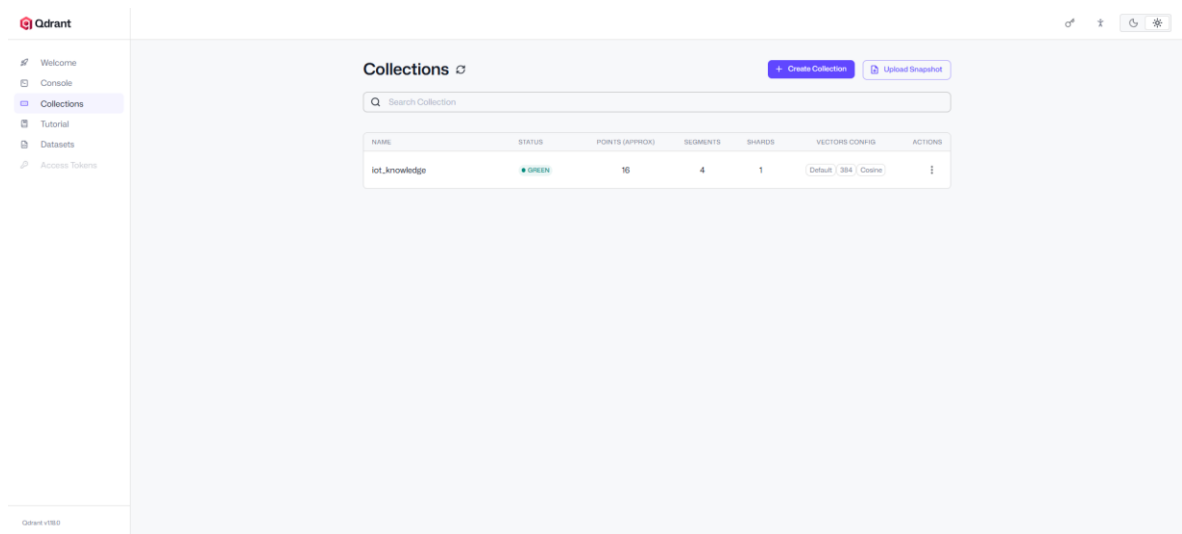
4.4.3. Kết quả triển khai Qdrant Vector Database

Qdrant được sử dụng làm Vector Database trong hệ thống AI Backend nhằm:

- lưu trữ vector embedding,
- thực hiện semantic retrieval,
- hỗ trợ Retrieval-Augmented Generation (RAG).

Dữ liệu tri thức kỹ thuật sau khi được xử lý sẽ được:

- chia chunk,
- embedding thành vector,
- lưu vào collection trong Qdrant.



Hình 4.36. Collection *iot_knowledge* trong Qdrant

Hình trên thể hiện collection *iot_knowledge* được tạo trong Qdrant Dashboard. Kết quả cho thấy hệ thống đã kết nối thành công với Qdrant và tạo collection phục vụ semantic retrieval.

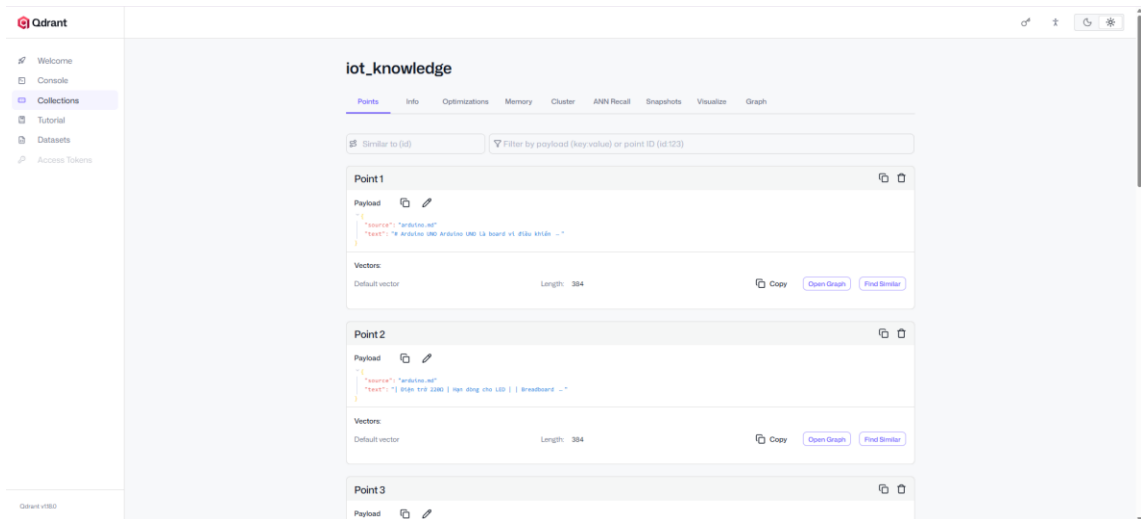
Ngoài ra collection hiện đang lưu:

- 16 vector embedding,
- 4 payload index,

- trạng thái hoạt động green.

Điều này chứng minh:

- dữ liệu embedding đã được lưu thành công,
- Qdrant hoạt động ổn định,
- Vector Database đã sẵn sàng phục vụ truy xuất cho AI Backend.



Hình 4.37. Dữ liệu vector embedding được lưu trong Qdrant

Hình trên thể hiện các point dữ liệu được lưu trong collection iot_knowledge.

Kết quả cho thấy:

- dữ liệu văn bản đã được chia thành nhiều chunk,
- mỗi chunk được embedding thành vector riêng,
- vector embedding đã được lưu thành công vào Qdrant.

Trong mỗi point, hệ thống lưu:

- source của tài liệu,
- nội dung text chunk,
- vector embedding tương ứng.

Ví dụ:

- Point 1 lưu dữ liệu từ file arduino.md,
- vector embedding có kích thước 384 chiều.

Điều này cho thấy hệ thống đã xây dựng thành công Vector Database phục vụ semantic truy xuất cho AI Backend.

```

===== AFTER SUPERVISOR ROUTER =====
{"agent": "general_agent", "latency_us": 62.1}

===== AFTER HYBRID RETRIEVAL =====
{"sources": ["weather_station.md", "dht22.md", "oled_ssd1306.md", "dht11.md", "smart_home.md"], "latency_us": 79053.4}

===== AFTER RETRIEVAL GRADER =====
{"retrieval_score": 0.444, "keyword_match_ratio": 0.5, "matched_words": ["thời", "tiết", "chi", "minh"], "latency_us": 385.9}

===== WEB FALLBACK ACTIVATED =====

===== WEB RESULTS =====

TITLE:
Thời tiết hàng giờ ở Thành phố Hồ Chí Minh, Hồ Chí ... - AccuWeather

CONTENT:
RealFeel Shade™93°. Chỉ số nhiệt90°. Gió1 5 mi/h. Chất lượng không khíCó hại. Chỉ số UV tối đa0.4 (Thấp). Gió giật12 mi/h. Độ ẩm86%. Điểm sương77° F.

TITLE:
Thời tiết TP.Hồ Chí Minh hôm nay, ngày mai và những ngày tới - Dự báo thời tiết

CONTENT:
# Dự báo thời tiết TP.Hồ Chí Minh hôm nay, ngày mai theo giờ. ## Nhiệt độ TP.Hồ Chí Minh. Nhiệt độ ban ngày tại TP.Hồ Chí Minh. Nhiệt độ ban đêm tại TP.Hồ Chí Minh. Nhiệt độ sáng tại TP.Hồ Chí Minh. Nhiệt độ đêm tại TP.Hồ Chí Minh. Thời tiết TP.Hồ Chí Minh. # Dự báo thời tiết TP.Hồ Chí Minh. ## Dự báo thời tiết TP.Hồ Chí Minh sắp tới. Thời tiết TP.Hồ Chí Minh hôm nay, ngày mai và những ngày tới. ## Chất lượng không khí tại TP.Hồ Chí Minh. # **Thời Tiết Hồ Chí Minh - Dự Báo Nắng Gát Và Mưa Đông**. Với đặc trưng khí hậu nhiệt đới gió mùa, **thời tiết thành phố Hồ Chí Minh** được phn chia rõ rệt thành hai mùa riêng biệt:. | 15 | Quận 12 | Phường Tân Chánh H

===== GENERATE NODE =====
AGENT = web_search_agent
API KEY: sk-or-v1-6e08bb148b5
[INFO: 127.0.0.1:57940 - "POST /user-ai HTTP/1.1" 200 OK

```

Hình 4.38. Kết quả truy xuất dữ liệu từ Qdrant

Hình trên thể hiện terminal log của AI Backend trong quá trình truy xuất dữ liệu.

Khi người dùng gửi truy vấn, hệ thống thực hiện:

- Supervisor Router chọn Agent phù hợp,
- Hybrid Truy xuất truy xuất dữ liệu liên quan,
- Truy xuất Grader đánh giá truy xuất score,
- AI generation sinh phản hồi cuối cùng.

Kết quả terminal log cho thấy hệ thống đã truy xuất thành công các source:

- weather_station.md
- dht22.md
- oled_ssd1306.md
- dht11.md
- smart_home.md

Ngoài ra hệ thống còn hiển thị:

- truy xuất score,
- keyword matching ratio,
- latency xử lý.

Kết quả cho thấy truy xuất score đạt 0.444, chưa đủ cao nên hệ thống tự động kích hoạt cơ chế Web Fallback để bổ sung thêm dữ liệu từ Internet trước khi sinh phản hồi cuối cùng.

Điều này cho thấy hệ thống không chỉ thực hiện semantic truy xuất từ Qdrant mà còn có khả năng:

- đánh giá chất lượng retrieval,
- tự động fallback sang web search,
- kết hợp dữ liệu nội bộ và dữ liệu Internet để tăng độ chính xác phản hồi AI.

Kết quả triển khai chứng minh:

- Qdrant đã tham gia trực tiếp vào workflow AI Backend,
- semantic truy xuất hoạt động thành công,
- hệ thống RAG hoạt động đúng theo thiết kế đề ra.

4.4.4. Kết quả đánh giá khả năng định tuyến câu hỏi giữa các Agent

Để đánh giá khả năng định tuyến của hệ thống AI Backend, bộ kiểm thử gồm 35 câu hỏi thuộc nhiều nhóm khác nhau đã được xây dựng:

- tư vấn kỹ thuật,
- truy vấn tồn kho,
- gợi ý combo linh kiện,
- tìm kiếm thông tin Internet.

Mỗi câu hỏi được gán trước Agent mong đợi nhằm so sánh với Agent thực tế do hệ thống lựa chọn.

Bảng 4.1: Các testcase dùng kiểm thử

id	Truy vấn	agent mong đợi	agent dự đoán
1	ESP32 có hỗ trợ MQTT không?	consultant_agent	consultant_agent
2	RC522 dùng để làm gì?	consultant_agent	consultant_agent

3	DHT11 dùng để đo gì?	consultant_agent	consultant_agent
4	LCD I2C kết nối với vi điều khiển như thế nào?	consultant_agent	consultant_agent
5	Gợi ý combo làm robot car	combo_agent	combo_agent
6	Gợi ý combo học IoT cơ bản	combo_agent	combo_agent
7	Sản phẩm nào sắp hết hàng?	inventory_agent	inventory_agent
8	Cần nhập thêm linh kiện nào?	inventory_agent	inventory_agent
9	Smart locker hoạt động như thế nào?	consultant_agent	combo_agent
10	iPhone 16 có gì mới?	web_search_agent	web_search_agent
11	Cảm biến siêu âm HC-SR04 đo được khoảng cách bao xa?	consultant_agent	inventory_agent
12	Sự khác biệt giữa Arduino Uno và ESP8266 là gì?	consultant_agent	consultant_agent
13	Mạch L298N dùng để điều khiển động cơ gì?	consultant_agent	general_agent
14	Làm sao để kết nối cảm biến mưa với Arduino?	consultant_agent	consultant_agent
15	Nguyên lý hoạt động của cảm biến khí gas MQ2	consultant_agent	consultant_agent
16	Tư vấn bộ combo làm nhà thông minh smarthome giá rẻ	combo_agent	combo_agent
17	Có combo nào cho người mới bắt đầu học Arduino không shop?	combo_agent	combo_agent
18	Set linh kiện lắp ráp cánh tay robot 4 bậc	combo_agent	general_agent
19	Gợi ý trọn bộ làm máy rót rượu tự động	combo_agent	general_agent
20	Kiểm tra xem module sim A7670C còn bao nhiêu cái trong kho?	inventory_agent	web_search_agent
21	Danh sách các mặt hàng bị âm kho hoặc bằng 0	inventory_agent	inventory_agent
22	Báo cáo số lượng linh kiện điện tử hiện tại	inventory_agent	inventory_agent

23	Mặt hàng nào đang bán chạy nhất tháng này để nhập thêm?	inventory_agent	inventory_agent
24	Giá vàng hôm nay tăng hay giảm?	web_search_agent	web_search_agent
25	Tin tức mới nhất về cuộc bầu cử tổng thống Mỹ	web_search_agent	web_search_agent
26	Thời tiết Hà Nội cuối tuần này có mưa không?	web_search_agent	general_agent
27	Kết quả trận bóng đá tối qua giữa Real Madrid và Barca	web_search_agent	web_search_agent
28	Cảm biến dht22 sai số bao nhiêu?	consultant_agent	consultant_agent
29	Shop có bán nguyên bộ linh kiện làm mạch đo nhịp tim ko	combo_agent	combo_agent
30	Ktra hàng tồn của servo sg90	inventory_agent	consultant_agent
31	Cách lập trình esp32 deep sleep	consultant_agent	consultant_agent
32	Giá cổ phiếu Vinfast hôm nay thế nào	web_search_agent	web_search_agent
33	Combo xe tự hành dò line	combo_agent	combo_agent
34	Những linh kiện nào đã lưu kho quá 6 tháng chưa bán được?	inventory_agent	inventory_agent
35	Phim rạp hot nhất hiện nay là phim gì	web_search_agent	web_search_agent

Bảng 4.2. Kết quả đánh giá định tuyến Agent

Hạng mục	Giá trị
Tổng số câu kiểm thử	35
Số câu route đúng	27
Số câu route sai	8
Routing Accuracy	77.14%

Accuracy được tính theo công thức:

$$\text{Accuracy} = (\text{Số câu route đúng} / \text{Tổng số câu kiểm thử}) \times 100\%$$

Kết quả:

$$\text{Accuracy} = (27 / 35) \times 100\% = 77.14\%$$

```
AI BACKEND EVALUATION REPORT

Total queries: 35

Agent Accuracy:
27/35 = 77.14%

Retrieval Keyword Accuracy:
34/35 = 97.14%

Average Retrieval Score:
0.817

Average Latency:
4294.72 ms
```

Hình 4.39. Kết quả đánh giá báo cáo đánh giá từ terminal



Hình 4.40. Biểu đồ tỷ lệ định tuyến đúng và sai của hệ thống AI

Kết quả biểu đồ cho thấy phần lớn câu hỏi đã được route đúng đến Agent tương ứng. Tuy nhiên vẫn còn khoảng 22.86% query bị định tuyến sai.

Các lỗi route sai chủ yếu xuất hiện ở:

- query mô tả dài,
- query chứa nhiều semantic context,
- query kỹ thuật nhưng chứa keyword linh kiện,
- query viết tắt.

Điều này cho thấy hệ thống routing hiện tại vẫn còn phụ thuộc khá nhiều vào:

- keyword matching,
- rule-based routing,
- pattern detection.

Bảng 4.3. Tính chính xác theo từng Agent

Agent	Số câu test	Accuracy
consultant_agent	12	75.00%
inventory_agent	8	75.00%
combo_agent	8	75.00%
web_search_agent	7	85.71%



Hình 4.41. Biểu đồ độ chính xác định tuyến theo từng Agent

Kết quả cho thấy:

- web_search_agent đạt độ chính xác cao nhất với khoảng 85.71%,
- consultant_agent, inventory_agent và combo_agent đều đạt khoảng 75%.

Nguyên nhân là do các query web thường chứa keyword rõ ràng như:

- “thời tiết”,
- “giá vàng”,
- “tin tức”

Nên hệ thống dễ nhận diện intent hơn.

Trong khi đó các query liên quan:

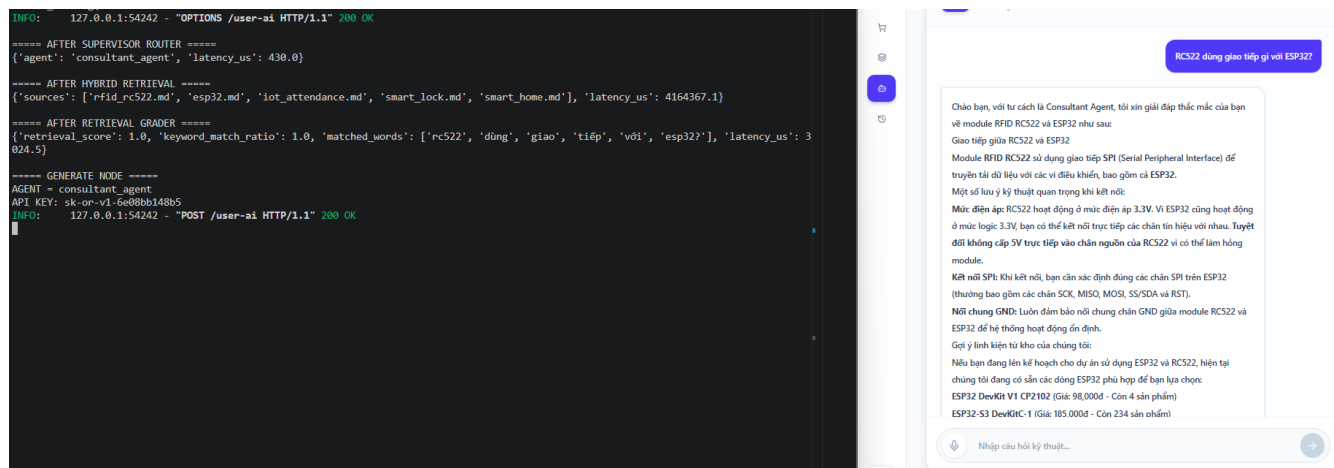
- combo linh kiện,
- tư vấn kỹ thuật,
- truy vấn đa semantic context

Thường phức tạp hơn nên dễ xảy ra route sai.

MỘT SỐ KẾT QUẢ ĐỊNH TUYẾN ĐÚNG CỦA TỪNG AGENT

Kết quả định tuyến consultant agent

Trong quá trình kiểm thử, các truy vấn khác nhau sẽ được Router Node phân loại về các agent khác nhau. Ví dụ, truy vấn liên quan đến linh kiện IoT sẽ được định tuyến sang consultant_agent, truy vấn liên quan tồn kho hoặc dữ liệu quản trị sẽ được định tuyến sang inventory_agent, còn truy vấn đề xuất bộ linh kiện sẽ được định tuyến sang combo_agent.



Hình 4.42. Kết quả định tuyến consultant agent

Hình 4.42 thể hiện kết quả kiểm thử AI Backend với truy vấn: “RC522 dùng giao tiếp gì với ESP32?”. Có thể quan sát toàn bộ workflow AI đã hoạt động đúng theo thiết kế thông qua các log xử lý tại terminal backend và phản hồi trên giao diện Web.

Đầu tiên, hệ thống thực hiện bước định tuyến truy vấn thông qua Supervisor Router. Kết quả log cho thấy truy vấn được định tuyến tới consultant_agent với thời gian xử lý

latency_us = 430.0 μ s. Điều này cho thấy Router Node đã nhận diện đúng đây là truy vấn kỹ thuật liên quan đến linh kiện IoT và chuyển truy vấn sang agent tư vấn kỹ thuật.

Tiếp theo, hệ thống kích hoạt Hybrid Truy xuất để truy xuất tri thức từ knowledge base nội bộ. Các tài liệu được trả về gồm rfid_rc522.md, esp32.md, iot_attendance.md, smart_lock.md và smart_home.md. Điều này chứng minh hệ thống có khả năng truy xuất đồng thời nhiều nguồn tri thức liên quan thay vì chỉ sử dụng một tài liệu đơn lẻ.

Sau bước retrieval, Truy xuất Grader tiếp tục đánh giá mức độ phù hợp của context với retrieval_score = 1.0 và keyword_match_ratio = 1.0. Các từ khóa được nhận diện gồm rc522, giao tiếp và esp32. Kết quả này cho thấy dữ liệu truy xuất có độ liên quan rất cao với truy vấn người dùng trước khi được đưa vào mô hình ngôn ngữ lớn.

Tại bước Generate Node, hệ thống sử dụng consultant_agent để xây dựng prompt cuối cùng và gửi tới OpenRouter LLM nhằm sinh phản hồi tự nhiên. Backend sau đó trả về trạng thái POST /user-ai HTTP/1.1 200 OK, chứng minh toàn bộ pipeline AI đã xử lý thành công.

Quan sát trên giao diện Web cho thấy AI không chỉ trả lời đúng giao tiếp SPI của RC522 mà còn giải thích thêm về mức điện áp hoạt động, các chân kết nối SPI và gợi ý linh kiện ESP32 phù hợp. Điều này cho thấy hệ thống không đơn thuần truy xuất dữ liệu mà còn có khả năng tổng hợp và mở rộng phản hồi dựa trên context retrieval.

Kết quả định tuyến inventory agent

The image displays the output of an inventory agent query. On the left, a terminal window shows the following log:

```
INFO: 127.0.0.1:54242 - "POST /user-ai HTTP/1.1" 200 OK
===== AFTER SUPERVISOR ROUTER =====
{'agent': 'inventory_agent', 'latency_us': 30.1}
===== AFTER HYBRID RETRIEVAL =====
{'sources': ['inventory_policy.md', 'inventory_strategy.md', 'smart_home.md', 'stock_analysis.md', 'robot_car.md'], 'latency_us': 4883754.9}
===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 0.8, 'keyword_match_ratio': 0.5, 'matched_words': ['linh', 'kiện', 'hết'], 'latency_us': 3092.1}
===== GENERATE NODE =====
AGENT = inventory_agent
API KEY: sk-or-v1-be08bb148b5
INFO: 127.0.0.1:58767 - "POST /user-ai HTTP/1.1" 200 OK
```

On the right, a web interface titled "AI Agent" displays the results. It includes a button "Linh kiện nào sắp hết hàng?" and a section "Dựa trên dữ liệu Admin hiện tại và chính sách phân tích tồn kho, dưới đây là báo cáo tình trạng các linh kiện cần chú ý:".

1. Nhận định tổng quan
Hệ thống đang có tình trạng thiếu hụt nghiêm trọng ở các dòng vi điều khiển phổ biến (ESP32).
Sản phẩm Bo mạch ESP32-WROOM-32E đã hết hàng hoàn toàn (CRITICAL).
Sản phẩm ESP32 DevKit V1 CP2102 và Raspberry Pi Pico RP2040 đang ở ngưỡng báo động đỏ, cần bổ sung ngay để đảm bảo nhu cầu dự án IoT.

2. Bảng sản phẩm cần chú ý

Sản phẩm	Tình trạng	Nguyên nhân	Mức độ ưu tiên
Bo mạch ESP32-WROOM-32E	Hết hàng	Stock = 0	Cao nhất
ESP32 DevKit V1 CP2102	CRITICAL	Stock = 4	Cao
Raspberry Pi Pico RP2040	CRITICAL	Stock = 7	Cao

At the bottom, there is a search bar with the placeholder text "Nhập câu hỏi kỹ thuật..." and a search icon.

Hình 4.43. Kết quả định tuyến inventory agent

Hình 4.43 thể hiện kết quả kiểm thử với truy vấn: “Linh kiện nào sắp hết hàng?”. Kết quả log cho thấy Router Node đã định tuyến truy vấn tới inventory_agent với latency_us = 30.1 μ s, chứng minh hệ thống đã nhận diện đúng đây là truy vấn liên quan đến quản trị tồn kho.

Tiếp theo, Hybrid Truy xuất truy xuất được các tài liệu liên quan như inventory_policy.md, inventory_strategy.md và stock_analysis.md. Truy xuất Grader trả về retrieval_score = 0.8 và keyword_match_ratio = 0.5, cho thấy context truy xuất có độ liên quan phù hợp với câu hỏi người dùng.

Trên giao diện Web, AI trả về danh sách các linh kiện có mức tồn kho thấp như ESP32-WROOM-32E, ESP32 DevKit V1 CP2102 và Raspberry Pi Pico RP2040, đồng thời phân loại mức độ ưu tiên nhập hàng. Kết quả này cho thấy inventory_agent không chỉ truy xuất dữ liệu tồn kho mà còn có khả năng phân tích và đưa ra cảnh báo quản trị kho.

Kết quả kiểm thử chứng minh hệ thống có khả năng định tuyến truy vấn theo từng miền chức năng thay vì sử dụng một prompt cố định cho toàn bộ câu hỏi.

kết quả định tuyến combo agent

Log terminal:

```
INFO: 127.0.0.1:58767 - "POST /user-a1 HTTP/1.1" 200 OK
===== AFTER SUPERVISOR ROUTER =====
{"agent": "combo_agent", "latency_us": 58.2}
===== AFTER HYBRID RETRIEVAL =====
{"sources": ["smart_home.md", "combo_strategy.md", "rfid_rc522.md", "gas_mq2.md", "robot_car.md"], "latency_us": 4087122.4}
===== AFTER RETRIEVAL GRADER =====
{"retrieval_score": 0.714, "keyword_match_ratio": 0.625, "matched_words": ["combo", "lâm", "nhà", "thông", "minh"], "latency_us": 2473.2}
===== GENERATE NODE =====
AGENT = combo_agent
API KEY: sk-or-v1-6e8bb148b5
INFO: 127.0.0.1:55627 - "POST /user-a1 HTTP/1.1" 200 OK
```

Giao diện web:

Combo làm nhà thông minh gồm những gì?

Chào bạn, với nhu cầu xây dựng hệ thống Smart Home (Nhà thông minh), tôi xin gợi ý combo linh kiện cơ bản giúp bạn bắt đầu dự án một cách hiệu quả nhất dựa trên dữ liệu kho hiện có.

1. Combo đề xuất: Smart Home Cơ bản

Combo này tập trung vào khả năng điều khiển thiết bị qua WiFi và giám sát môi trường sống.

2. Bảng linh kiện

Linh kiện	Vai trò	Lý do đề xuất
ESP32 DevKit V1 CP2102	Bộ điều khiển trung tâm	Tích hợp sẵn WiFi/Bluetooth, nhiều GPIO, phù hợp để kết nối Blynk/MQTT.
Relay module (1 kênh/nhiều kênh)	Điều khiển bật/tắt thiết bị	Dùng để đóng ngắt các thiết bị điện gia dụng (đèn, quạt).
Cảm biến DHT11	Đo nhiệt độ ẩm	Giúp theo dõi môi trường phòng khách hoặc phòng ngủ.

Nhập câu hỏi kỹ thuật...

Hình 4.44. Kết quả định tuyến combo agent

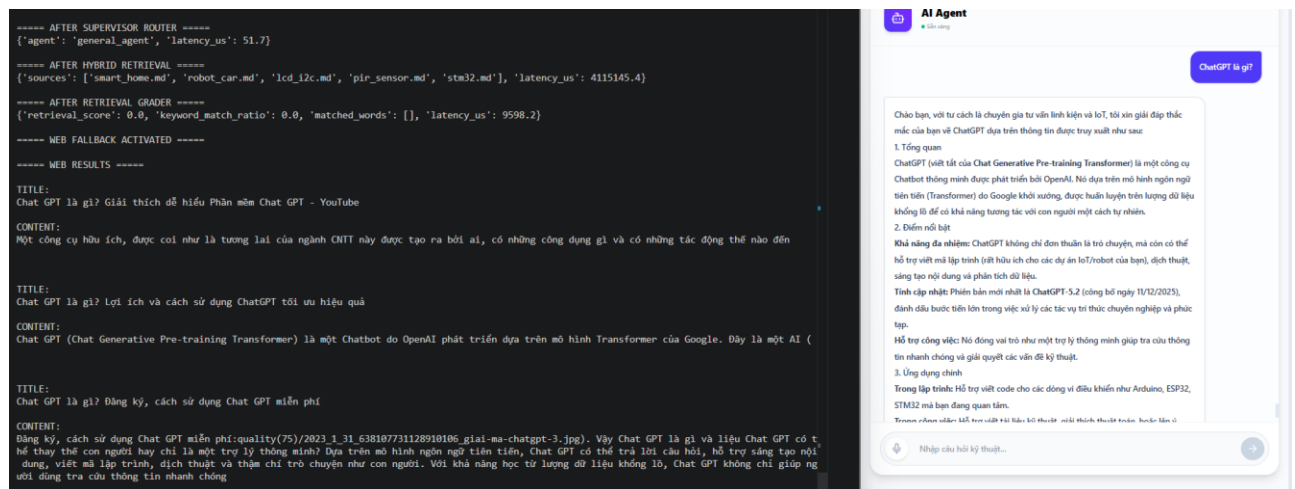
Hình 4.44 thể hiện kết quả kiểm thử với truy vấn: “Combo làm nhà thông minh gồm những gì?”. Kết quả log cho thấy Router Node đã định tuyến truy vấn tới combo_agent với latency_us = 58.2 μ s, chứng minh hệ thống đã nhận diện đúng đây là truy vấn đề xuất bộ linh kiện Smart Home.

Sau bước định tuyến, hệ thống thực hiện Hybrid Truy xuất và truy xuất được các tài liệu liên quan như smart_home.md, combo_strategy.md, rfid_rc522.md và gas_mq2.md. Truy xuất Grader trả về retrieval_score = 0.714 và keyword_match_ratio = 0.625, cho thấy context truy xuất có độ liên quan phù hợp với truy vấn người dùng trước khi được đưa vào mô hình ngôn ngữ lớn.

Trên giao diện Web, AI đã đề xuất bộ linh kiện Smart Home gồm ESP32 DevKit V1 CP2102, relay module và cảm biến DHT11, đồng thời giải thích vai trò của từng linh kiện trong hệ thống. Điều này cho thấy combo_agent không chỉ truy xuất dữ liệu mà còn có khả năng tổng hợp và tư vấn giải pháp hoàn chỉnh cho người dùng.

Backend cuối cùng trả về trạng thái POST /user-ai HTTP/1.1 200 OK, chứng minh workflow AI đã xử lý thành công từ bước định tuyến agent, retrieval, grading cho đến sinh phản hồi cuối cùng.

Kết quả định tuyến web fallback



Hình 4.45. Kết quả Web Fallback

Hình 4.45 thể hiện kết quả kiểm thử với truy vấn: “ChatGPT là gì?”. Kết quả log cho thấy Router Node đã định tuyến truy vấn tới general_agent với latency_us = 51.7 μ s, chứng minh hệ thống đã nhận diện đây là câu hỏi ngoài miền tri thức IoT và tồn kho.

Sau bước định tuyến, hệ thống thực hiện Hybrid Truy xuất và truy xuất một số tài liệu nội bộ như smart_home.md, robot_car.md và stm32.md. Tuy nhiên, Truy xuất Grader trả về

retrieval_score = 0.0 và keyword_match_ratio = 0.0, cho thấy các tài liệu truy xuất không phù hợp với truy vấn người dùng.

Do context nội bộ không đủ liên quan, workflow đã kích hoạt WEB FALLBACK ACTIVATED để truy xuất thêm dữ liệu từ Internet. Hệ thống sau đó trả về các kết quả Web liên quan đến ChatGPT và sử dụng dữ liệu này để sinh phản hồi cuối cùng trên giao diện Web.

Backend cuối cùng trả về trạng thái 200 OK, chứng minh toàn bộ pipeline AI đã xử lý thành công. Kết quả kiểm thử cho thấy hệ thống có khả năng mở rộng phạm vi tri thức khi knowledge base nội bộ không đủ dữ liệu để trả lời người dùng.

CÁC TRƯỜNG HỢP ĐỊNH TUYẾN SAI

Trường hợp 1

Trường hợp đầu tiên xảy ra với truy vấn “Smart locker hoạt động như thế nào?”.

```
===== AFTER SUPERVISOR ROUTER =====
{'agent': 'combo_agent', 'latency_us': 23.2}

===== AFTER HYBRID RETRIEVAL =====
{'sources': ['smart_lock.md', 'firebase.md', 'solenoid_lock.md', 'smart_home.md', 'esp8266.md'], 'latency_us': 4062838.6}

===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 0.8, 'keyword_match_ratio': 0.714, 'matched_words': ['smart', 'locker', 'hoạt', 'động', 'như'], 'latency_us': 1027.4}

===== GENERATE NODE =====
AGENT = combo_agent
=====
[9] QUERY: Smart locker hoạt động như thế nào?
Expected agent : consultant_agent
Predicted agent: combo_agent
Agent OK       : False
Keyword hit    : 2/3
Keyword OK     : True
Score          : 0.8
Latency        : 4070.95 ms
Sources        : ['smart_lock.md', 'firebase.md', 'solenoid_lock.md', 'smart_home.md', 'esp8266.md']
```

Hình 4.46. Kết quả của truy vấn “Smart locker hoạt động như thế nào?”.

Kết quả mong đợi là consultant_agent, tuy nhiên workflow lại định tuyến sang combo_agent. Nguyên nhân là do truy vấn chứa từ khóa “smart locker”, trong khi cơ sở tri thức của hệ thống có nhiều tài liệu liên quan tới mô hình Smart Locker và các bộ combo IoT. Vì vậy workflow đánh giá truy vấn nghiêng về nhóm đề xuất sản phẩm thay vì tư vấn kỹ thuật.

Trường hợp 2:

Đối với truy vấn “Cảm biến siêu âm HC-SR04 đo được khoảng cách bao xa?”,

```
===== AFTER SUPERVISOR ROUTER =====
{'agent': 'inventory_agent', 'latency_us': 6.8}

===== AFTER HYBRID RETRIEVAL =====
{'sources': ['ultrasonic_hcsr04.md', 'robot_car.md', 'pir_sensor.md', 'upsell_strategy.md', 'arduino_uno.md'], 'latency_us': 4055200.4}

===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 0.8, 'keyword_match_ratio': 0.778, 'matched_words': ['cảm', 'biến', 'siêu', 'hc-sr04', 'được', 'khoảng', 'cách'], 'latency_us': 602.7}

===== GENERATE NODE =====
AGENT = inventory_agent
=====
[11] QUERY: Cảm biến siêu âm HC-SR04 đo được khoảng cách bao xa?
Expected agent : consultant_agent
Predicted agent: inventory_agent
Agent OK       : False
Keyword hit    : 3/3
Keyword OK     : True
Score          : 0.8
Latency        : 4059.74 ms
Sources        : ['ultrasonic_hcsr04.md', 'robot_car.md', 'pir_sensor.md', 'upsell_strategy.md', 'arduino_uno.md']
```

Hình 4.47. Kết quả của truy vấn cảm biến siêu âm HC-SR04 đo được khoảng cách bao xa?”.

Workflow lại định tuyến sang `inventory_agent` thay vì `consultant_agent`. Nguyên nhân là do câu hỏi chứa tên module cụ thể “HC-SR04”, khiến Router Node ưu tiên truy xuất dữ liệu liên quan tới sản phẩm và tồn kho thay vì tài liệu tư vấn kỹ thuật.

Trường hợp 3:

Một trường hợp khác xuất hiện với truy vấn “Mạch L298N dùng để điều khiển động cơ gì?”.

```
===== AFTER SUPERVISOR ROUTER =====
{'agent': 'general_agent', 'latency_us': 15.5}

===== AFTER HYBRID RETRIEVAL =====
{'sources': ['robot_car.md', 'dc_motor.md', 'led_circuit.md', 'stepper_motor.md', 'arduino_uno.md'], 'latency_us': 4019341.7}

===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 1.0, 'keyword_match_ratio': 0.857, 'matched_words': ['mạch', 'l298n', 'dùng', 'để', 'điều', 'khiển', 'động'], 'latency_us': 1028.7}

===== GENERATE NODE =====
AGENT = general_agent
=====
[13] QUERY: Mạch L298N dùng để điều khiển động cơ gì?
Expected agent : consultant_agent
Predicted agent: general_agent
Agent OK       : False
Keyword hit    : 2/2
Keyword OK     : True
Score          : 1.0
Latency        : 4026.32 ms
Sources        : ['robot_car.md', 'dc_motor.md', 'led_circuit.md', 'stepper_motor.md', 'arduino_uno.md']
```

Hình 4.48. Kết quả của truy vấn mạch L298N dùng để điều khiển động cơ gì?”.

Mặc dù đây là truy vấn tư vấn kỹ thuật, workflow lại định tuyến sang `general_agent`. Nguyên nhân là do câu hỏi tương đối ngắn và thiếu các từ khóa kỹ thuật hỗ trợ như “thông số”, “nguyên lý” hoặc “PWM”, khiến Router Node không xác định rõ mục đích của truy vấn.

Trường hợp 4:

Đối với truy vấn “Set linh kiện lắp ráp cánh tay robot 4 bậc”, workflow định tuyến sang `general_agent` thay vì `combo_agent`.

```
===== AFTER SUPERVISOR ROUTER =====
{'agent': 'general_agent', 'latency_us': 22.1}

===== AFTER HYBRID RETRIEVAL =====
{'sources': ['robot_car.md', 'upsell_strategy.md', 'arduino.md', 'combo_strategy.md', 'buzzer.md'], 'latency_us': 4082593.6}

===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 0.556, 'keyword_match_ratio': 0.556, 'matched_words': ['linh', 'kiện', 'lắp', 'ráp', 'robot'], 'latency_us': 1162.0}

===== GENERATE NODE =====
AGENT = general_agent
=====
[18] QUERY: Set linh kiện lắp ráp cánh tay robot 4 bậc
Expected agent : combo_agent
Predicted agent: general_agent
Agent OK       : False
Keyword hit    : 1/3
Keyword OK     : True
Score          : 0.5555555555555556
Latency        : 4091.08 ms
Sources        : ['robot_car.md', 'upsell_strategy.md', 'arduino.md', 'combo_strategy.md', 'buzzer.md']
```

Hình 4.49. Kết quả của truy vấn “Set linh kiện lắp ráp cánh tay robot 4 bậc”

Truy vấn này đồng thời chứa yêu cầu gợi ý linh kiện, tư vấn kỹ thuật và xây dựng mô hình thực tế. Do câu hỏi chứa nhiều mục tiêu cùng lúc nên Router Node chưa xác định được agent phù hợp nhất.

Trường hợp 5:

Tương tự, truy vấn “Gợi ý trọn bộ làm máy rót rượu tự động” cũng bị workflow định tuyến sang `general_agent` thay vì `combo_agent`.

```
===== AFTER SUPERVISOR ROUTER =====
{'agent': 'general_agent', 'latency_us': 36.4}

===== AFTER HYBRID RETRIEVAL =====
{'sources': ['wifi.md', 'combo_strategy.md', 'gas_mq2.md', 'stepper_motor.md', 'seven_segment.md'], 'latency_us': 4096400.3}

===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 0.667, 'keyword_match_ratio': 0.571, 'matched_words': ['gợi', 'làm', 'máy', 'động'], 'latency_us': 1045.5}

===== GENERATE NODE =====
AGENT = general_agent
=====
[19] QUERY: Gợi ý trọn bộ làm máy rót rượu tự động
Expected agent : combo_agent
Predicted agent: general_agent
Agent OK       : False
Keyword hit    : 2/3
Keyword OK     : True
Score          : 0.6666666666666666
Latency        : 4104.85 ms
Sources        : ['wifi.md', 'combo_strategy.md', 'gas_mq2.md', 'stepper_motor.md', 'seven_segment.md']
```

Hình 4.50. Kết quả của truy vấn “Gợi ý trọn bộ làm máy rót rượu tự động”

Nguyên nhân là do truy vấn đồng thời chứa ý tưởng dự án IoT, đề xuất linh kiện và mô hình tự động hóa. Workflow hiện tại vẫn phụ thuộc nhiều vào luật định tuyến thủ công và so khớp từ khóa nên chưa xử lý tốt các truy vấn ngữ nghĩa dài.

Trường hợp 6:

```

===== AFTER SUPERVISOR ROUTER =====
{'agent': 'inventory_agent', 'latency_us': 15.8}

===== AFTER HYBRID RETRIEVAL =====
{'sources': ['upsell_strategy.md', 'combo_strategy.md', 'smart_home.md', 'robot_car.md', 'soil_moisture.md'], 'latency_us': 4114911.8}

===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 0.082, 'keyword_match_ratio': 0.25, 'matched_words': ['tra', 'module', 'trong'], 'latency_us': 1204.0}

===== WEB FALLBACK ACTIVATED =====

===== WEB RESULTS =====

TITLE:
mọi người cho em hỏi có module sim 4g nào có thể lấy được các giá ...

CONTENT:
mọi người cho em hỏi có module sim 4g nào có thể lấy được các giá trị LAC và CID từ trạm BTS không ạ. em đang dùng A7670C nhưng không được ạ.

TITLE:
Mọi người ai đã làm việc với module 4g simcom a7670c cho em hỏi ...

CONTENT:
Mọi người ai đã làm việc với module 4g simcom a7670c cho em hỏi với ạ. Em có sử dụng module 4g kết nối với arduino điều khiển gửi tin nhắn, gọi

TITLE:
Cách kiểm tra dung lượng sim data - YouTube

CONTENT:
https://sim4.me/mobile/traffic.

===== GENERATE NODE =====
AGENT = web_search_agent
=====
[20] QUERY: Kiểm tra xem module sim A7670C còn bao nhiêu cái trong kho?
Expected agent : inventory_agent
Predicted agent: web_search_agent
Agent OK      : False
Keyword hit   : 1/3
Keyword OK    : True
Score         : 1.0
Latency       : 5175.19 ms
Sources       : ['web_fallback']

```

Hình 4.51. Kết quả của truy vấn “Kiểm tra xem module sim A7670C còn bao nhiêu cái trong kho?”

Đối với truy vấn “Kiểm tra xem module sim A7670C còn bao nhiêu cái trong kho?”, workflow lại định tuyến sang web_search_agent thay vì inventory_agent. Nguyên nhân là do từ khóa “A7670C” thường xuất hiện trong các tài liệu GSM và IoT networking nên workflow đánh giá đây là truy vấn tìm kiếm thông tin Internet thay vì truy vấn tồn kho nội bộ.

Trường hợp 7:

Một lỗi khác xuất hiện ở truy vấn “Thời tiết Hà Nội cuối tuần này có mưa không?”.

```

===== AFTER SUPERVISOR ROUTER =====
{'agent': 'general_agent', 'latency_us': 39.8}

===== AFTER HYBRID RETRIEVAL =====
{'sources': ['weather_station.md', 'rain_sensor.md', 'upsell_strategy.md', 'combo_strategy.md', 'inventory_policy.md'], 'latency_us': 4047524.0}

===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 0.571, 'keyword_match_ratio': 0.5, 'matched_words': ['thời', 'tiết', 'này', 'mưa'], 'latency_us': 886.5}

===== GENERATE NODE =====
AGENT = general_agent
=====
[26] QUERY: Thời tiết Hà Nội cuối tuần này có mưa không?
Expected agent : web_search_agent
Predicted agent: general_agent
Agent OK      : False
Keyword hit   : 0/1
Keyword OK    : False
Score         : 0.5714285714285714
Latency       : 4055.51 ms
Sources       : ['weather_station.md', 'rain_sensor.md', 'upsell_strategy.md', 'combo_strategy.md', 'inventory_policy.md']

```

Hình 4.52. Kết quả của truy vấn “Thời tiết Hà Nội cuối tuần này có mưa không?”.

Kết quả mong đợi là web_search_agent, tuy nhiên workflow lại định tuyến sang general_agent. Nguyên nhân là do cơ sở tri thức nội bộ có nhiều tài liệu liên quan tới weather station, cảm biến mưa và hệ thống thời tiết IoT, khiến workflow nhầm đây là truy vấn IoT nội bộ thay vì truy vấn thời tiết thực tế.

Trường hợp 8:

Trường hợp cuối cùng là truy vấn “Ktra hàng tồn của servo sg90”.

```
===== AFTER SUPERVISOR ROUTER =====
{'agent': 'consultant_agent', 'latency_us': 11.4}

===== AFTER HYBRID RETRIEVAL =====
{'sources': ['servo_sg90.md', 'inventory_strategy.md', 'stock_analysis.md', 'esp32_s3.md', 'arduino.md'], 'latency_us': 4059457.3}

===== AFTER RETRIEVAL GRADER =====
{'retrieval_score': 0.8, 'keyword_match_ratio': 0.833, 'matched_words': ['hàng', 'tồn', 'của', 'servo', 'sg90'], 'latency_us': 425.7}

===== GENERATE NODE =====
AGENT = consultant_agent
=====
[30] QUERY: Ktra hàng tồn của servo sg90
Expected agent : inventory_agent
Predicted agent: consultant_agent
Agent OK      : False
Keyword hit   : 2/3
Keyword OK    : True
Score         : 0.8
Latency       : 4063.27 ms
Sources       : ['servo_sg90.md', 'inventory_strategy.md', 'stock_analysis.md', 'esp32_s3.md', 'arduino.md']
```

Hình 4.53. Kết quả của truy vấn “Kiểm tra hàng tồn của servo sg90”.

Workflow định tuyến sang consultant_agent thay vì inventory_agent. Nguyên nhân là do người dùng sử dụng từ viết tắt “Ktra”, câu hỏi không đúng cấu trúc ngữ pháp đầy đủ và thiếu các từ khóa tồn kho mạnh như “số lượng”, “còn bao nhiêu” hoặc “trong kho”. Trong khi đó, từ khóa “servo sg90” lại thường xuất hiện trong các tài liệu tư vấn kỹ thuật nên workflow bị lệch sang nhóm tư vấn kỹ thuật.

4.4.5. Kết quả đánh giá Truy xuất và Hybrid Search

Hệ thống sử dụng Hybrid Truy xuất kết hợp:

- Sparse Retrieval,
- Dense Retrieval,
- Semantic Search

Nhằm tăng độ chính xác retrieval.

Bảng 4.4. Kết quả Retrieval

Agent	Truy xuất Score trung bình
web_search_agent	0.939
consultant_agent	0.818
inventory_agent	0.813
combo_agent	0.713

Bảng 4.5. Truy xuất Score theo từng Agent

Agent	Truy xuất Score trung bình
web_search_agent	0.939
consultant_agent	0.818
inventory_agent	0.813
combo_agent	0.713



Hình 4.54. Biểu đồ truy xuất Score trung bình theo từng Agent

Hình 4.55. Kết quả Truy xuất Grading trong AI Backend

Trong đó:

- retrieval_score biểu diễn mức độ liên quan giữa câu hỏi và knowledge retrieval,

- `keyword_match_ratio` biểu diễn tỷ lệ khớp keyword giữa câu hỏi và context retrieval,
- `matched_words` là các keyword được phát hiện trong knowledge retrieval,
- `force_web_fallback` cho biết workflow có cần chuyển sang Web Fallback hay không.

Kết quả trên cho thấy truy xuất score đạt: `retrieval_score = 1.0`

Điều này chứng minh context truy xuất có độ liên quan rất cao đối với câu hỏi người dùng. Đồng thời: `keyword_match_ratio = 0.75` cho thấy phần lớn keyword quan trọng trong câu hỏi đã xuất hiện trong knowledge retrieval.

Ngoài ra, hệ thống còn hiển thị: `matched_words = ['esp32', 'mqtt', 'hỗ trợ']` Kết quả này chứng minh Truy xuất Layer đã truy xuất đúng miền tri thức kỹ thuật liên quan đến ESP32 và giao thức MQTT.

Trong workflow AI Backend, nếu: truy xuất score thấp, hoặc keyword match ratio thấp, workflow sẽ không tiếp tục generate response bằng context nội bộ mà chuyển sang Web Fallback để truy xuất dữ liệu bên ngoài.

Cơ chế điều hướng này được triển khai trong: `route_after_grade()`

Nếu truy xuất đạt chất lượng tốt: `generate` sẽ được kích hoạt để tiếp tục Prompt Builder và LLM Generation.

Ngược lại, nếu truy xuất không đủ liên quan: `web_fallback` sẽ được kích hoạt nhằm tránh đưa context nhiễu vào LLM.

Kết quả kiểm thử cho thấy hệ thống không “mù quáng” sử dụng mọi dữ liệu truy xuất để sinh phản hồi. Đây là điểm quan trọng trong kiến trúc Agentic RAG vì truy xuất không phải lúc nào cũng trả về tài liệu chính xác.

Trong thực tế, nếu người dùng đặt câu hỏi nằm ngoài knowledge base nội bộ, truy xuất có thể lấy nhầm tài liệu không liên quan. Nếu đưa trực tiếp context nhiều vào LLM, mô hình có thể sinh phản hồi sai hoặc hallucination.

Truy xuất Grader giúp hạn chế vấn đề này bằng cách:

- đánh giá chất lượng retrieval,
- phát hiện context không phù hợp,
- kích hoạt Web Fallback khi cần thiết.

Nhờ đó, AI Backend có khả năng kiểm soát chất lượng context tốt hơn so với pipeline RAG truyền thống chỉ truy xuất rồi genera

Kết quả cho thấy:

- web_search_agent có truy xuất score cao nhất,
- combo_agent có truy xuất score thấp nhất.

Nguyên nhân là do các query web thường:

- rõ nghĩa,
- semantic đơn giản,
- truy xuất domain đơn lẻ.

Trong khi đó các query combo thường:

- dài,
- chứa nhiều semantic concept,
- truy xuất nhiều loại linh kiện khác nhau.

Bảng 4.6. Một số kết quả truy xuất tiêu biểu

Câu hỏi	Source truy xuất	Truy xuất Score
---------	------------------	-----------------

ESP32 có hỗ trợ MQTT không?	esp32.md	1
RFID RC522 dùng giao tiếp gì?	rfid_rc522.md	1
DHT11 dùng để đo gì?	dht11.md	1

Trong quá trình kiểm thử, cùng một truy vấn:

“ESP32 có hỗ trợ MQTT không?”

được sử dụng để đánh giá cả Truy xuất Layer và API Response Layer của hệ thống.

Đầu tiên, terminal backend hiển thị log Hybrid Truy xuất như Hình 4.54.

```
===== AFTER HYBRID RETRIEVAL =====
{'sources': ['esp32.md', 'bluetooth.md', 'upsell_strategy.md', 'mqtt.md', 'smart_home.md'], 'latency_us': 4091463.1}
```

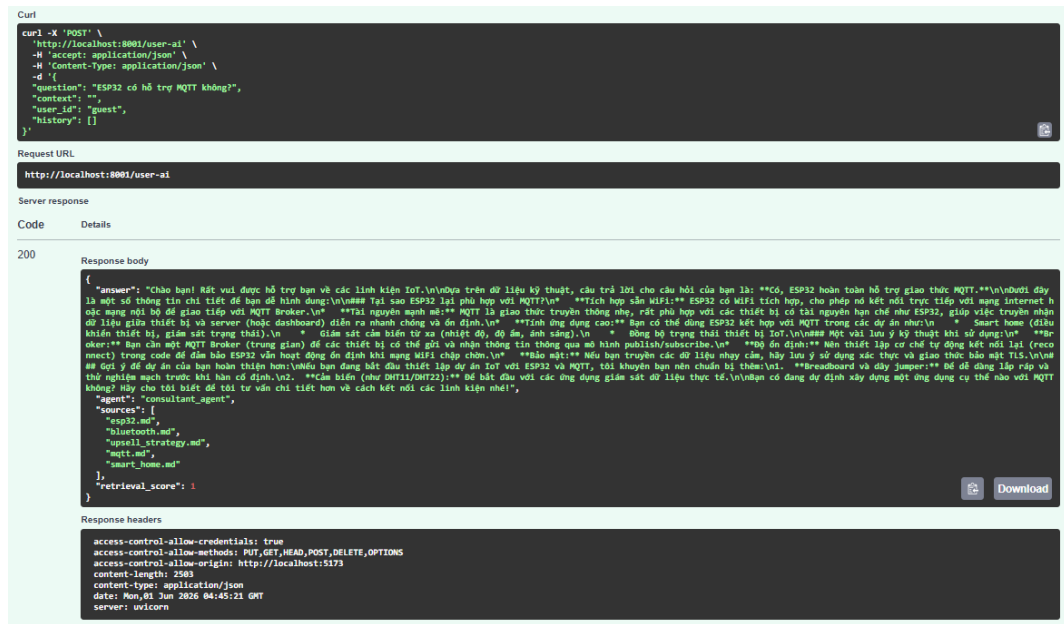
Hình 4.56. Log Hybrid Truy xuất trong terminal backend

Log này phản ánh hoạt động nội bộ của Truy xuất Layer. Hệ thống đã truy xuất nhiều tài liệu liên quan từ knowledge base nội bộ trước khi sinh phản hồi bằng LLM.

Sau bước retrieval, workflow tiếp tục thực hiện:

- Prompt Building,
- Agent Routing,
- LLM Generation.

Kết quả cuối cùng được trả về qua API như Hình 4.55.



Hình 4.57. Response API của AI Backend

Response API cho thấy dữ liệu truy xuất đã được sử dụng thành công để sinh phản hồi cuối cùng cho người dùng. Trong đó:

agent cho biết workflow đã định tuyến truy vấn sang consultant_agent,

sources biểu diễn các tài liệu được truy xuất sử dụng,

retrieval_score phản ánh mức độ liên quan của context retrieval.

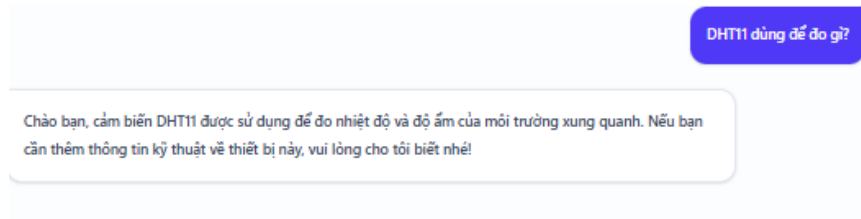
Việc cùng một câu hỏi xuất hiện ở cả terminal truy xuất và API response chứng minh workflow AI Backend hoạt động theo pipeline nhiều tầng thay vì sinh phản hồi trực tiếp từ LLM. Truy xuất Layer, Prompt Builder và LLM Generation được triển khai tách biệt nhằm giúp hệ thống dễ kiểm soát, debug và đánh giá chất lượng context trong quá trình vận hành.

4.4.6. Đánh giá khả năng giảm Hallucination bằng RAG

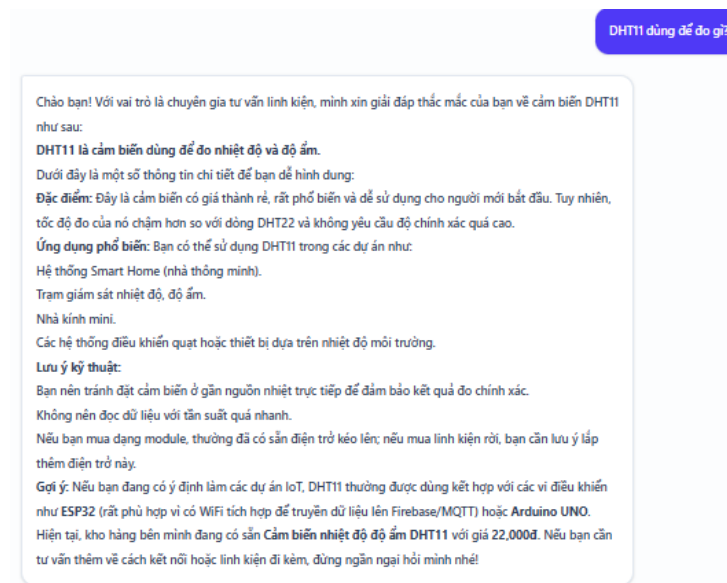
Hệ thống sử dụng Retrieval-Augmented Generation (RAG) nhằm giảm hiện tượng hallucination của Large Language Model.

Để đánh giá ảnh hưởng của RAG đối với chất lượng phản hồi AI, hệ thống được kiểm thử trong hai trường hợp:

- không sử dụng retrieval,
- có sử dụng truy xuất từ tập tri thức nội bộ.
-



Hình 4.58. Phản hồi AI khi không sử dụng RAG



Hình 4.59. Phản hồi AI khi sử dụng RAG

```
===== AFTER HYBRID RETRIEVAL =====
{'sources': ['dht11.md', 'smart_home.md', 'ultrasonic_hcsr04.md', 'weather_station.md', 'ldr_sensor.md'], 'latency_us': 85362.9}
```

Hình 4.60. Truy xuất log thể hiện source được truy xuất

Kết quả kiểm thử cho thấy khi không sử dụng retrieval, AI chủ yếu trả lời dựa trên kiến thức tổng quát của mô hình ngôn ngữ nên phản hồi thường:

- thiếu thông tin kỹ thuật chi tiết,
- không bám sát dữ liệu nội bộ,

- hoặc mang tính chất mô tả chung chung.

Ngược lại, khi sử dụng RAG, hệ thống truy xuất đúng tài liệu liên quan trước khi sinh phản hồi. Điều này giúp AI:

- bám sát dữ liệu nội bộ hơn,
- hạn chế hallucination,
- tăng độ chính xác kỹ thuật.

Kết quả này cho thấy chất lượng truy xuất đóng vai trò rất quan trọng trong việc giảm hallucination và tăng độ tin cậy của hệ thống AI Backend.

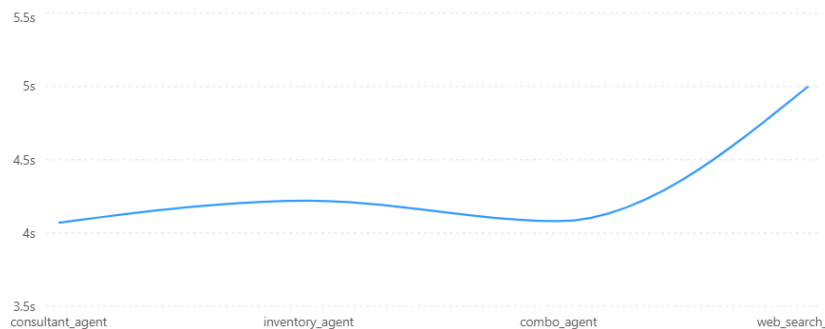
4.4.7. Kết quả đánh giá thời gian phản hồi AI

Thời gian phản hồi AI được đo bao gồm:

- Routing Agent,
- Truy xuất dữ liệu,
- Context Injection,
- AI Generation.

Bảng 4.7. Kết quả đo thời gian phản hồi

Agent	Thời gian trung bình
consultant_agent	4.07s
inventory_agent	4.22s
combo_agent	4.08s
web_search_agent	5.00s
Tổng trung bình	4.29s



Hình 4.61. Biểu đồ thời gian phản hồi trung bình theo từng Agent

Kết quả cho thấy:

- web_search_agent có latency cao nhất do cần truy cập dữ liệu Internet.
- consultant_agent và combo_agent có thời gian xử lý tương đối ổn định khoảng 4 giây.
- inventory_agent có thời gian phản hồi thấp hơn do truy xuất đơn giản hơn.

Mặc dù chưa đạt realtime tức thời, kết quả hiện tại vẫn phù hợp với:

- AI chatbot,
- Smart Retail,
- AIoT Web Application.

4.4.8. Đánh giá tổng thể hệ thống AI Backend

Bảng 4.8. Kết quả tổng hợp hệ thống AI

Hạng mục	Kết quả
Tổng số câu kiểm thử	35
Routing Accuracy	77.14%
Truy xuất Score trung bình	817
Thời gian phản hồi trung bình	4.29s
Số câu route sai	8

Kết quả thực nghiệm cho thấy hệ thống AI Backend có khả năng hoạt động tương đối ổn định trong các tác vụ:

- tư vấn kỹ thuật linh kiện,
- phân tích tồn kho,
- gợi ý combo linh kiện,
- truy xuất tri thức nội bộ,
- đồng bộ dữ liệu realtime với Firebase.

Kiến trúc:

- Multi-Agent,

- LangGraph Workflow,
- Hybrid Retrieval,
- Qdrant Vector Database

giúp hệ thống tăng:

- khả năng retrieval,
- semantic understanding,
- workflow orchestration.

Mặc dù vẫn còn tồn tại một số lỗi route sai đối với:

- query dài,
- query viết tắt,
- query đa semantic context,

hệ thống hiện tại vẫn cho thấy khả năng hoạt động như một trợ lý AI thông minh trong môi trường Smart Retail và AIoT.

4.5. Kết quả phân cứng smart locker



Hình 4.62. Kết quả khởi động LCD và kết nối hệ thống Smart Locker

Hình trên thể hiện trình tự khởi động và kết nối của hệ thống Smart Locker sử dụng ESP32, LCD I2C và Firebase Realtime Database. Sau khi hệ thống được cấp nguồn, màn hình LCD lần lượt hiển thị các trạng thái hoạt động nhằm thông báo tiến trình khởi tạo của thiết bị.

Đầu tiên, LCD hiển thị thông báo “Khởi động hệ thống”, cho biết ESP32 đã bắt đầu quá trình khởi tạo phần cứng và các module ngoại vi. Tiếp theo, hệ thống chuyển sang bước “Kết nối WiFi” để thiết lập kết nối mạng Internet phục vụ giao tiếp với Firebase.

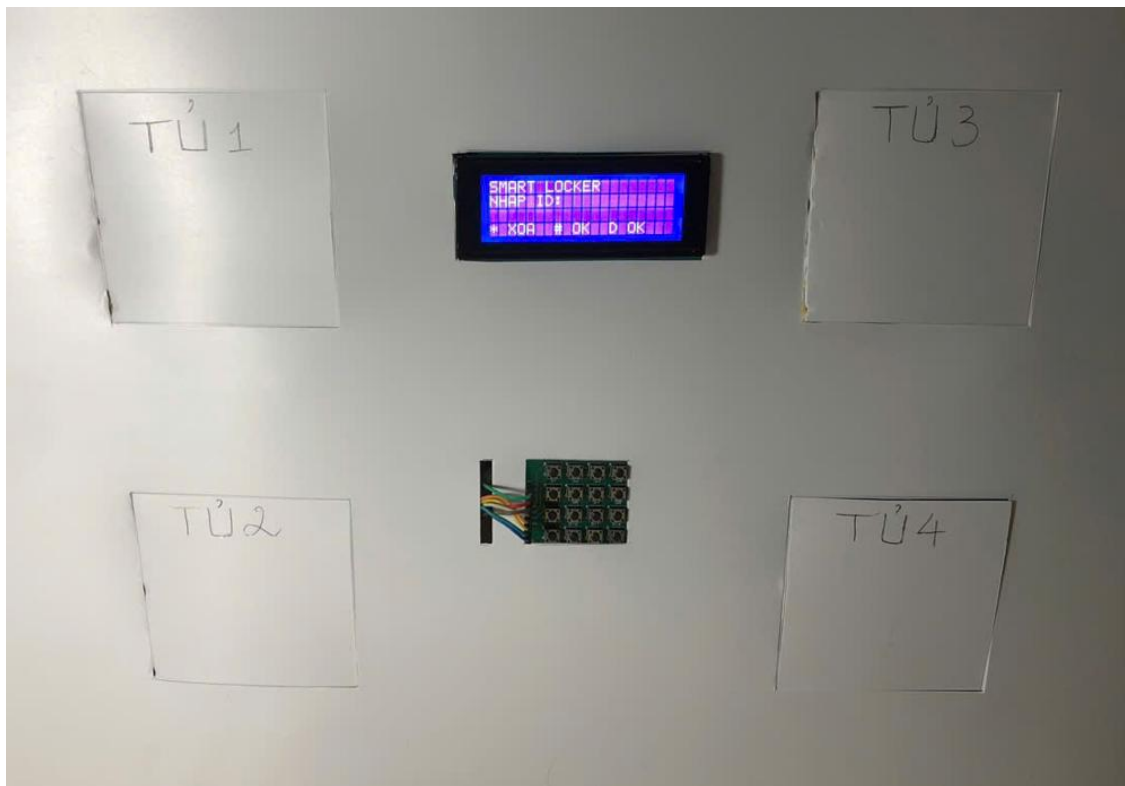
Sau khi kết nối thành công, LCD hiển thị trạng thái “WiFi đã kết nối”, cho thấy ESP32 đã online và sẵn sàng truyền dữ liệu. Hệ thống tiếp tục thực hiện bước “Kết nối Firebase” nhằm đồng bộ với Firebase Realtime Database. Khi quá trình xác thực và kết nối hoàn tất, LCD hiển thị thông báo “Firebase đã kết nối”.

Cuối cùng, hệ thống chuyển sang trạng thái “Bắt đầu nhập ID”, cho phép người dùng nhập mã nhận hàng thông qua keypad để mở Smart Locker. Trình tự hiển thị này giúp người dùng và quản trị viên dễ dàng theo dõi trạng thái hoạt động của hệ thống trong thời gian thực.

Kết quả thực nghiệm cho thấy ESP32 đã kết nối ổn định với WiFi và Firebase trước khi hệ thống chuyển sang chế độ xử lý dữ liệu Smart Locker. Điều này chứng minh cơ chế khởi tạo và kết nối mạng của hệ thống hoạt động chính xác và ổn định trong thực tế.

Kết quả thực nghiệm cho thấy cơ chế xác thực mã nhận hàng giữa Website, Firebase và ESP32 đã hoạt động ổn định. Hệ thống có khả năng phản hồi trực quan trên LCD, xử lý dữ liệu realtime và tự động điều khiển Smart Locker theo đúng thông tin đơn hàng của khách hàng.

Kết quả mô hình ở trạng thái chờ nhập ID

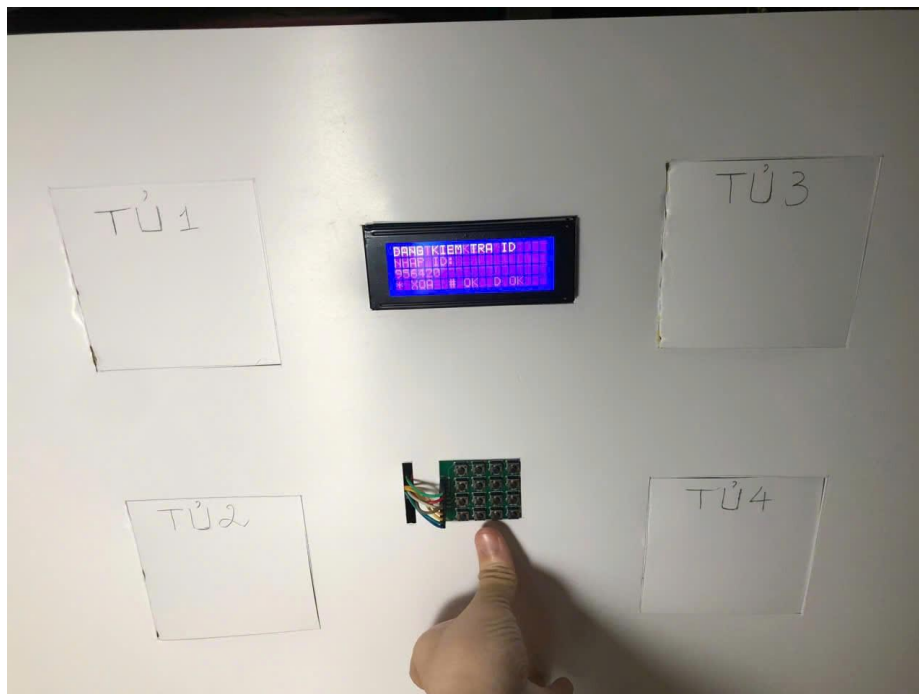


Hình 4.63. Mô hình Smart Locker ở trạng thái chờ nhập ID

Hình 4.63 thể hiện mô hình phần cứng Smart Locker sau khi khởi động và chuyển sang trạng thái sẵn sàng nhận mã. Trên mặt trước mô hình có bốn ngăn tủ được đánh dấu Tủ 1, Tủ 2, Tủ 3 và Tủ 4. Ở giữa mô hình là màn hình LCD 20x4 dùng để hiển thị trạng thái hệ thống và hướng dẫn thao tác cho người dùng. Bên dưới LCD là keypad dùng để nhập mã nhận hàng.

Ở trạng thái ban đầu, LCD hiển thị nội dung SMART LOCKER, NHAP ID và các phím chức năng như * XOA, # OK, D OK. Điều này cho thấy ESP32 đã hoàn tất quá trình khởi tạo phần cứng, màn hình LCD hoạt động ổn định và hệ thống đang chờ người dùng nhập mã nhận hàng. Giao diện hiển thị trực tiếp trên LCD giúp người dùng dễ dàng nhận biết trạng thái hiện tại của thiết bị trước khi thao tác.

Kết quả nhập mã nhận hàng trên keypad



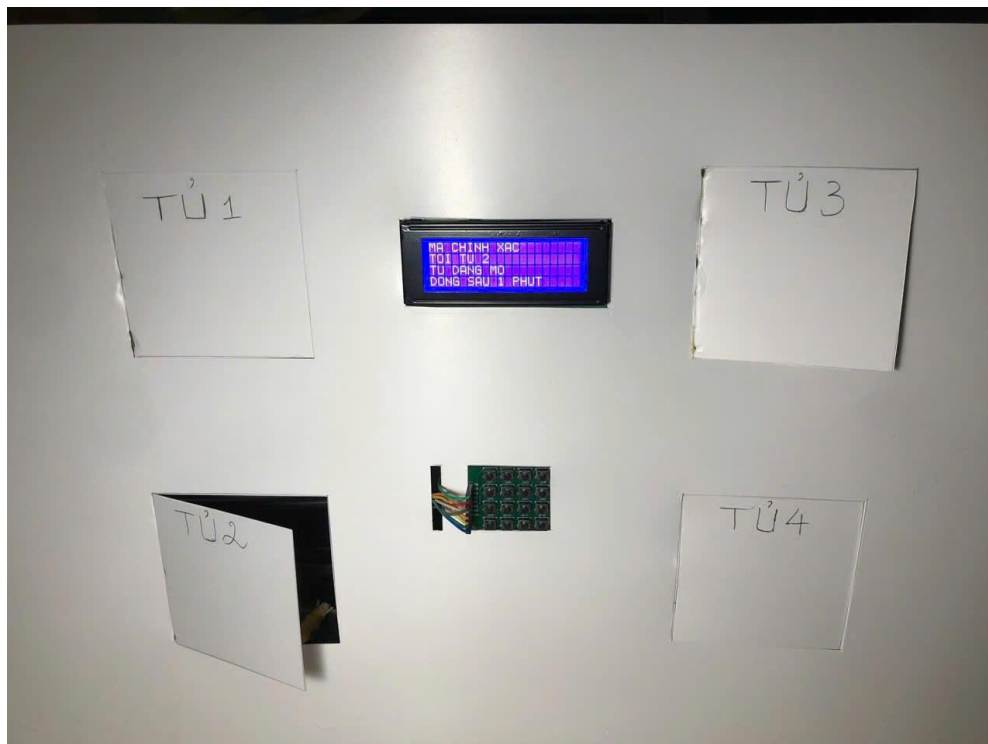
Hình 4.64. Người dùng thao tác nhập mã nhận hàng trên keypad

Hình 4.64 thể hiện quá trình người dùng nhập mã nhận hàng thông qua keypad. Trong quá trình nhập, mã được cập nhật trực tiếp lên màn hình LCD để người dùng

kiểm tra trước khi xác nhận. Chức năng xóa và xác nhận được bố trí rõ ràng trên LCD, giúp thao tác nhập mã trở nên trực quan hơn.

Sau khi người dùng nhập đủ số ký tự của mã nhận hàng và nhấn phím xác nhận, ESP32 thực hiện truy vấn dữ liệu từ Firebase Realtime Database. Hệ thống duyệt các bản ghi trong nhánh `locker_access`, so sánh mã đã nhập với trường `current_pin`. Nếu không tìm thấy mã trùng khớp, LCD hiển thị thông báo mã chưa chính xác và yêu cầu người dùng nhập lại. Trong trường hợp này, servo không quay và không có ngăn tủ nào được mở.

Kết quả xác thực mã và mở đúng ngăn tủ



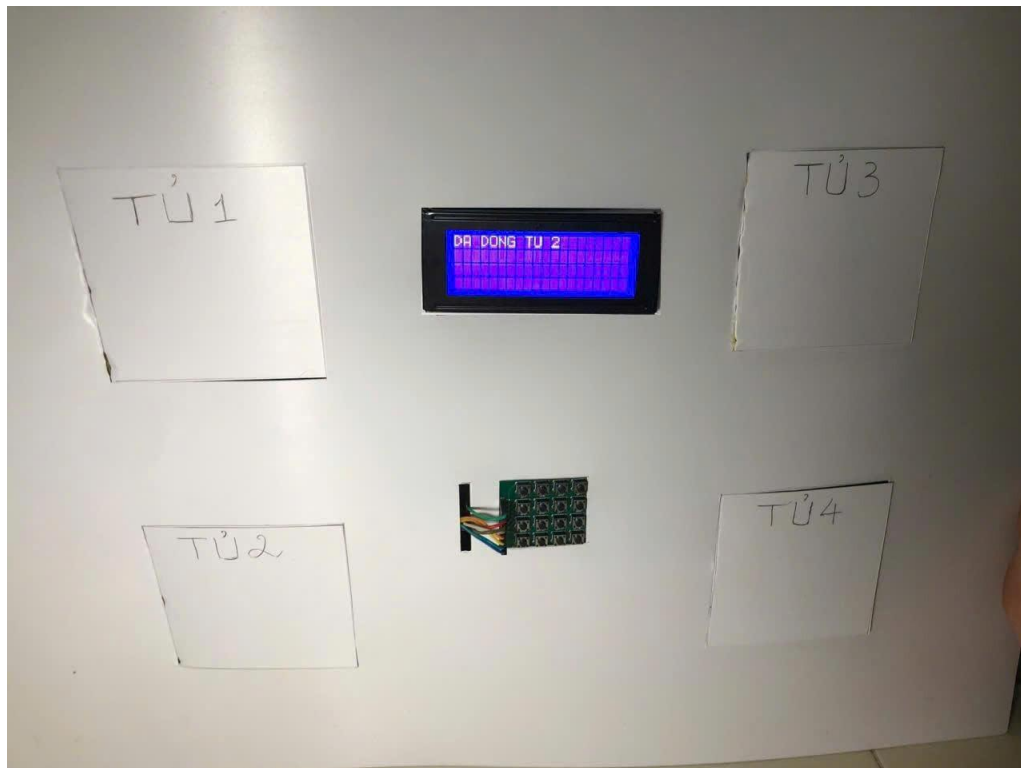
Hình 4.65. Kết quả xác thực mã nhận hàng và mở ngăn tủ tương ứng

Hình 4.65 thể hiện trường hợp người dùng nhập đúng mã nhận hàng. Sau khi ESP32 xác thực mã nhập với trường `current_pin` trên Firebase, hệ thống tiếp tục đọc trường `box_no` của đơn hàng để xác định ngăn tủ cần mở. Trong ảnh minh họa, LCD hiển thị thông báo MA CHINH XAC, TOI TU 4, TU DANG MO và DONG SAU 1 PHUT, đồng thời cửa Tủ 4 được mở bằng cơ cấu servo.

Kết quả này chứng minh quá trình liên kết giữa dữ liệu đơn hàng trên Firebase và phần cứng điều khiển đã hoạt động đúng. Việc mở tủ không phụ thuộc vào mã được lập trình cứng trong ESP32, mà hoàn toàn dựa trên dữ liệu `current_pin` và `box_no` được lưu trong Firebase. Nhờ đó, hệ thống có thể thay đổi mã nhận hàng và ngăn tủ tương ứng theo từng đơn hàng mà không cần nạp lại chương trình cho ESP32.

Các ngăn tủ còn lại hoạt động theo cùng một logic. Nếu Firebase lưu `box_no` là 1, 2, 3 hoặc 4, ESP32 sẽ lần lượt điều khiển các kênh CH0, CH1, CH2 hoặc CH3 trên PCA9685. Vì vậy, chỉ cần một ảnh minh họa quá trình mở một ngăn tủ cũng có thể đại diện cho nguyên lý hoạt động của toàn bộ bốn ngăn tủ trong mô hình.

Kết quả tự động đóng tủ sau thời gian `time_out`



Hình 4.66. Kết quả tự động đóng tủ sau thời gian timeout

Hình 4.66 thể hiện kết quả sau khi hệ thống tự động đóng tủ. Trong chương trình điều khiển, thời gian mở tủ được đặt là 1 phút, tương ứng với giá trị `time_out = "1 min"` và `openDurationMs = 60000` trên Firebase. Sau khi servo mở cửa, ESP32 ghi

nhận thời điểm mở và bắt đầu đếm thời gian bằng bộ đếm nội bộ. Khi đủ 60 giây, servo được điều khiển quay về vị trí đóng.

Sau khi tủ đóng, LCD hiển thị thông báo DA DONG TU X, trong đó X là số tủ đã được mở trong lần nhận hàng. Việc hiển thị thông báo này giúp người dùng và người vận hành biết rằng quá trình nhận hàng đã hoàn tất. Đồng thời, trạng thái trên Firebase cũng được cập nhật sang closed nhằm phản ánh đúng trạng thái thực tế của tủ.

Kết quả đồng bộ trạng thái opened và closed trên Firebase



Hình 4.67. Dữ liệu Firebase khi trạng thái tủ chuyển từ opened sang closed

Hình 4.67 thể hiện dữ liệu trên Firebase Realtime Database trong hai thời điểm khác nhau của cùng một quy trình nhận hàng. Ở thời điểm tủ được mở, bản ghi đơn hàng có status là opened, đồng thời Firebase lưu openedAt để ghi nhận thời điểm mở tủ. Trường box_no xác định ngăn tủ được mở, trường current_pin lưu mã nhận hàng, còn time_out và openDurationMs thể hiện thời gian cho phép tủ mở.

Sau khi hết thời gian chờ, ESP32 điều khiển servo đóng tủ và cập nhật lại dữ liệu Firebase. Khi đó, trường status được chuyển sang closed và closedAt được ghi nhận để lưu lại thời điểm đóng tủ. Cơ chế này giúp hệ thống quản trị có thể theo dõi được

toàn bộ quá trình mở và đóng tủ theo thời gian thực, đồng thời tạo cơ sở dữ liệu lịch sử cho mỗi đơn hàng.

Kết quả trên cho thấy dữ liệu trạng thái giữa phần cứng và Firebase được đồng bộ đúng theo thực tế vận hành. Khi tủ mở, Firebase ghi nhận opened; khi tủ đóng, Firebase ghi nhận closed. Đây là yêu cầu quan trọng để Website quản trị và hệ thống giám sát có thể xác định chính xác trạng thái của từng đơn hàng và từng ngăn tủ.

Đánh giá kết quả thực nghiệm phần cứng

Qua quá trình kiểm thử, mô hình Smart Locker đã đáp ứng được các chức năng phần cứng chính gồm: hiển thị trạng thái trên LCD, nhập mã nhận hàng bằng keypad, xác thực mã với Firebase, đọc số tủ từ trường box_no, mở đúng ngăn tủ bằng servo thông qua PCA9685, tự động đóng tủ sau thời gian quy định và cập nhật trạng thái lên Firebase.

Kết quả thực nghiệm cũng cho thấy hệ thống có khả năng phản hồi trực quan cho người dùng. Khi mã đúng, LCD hiển thị tủ cần đến và trạng thái tủ đang mở; khi tủ tự động đóng, LCD hiển thị thông báo đã đóng tủ. Khi mã sai, hệ thống không điều khiển servo và yêu cầu người dùng nhập lại. Nhờ đó, quy trình nhận hàng được đảm bảo rõ ràng, hạn chế nhầm lẫn và tăng tính an toàn trong quá trình sử dụng.

Về phía quản trị, dữ liệu trên Firebase cung cấp đầy đủ thông tin về mã nhận hàng, số tủ, thời điểm mở, thời điểm đóng và trạng thái hiện tại. Sự kết hợp giữa ESP32, Firebase Realtime Database, LCD, keypad, PCA9685 và servo đã tạo thành một quy trình nhận hàng tự động, có thể giám sát từ xa và phù hợp với mục tiêu xây dựng hệ thống Smart Locker trong đề tài.

Từ các kết quả trên, có thể kết luận phần cứng Smart Locker đã hoạt động đúng theo thiết kế. Hệ thống có thể xử lý linh hoạt nhiều đơn hàng khác nhau, trong đó mỗi đơn hàng có một mã current_pin và một giá trị box_no tương ứng. Điều này cho phép quản trị viên gán hàng vào từng ngăn tủ, khách hàng nhận hàng bằng mã được cấp, còn hệ thống tự động điều khiển và ghi nhận trạng

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đồ án đã nghiên cứu, thiết kế và triển khai thành công mô hình AIoT Smart Retail nhằm hỗ trợ quản lý bán hàng, quản lý kho linh kiện điện tử và tư vấn kỹ thuật IoT thông minh trên nền tảng Web. Hệ thống được xây dựng theo kiến trúc tích hợp giữa công nghệ Internet of Things (IoT), điện toán đám mây và trí tuệ nhân tạo, tạo nên một nền tảng quản lý hiện đại có khả năng tự động hóa nhiều nghiệp vụ trong hoạt động bán lẻ.

Về phần cứng, hệ thống sử dụng vi điều khiển ESP32 để thu thập và truyền dữ liệu từ các thiết bị cảm biến lên Firebase Realtime Database theo thời gian thực. Cơ chế đồng bộ dữ liệu trực tuyến giúp các thông tin về trạng thái thiết bị, số lượng sản phẩm và dữ liệu giám sát luôn được cập nhật liên tục giữa phần cứng và phần mềm.

Về phần mềm, hệ thống Web được phát triển bằng ReactJS kết hợp với Firebase Authentication và Firebase Realtime Database, cho phép quản lý người dùng, sản phẩm, đơn hàng, tồn kho và dữ liệu IoT trên cùng một nền tảng. Giao diện được thiết kế theo hướng trực quan, dễ sử dụng và hỗ trợ truy cập từ nhiều thiết bị khác nhau.

Đối với thành phần trí tuệ nhân tạo, đồ án đã xây dựng AI Backend dựa trên FastAPI kết hợp kiến trúc Multi-Agent. Hệ thống sử dụng Supervisor Router để phân tích câu hỏi và lựa chọn Agent phù hợp cho từng nhiệm vụ cụ thể như tư vấn linh kiện, gợi ý combo sản phẩm, phân tích tồn kho và hỗ trợ kỹ thuật IoT. Bên cạnh đó, mô hình Hybrid RAG được triển khai bằng cách kết hợp BM25 Truy xuất và Vector Truy xuất trên cơ sở dữ liệu vector Qdrant nhằm nâng cao khả năng truy xuất tri thức và giảm hiện tượng sinh thông tin không chính xác của mô hình ngôn ngữ lớn.

Kết quả thực nghiệm cho thấy hệ thống có khả năng phản hồi hiệu quả các câu hỏi liên quan đến linh kiện điện tử, dự án IoT, tình trạng tồn kho và thông tin sản phẩm. Việc kết hợp giữa AI Agent và Hybrid RAG giúp cải thiện độ chính xác của câu trả lời, đồng thời tận dụng được nguồn tri thức nội bộ của hệ thống. Các chức năng quản lý dữ liệu, đồng bộ thời gian thực và giám sát thiết bị IoT đều hoạt động ổn định trong quá trình thử nghiệm.

Nhìn chung, các mục tiêu đề ra ban đầu của đồ án đã được hoàn thành. Hệ thống không chỉ đáp ứng yêu cầu quản lý và giám sát trong môi trường bán lẻ linh kiện điện tử mà còn tạo nền tảng cho việc phát triển các ứng dụng AIoT thông minh trong tương lai.

5.2. Hạn chế của đề tài

Mặc dù đạt được các mục tiêu đề ra, hệ thống vẫn còn một số hạn chế nhất định.

Thứ nhất, cơ sở tri thức hiện tại chủ yếu được xây dựng thủ công từ tập tài liệu nội bộ nên phạm vi kiến thức còn hạn chế. Khi xuất hiện các công nghệ hoặc linh kiện mới, hệ thống cần được cập nhật dữ liệu để đảm bảo chất lượng tư vấn.

Thứ hai, hệ thống AI hiện mới hỗ trợ xử lý ngôn ngữ tự nhiên dưới dạng văn bản và chưa hỗ trợ đầy đủ các chức năng đa phương thức như nhận dạng hình ảnh linh kiện, phân tích sơ đồ mạch điện hoặc xử lý tài liệu kỹ thuật tự động.

Thứ ba, cơ chế đánh giá và tối ưu câu trả lời của AI hiện vẫn phụ thuộc nhiều vào chất lượng dữ liệu trong Knowledge Base. Trong một số trường hợp đặc biệt, mô hình ngôn ngữ vẫn có thể tạo ra các phản hồi chưa hoàn toàn chính xác hoặc chưa đầy đủ.

Thứ tư, hệ thống mới được triển khai ở quy mô thử nghiệm nên chưa đánh giá đầy đủ hiệu năng khi số lượng người dùng đồng thời tăng cao hoặc khi khối lượng dữ liệu trong Vector Database tăng lên đáng kể.

5.3. Hướng phát triển

Mặc dù hệ thống AIoT Smart Retail đã đáp ứng được các mục tiêu đề ra, tuy nhiên vẫn còn nhiều tiềm năng để tiếp tục nghiên cứu, hoàn thiện và phát triển trong tương lai nhằm nâng cao khả năng ứng dụng thực tế cũng như giá trị thương mại của hệ thống.

Trước hết, cơ sở tri thức của hệ thống có thể được mở rộng thông qua việc tự động thu thập và cập nhật dữ liệu từ các nguồn tài liệu kỹ thuật chính hãng như datasheet, hướng dẫn sử dụng, tài liệu đào tạo và tài liệu kỹ thuật từ các nhà sản xuất linh kiện điện tử. Việc xây dựng cơ chế cập nhật tri thức tự động sẽ giúp hệ thống luôn duy trì được nguồn dữ liệu mới nhất mà không cần bổ sung thủ công như hiện nay.

Bên cạnh đó, hệ thống Hybrid RAG có thể được nâng cấp bằng cách tích hợp thêm các kỹ thuật truy xuất tiên tiến như Query Expansion, Query Rewriting, Context Compression, Reranking và Knowledge Graph. Các kỹ thuật này giúp cải thiện khả năng tìm kiếm thông tin, tăng độ chính xác của dữ liệu truy xuất và nâng cao chất lượng câu trả lời của mô hình ngôn ngữ lớn.

Đối với kiến trúc AI, hệ thống có thể được mở rộng từ mô hình Multi-Agent hiện tại sang kiến trúc Agentic AI hoàn chỉnh. Trong kiến trúc này, mỗi Agent sẽ đảm nhiệm một vai trò chuyên biệt như Agent tư vấn kỹ thuật IoT, Agent hỗ trợ thiết kế hệ thống nhúng, Agent

phân tích tồn kho, Agent dự báo nhu cầu sản phẩm, Agent chăm sóc khách hàng và Agent hỗ trợ quản trị hệ thống. Việc tăng số lượng Agent chuyên biệt sẽ giúp nâng cao chất lượng xử lý và khả năng giải quyết các yêu cầu phức tạp của người dùng.

Một hướng phát triển quan trọng khác là tích hợp các mô hình thị giác máy tính (Computer Vision). Hệ thống có thể hỗ trợ nhận dạng linh kiện điện tử thông qua hình ảnh, đọc mã QR, nhận dạng mã vạch, phân tích sơ đồ mạch điện hoặc tự động trích xuất thông tin từ datasheet. Điều này sẽ giúp người dùng dễ dàng tìm kiếm và tra cứu thông tin linh kiện mà không cần nhập dữ liệu thủ công.

Ngoài khả năng xử lý văn bản, hệ thống có thể được phát triển thành hệ thống AI đa phương thức (Multimodal AI), cho phép xử lý đồng thời văn bản, hình ảnh, tài liệu PDF và dữ liệu cảm biến IoT. Khi đó người dùng có thể tải lên hình ảnh linh kiện, sơ đồ nguyên lý hoặc tài liệu kỹ thuật để hệ thống phân tích và đưa ra tư vấn phù hợp.

Trong lĩnh vực quản lý kho, hệ thống có thể được bổ sung các mô hình học máy để dự báo nhu cầu sản phẩm, dự báo tốc độ tiêu thụ linh kiện và dự báo nguy cơ hết hàng. Các mô hình này có thể sử dụng dữ liệu lịch sử bán hàng để hỗ trợ quản trị viên đưa ra quyết định nhập hàng và tối ưu hóa chi phí lưu kho.

Đối với hệ thống IoT, có thể mở rộng thêm nhiều thiết bị thông minh như Smart Locker, hệ thống kiểm kê tự động bằng RFID, cảm biến môi trường kho hàng, camera giám sát thông minh và các thiết bị điều khiển từ xa. Khi đó hệ thống không chỉ đóng vai trò quản lý bán hàng mà còn trở thành nền tảng quản lý kho thông minh toàn diện.

Ngoài ra, hệ thống có thể được tích hợp với các nền tảng thương mại điện tử nhằm đồng bộ dữ liệu sản phẩm, đơn hàng và tồn kho theo thời gian thực. Việc kết nối với các nền tảng bán hàng trực tuyến sẽ giúp mở rộng phạm vi ứng dụng của hệ thống và tăng khả năng thương mại hóa sản phẩm.

Về mặt triển khai, trong tương lai hệ thống có thể được container hóa bằng Docker và triển khai trên các nền tảng điện toán đám mây như Google Cloud Platform, Amazon Web Services hoặc Microsoft Azure. Điều này giúp hệ thống dễ dàng mở rộng quy mô, tăng khả năng chịu tải và cho phép nhiều người dùng truy cập đồng thời thông qua Internet.

Một hướng phát triển quan trọng khác là xây dựng hệ thống phân quyền thông minh dựa trên vai trò người dùng. Mỗi nhóm người dùng như khách hàng, nhân viên bán hàng, quản

lý kho và quản trị viên sẽ được cung cấp các chức năng và quyền truy cập khác nhau nhằm nâng cao tính bảo mật và hiệu quả quản lý.

Hệ thống cũng có thể được mở rộng để hỗ trợ giao tiếp bằng giọng nói thông qua công nghệ Speech-to-Text và Text-to-Speech. Khi đó người dùng có thể tương tác với AI bằng giọng nói thay vì chỉ sử dụng văn bản, góp phần nâng cao trải nghiệm sử dụng và tăng tính thân thiện của hệ thống.

Trong giai đoạn phát triển tiếp theo, việc xây dựng trợ lý AI chuyên biệt cho lĩnh vực IoT và hệ thống nhúng là một hướng nghiên cứu có nhiều tiềm năng. Trợ lý AI này không chỉ hỗ trợ tư vấn linh kiện mà còn có khả năng hỗ trợ viết mã nguồn cho ESP32, Arduino, STM32, kiểm tra lỗi chương trình, gợi ý sơ đồ kết nối phần cứng và hướng dẫn triển khai các dự án IoT hoàn chỉnh.

Cuối cùng, mô hình có thể được phát triển theo hướng nền tảng AIoT Smart Retail hoàn chỉnh phục vụ cho hoạt động bán lẻ thông minh. Trong mô hình này, AI không chỉ đóng vai trò hỗ trợ tìm kiếm và tư vấn mà còn tham gia vào quá trình phân tích dữ liệu kinh doanh, dự báo doanh thu, tối ưu hóa tồn kho, hỗ trợ ra quyết định và tự động hóa các quy trình vận hành của doanh nghiệp.

TÀI LIỆU THAM KHẢO

- [1] Espressif Systems, ESP32 Series Datasheet. Shanghai, China: Espressif Systems, 2024.
- [2] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures. PhD thesis, University of California, Irvine, 2000.
- [3] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 3rd ed. Upper Saddle River, NJ, USA: Pearson Education, 2010.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” Advances in Neural Information Processing Systems (NeurIPS 2017), Long Beach, California, USA, Dec. 2017, pp. 5998–6008.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems (NeurIPS 2020), Dec. 2020, pp. 9459–9474.
- [6] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge, United Kingdom: Cambridge University Press, 2008.
- [7] M. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed. Chichester, United Kingdom: John Wiley & Sons, 2009.

DATN-Ngoc Hai - Bao Tran

ORIGINALITY REPORT

7 %	9 %	9 %	1 %
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	www.ctu.edu.vn Internet Source	4 %
2	Phenikaa University Publication	2 %
3	Vietnam National University of Forestry Publication	1 %
4	Ton Duc Thang University Publication	1 %

Exclude quotes On

Exclude bibliography On

Exclude matches < 1%